

INF01151 – Sistemas Operacionais II

Aula 18 - Algoritmos distribuídos & Sincronização de relógios físicos

Prof. Alberto Egon Schaeffer Filho

Introdução

- Processos em um sistema distribuído cooperam entre si
- Tempo é uma questão importante/interessante em sistemas distribuídos
 - Tempo é a quantidade que desejamos medir de forma precisa
 - Transação de e-Commerce envolve eventos no computador do vendedor e no computador do banco, importante uso de timestamps precisos
 - Algoritmos dependem da sincronização de relógios
 - Manter consistência de dados distribuídos, coordenar acesso a recursos distribuídos, ordenar mensagens e eliminar updates duplicadas
- **Problema**: Não existe relógio físico absoluto!!
 - Necessita métodos para sincronizar **aproximadamente** relógios
 - Usando troca de mensagens
 - Limite na precisão de timestamps de eventos em nodos diferentes
 - Ordem em que cada par de eventos aconteceu (ou se aconteceram simultaneamente)

Sincronização de relógios físicos: o problema

- O tempo não é ambíguo em sistemas centralizados
 - Para obter hora, os processos A e B fazem chamada de sistema
 - Se A executa antes de B, então $t_A < t_B$
 - Existe um relógio absoluto que pode determinar t_A e t_B
 - Porém, problema não é trivial em sistemas distribuídos
 - Cada computador tem o seu relógio
 - Não podem ser sincronizados perfeitamente
 - Sincronização aproximada
 - Relógios atrasam/adiantam

➤ **Aula de HOJE: relógios físicos**

➤ **Tópico da próxima aula: relógios lógicos** (podem ser usados para definir uma ordem de eventos sem medir o tempo físico)

Relógios, eventos e estados

- Ordenação de eventos que ocorrem em um único processo p_i
 - Processo p_i executa em um processador e não compartilha memória
 - Processo p_i tem um estado s_i , que inclui estado de suas variáveis
 - Processos só se comunicam por troca de mensagens
 - Execução de p_i consiste em série de ações:

- Operação `send` em p_i
 - Operação `receive` em p_i
 - Alteração do estado s_i
- } Eventos

ATENÇÃO: ordenamento $\rightarrow_i$ é bem definido mesmo que processo seja multithread, pois assumimos execução em um único processador

- Sequência de eventos em um único processo pode ser colocado em ordem total
 - Bem definida pois processo p_i executa em um único processador
- $e \rightarrow_i e'$: se e somente se e executa antes de e' em p_i

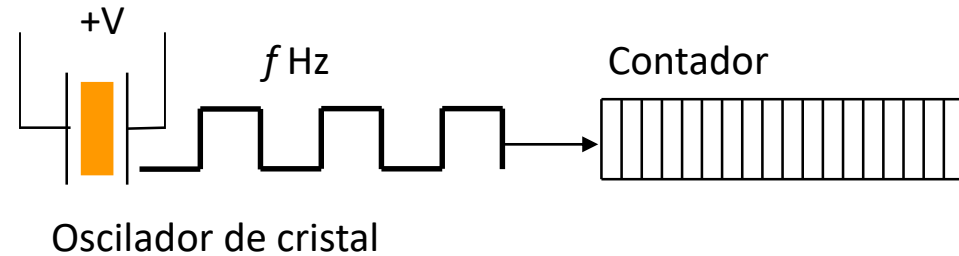
$\text{history}(p_i) = h_i = \langle e_i^0, e_i^1, e_i^2, \dots \rangle$

Relógios físicos

- Mas como atribuir timestamps?

- Cada computador tem seu próprio relógio físico

- Conta o número de oscilações em um cristal a determinada frequência, armazena em registrador



- Sistema operacional lê o valor do relógio físico, $H_i(t)$, escala e adiciona *offset* para produzir o *clock* de software $C_i(t)$

$$C_i(t) = \alpha H_i(t) + \beta$$

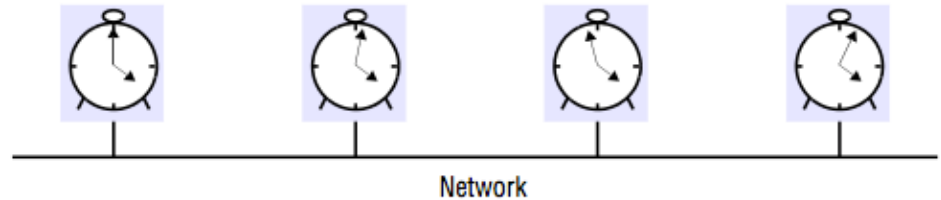
- $C_i(t)$ mede o tempo t real (físico) para processo p_i
 - E.g., número de 64-bits com número de milliseconds desde referência
 - Em geral, $C_i(t)$ será diferente de t por falta de precisão

Problemas: clock skew e clock drift

- Relógios físicos tendem a não concordarem perfeitamente...

- Sujeitos a **clock skew**

- Diferença instantânea entre leituras de dois relógios

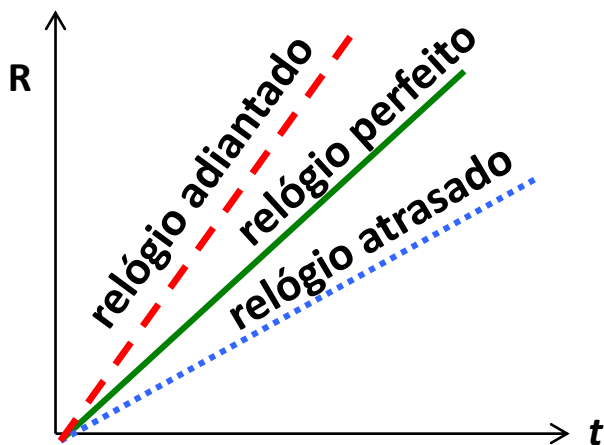


- Relógios baseados em cristal também sujeitos a **clock drift**

- Contam o tempo a taxas diferentes e, portanto, divergem
 - Osciladores estão sujeitos a variações físicas, mudanças de temperatura, etc
 - Possível compensar essa variação (α e β), mas não eliminá-la
 - Necessidade de acertos periódicos
- Relógios comuns baseados em quartz crystal
 - 10^{-6} segundos/segundos, diferença de 1 segundo a cada 1.000.000 segundos (11,6 dias)

Clock skew e clock drift

- Defasagem do relógio em relação a um relógio ideal (perfeito)
 - Relógio adiantado e relógio atrasado



Drift rate é a taxa de deslocamento entre um relógio local e um relógio de referência (ideal)

Diferença instantânea é a distorção (*skew*)

Fatores de correção

$$C_i(t) = \alpha H_i(t) + \beta$$

Relógio de hardware
(*clock ticks*)

Tempo Universal (curiosidades)

- Relógios de computadores podem ser sincronizados com fontes externas de alta precisão
- Relógio **atômico**
 - Relógios físicos mais precisos utilizam osciladores atômicos
 - Precisão de 10^{-13}
 - Standard second: 9.192.631.770 períodos de transição do átomo de Césio 133 (Cs^{133})
- Relógio **astronômico**
 - Base de tempo costumava ser o segundo solar médio
 - Definidos em termos da rotação da terra em torno de si mesmo, e de sua rotação ao redor do sol
 - Período de rotação da terra ao seu redor está se tornando maior
 - Efeito marés, efeitos atmosféricos, correntes de convecção, etc
 - Tempo astronômico e atômico tendem a sair de sincronismo
 - Segundo solar possui um drift em relação ao atômico...

Tempo Universal Coordenado (UTC)

- Coordinated Universal Time (UTC): padrão internacional
 - Baseado no tempo atômico
 - Ocasionalmente, um *leap second* → é inserido p/ mantê-lo sincronizado com tempo astronômico
- Sinais UTC são sincronizados e **broadcast** é feito regularmente
 - **Estações terrestres de rádio**
 - Sinais recebidos tem precisão de **0.1-10 milliseconds**
 - **Satélites (incluindo GPS)**
 - Sinais recebidos tem precisão de **1 microsecond**
- Computadores com receptores podem sincronizar seus relógios com esses sinais



Sincronização de relógios físicos

- Sincronização **externa**

- Ajustar os relógios C_i em relação a uma fonte externa $S(t)$
- Quando é necessário saber data/hora de eventos em sistema distribuído

$$|S(t) - C_i(t)| < D \text{ para } i = 1, 2, \dots, N$$

Relógios C_i “*são precisos*” dentro de um limite D

- Sincronização **interna**

- Ajustar os relógios em função dos relógios dos outros
- Objetivo é ter visão “igual” do tempo mesmo que difira do tempo real

$$|C_i(t) - C_j(t)| < D \text{ para } i, j = 1, 2, \dots, N$$

Relógios C_i “*concordam*” dentro de um limite D

Considerações gerais

- Relógios com sincronização interna não estão necessariamente sincronizados externamente
 - Podem sofrer **drift coletivo**, mesmo que concordem uns com os outros
- Porém, se sistema for sincronizado externamente com limite D
 - Também estará sincronizado internamente com um limite $2D$
- Noção de correção:
 - Relógios não dão saltos muito grandes:
 - Taxa de drift ($\rho > 0$) dentro de limite conhecido
 - Relógios não podem voltar no tempo:
 - Monotonicidade: $t' > t \Rightarrow C(t') > C(t)$
- Ajuste em software sem alteração do relógio físico: α e β

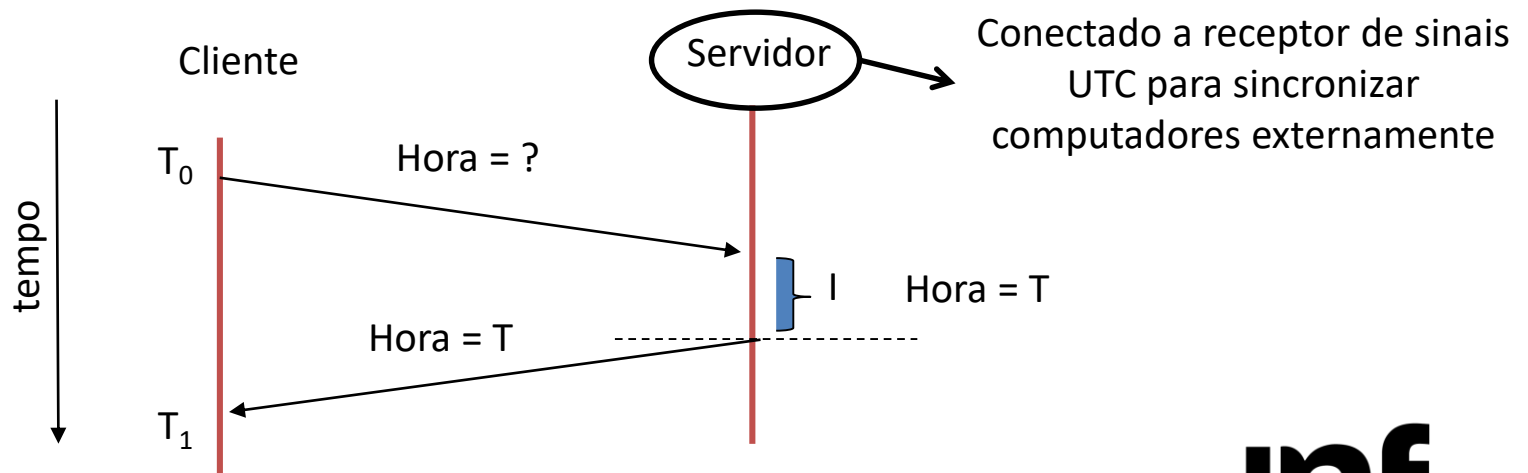
Importante: noção de correção não implica que o relógio deva marcar a hora exata!

Algoritmos de sincronização de relógios

- Modelo de falhas
 - Por colapso: quando pára de “tiquetaquear”
 - Bizantina (arbitrária): quando fornece valores errados
 - Year 2038: signed 32-bit integer representa número de segundos desde 00:00:00 de 1 janeiro 1970... contador irá “estourar” às 03:14:07 UTC de 19 janeiro 2038
 - Quebra da monotonicidade
 - Relógios com baterias fracas (*drift* se torna muito alto)
- Algoritmos de sincronização
 - Servidor passivo (Cristian, 1989)
 - Servidor ativo (Berkeley, 1989)
 - Network time protocol (NTP, 1985)

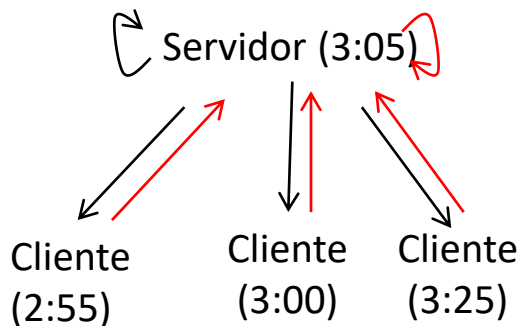
Algoritmo centralizado: servidor passivo (Cristian)

- Algoritmo de Cristian [1989]
 - Utiliza um servidor de “hora” para sincronizar computadores externamente
 - Atende requisições de “hora” fornecendo sua hora local (T)
 - Nova hora no cliente = $T + (T_1 - T_0)/2$
 - onde T_0 é a hora local da requisição e T_1 a hora local do recebimento da resposta

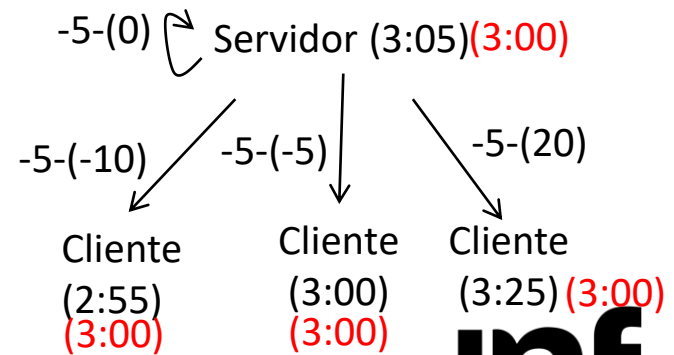


Algoritmo centralizado: servidor ativo (Berkeley)

- Algoritmo Berkeley [1989]
 - Objetivo é sincronização interna de um grupo de máquinas
 - Servidor – *master* – é escolhido por uma eleição (tolerância a falhas)
- Funcionamento
 - Periodicamente o servidor pergunta a hora aos clientes
 - Calcula média dos valores e informa diferença entre a média e os relógios locais
 - Envia a diferença a ser corrigida, não um valor de hora absoluto. Por que?

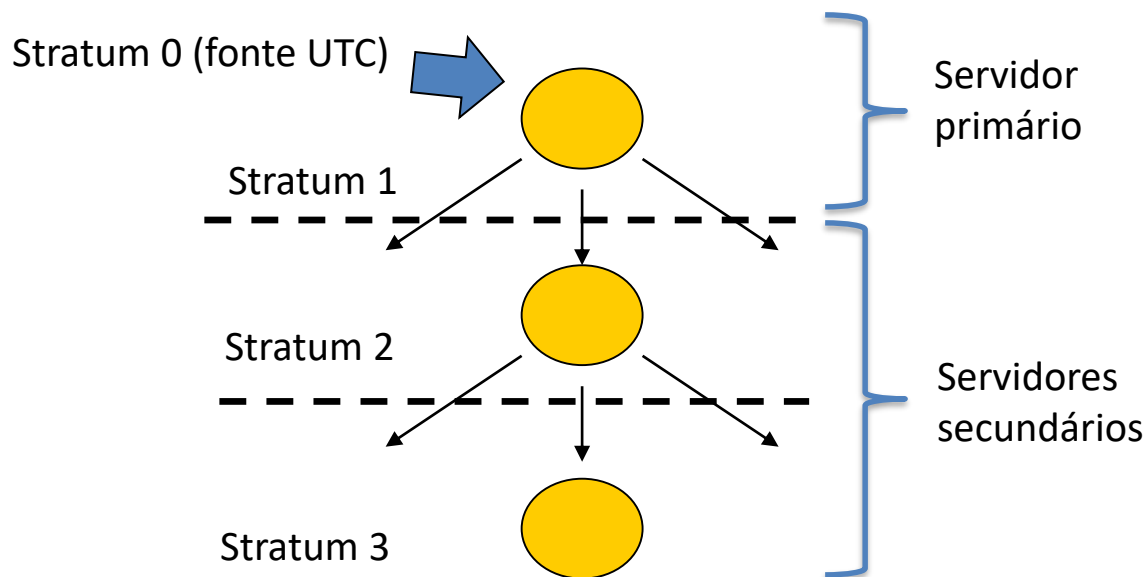


$$\begin{aligned} 3:05 - 3:05 &= 0 \\ 2:55 - 3:05 &= -10 \\ 3:00 - 3:05 &= -5 \\ \del{3:25 - 3:05 = +20} \end{aligned}$$
$$\text{média} = \frac{0 - 10 - 5}{3} = -5$$



Estudo de caso: Network Time Protocol (NTP)

- Projetado para sistemas distribuídos de grande porte (Internet)
 - Baseado em UDP (porta 123) e em operação desde 1985
 - Versão corrente: NTPv4 (RFC 5905)
- Organizado em uma estrutura hierárquica (*strata*)
 - Quanto mais próximo do topo, melhor a precisão



Objetivos do NTP

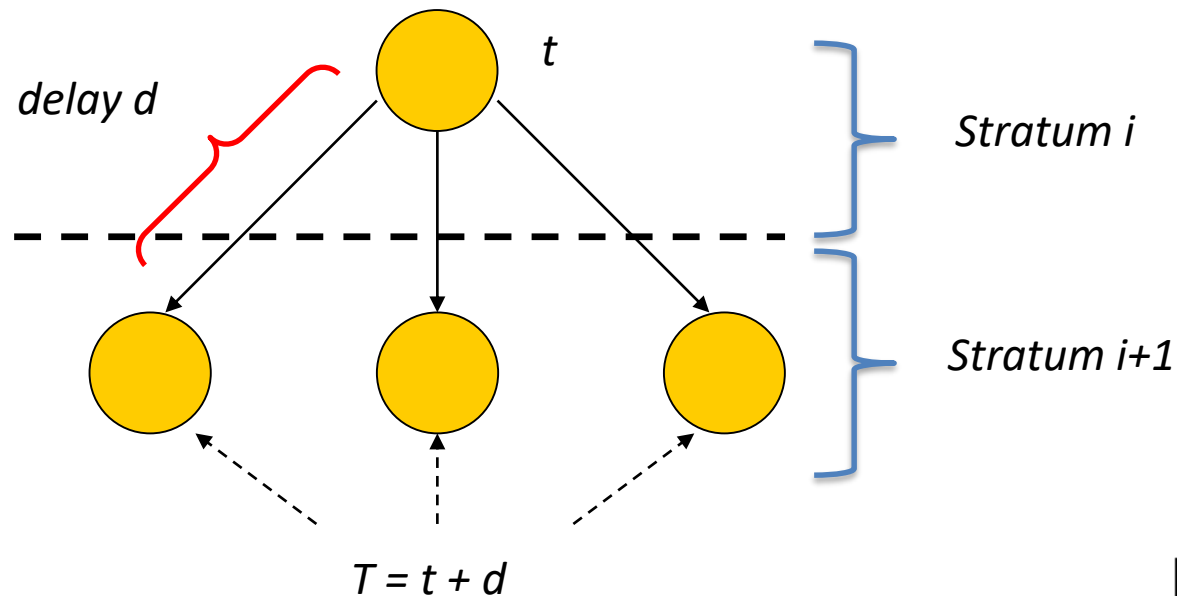
- ① Oferecer serviço para que clientes na Internet possam se sincronizar precisamente com base no UTC
 - Larga-escala, delay grande e variável pode ocorrer
 - Emprega técnicas estatísticas para filtrar atrasos e determina qualidade de dados
- ② Serviço confiável, deve continuar operacional mesmo na presença de falhas
 - Redundância de servidores e de rotas, reconfiguração de servidores
- ③ Permitir re-sincronização frequente de clientes
 - Estrutura hierárquica dos servidores aumenta escalabilidade
- ④ Proteção para evitar mal funcionamento em caso de interferências (acidentais ou propositais)
 - Autenticação e integridade das mensagens NTP

Funcionamento do NTP

- Baseado em três informações
 - **Clock offset**: diferença entre dois relógios (usado para ajuste)
 - **Round Trip delay**: RTT
 - **Dispersão**: máximo de erro permitido entre um relógio local e o relógio do servidor de nível superior
- Disponibiliza 3 modos de sincronização entre servidores NTP
 - Multicast
 - Cliente-servidor
 - Simétrico

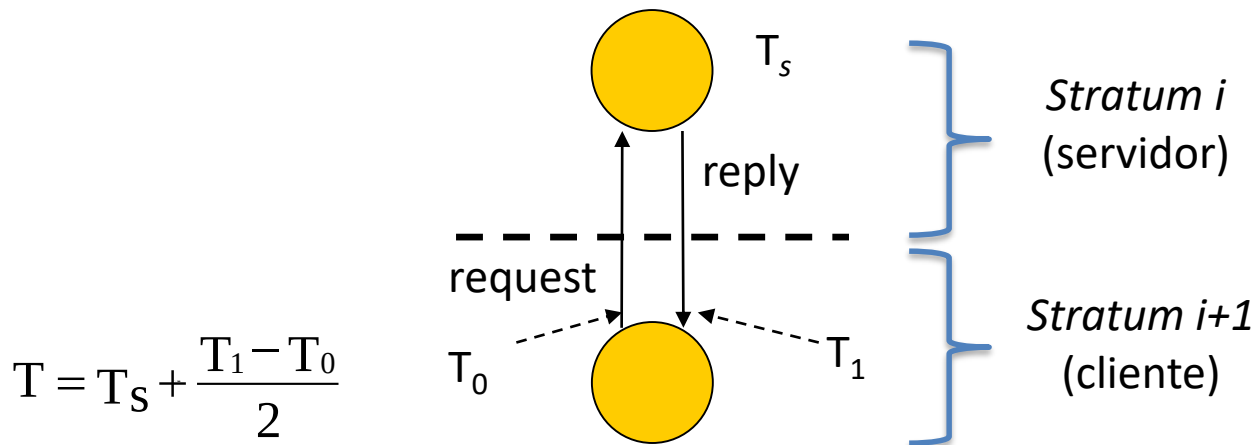
Modo multicast

- Usado em redes locais de alta velocidade
 - Rede precisa oferecer suporte multicast
- Um ou mais sevidores periodicamente fazem multicast de t
 - Receptores ajustam seus relógios assumindo um pequeno *delay*
 - Em geral, baixa precisão, devido à necessidade de se estimar d



Modo cliente-servidor

- Operação similar ao algoritmo de Cristian
 - Um servidor aceita requisição de outros servidores, e as responde com seu clock local
 - Em geral, melhor precisão que no modo multicast
- Alternativa quando não há multicast
 - Também normalmente usado em ambientes de redes locais

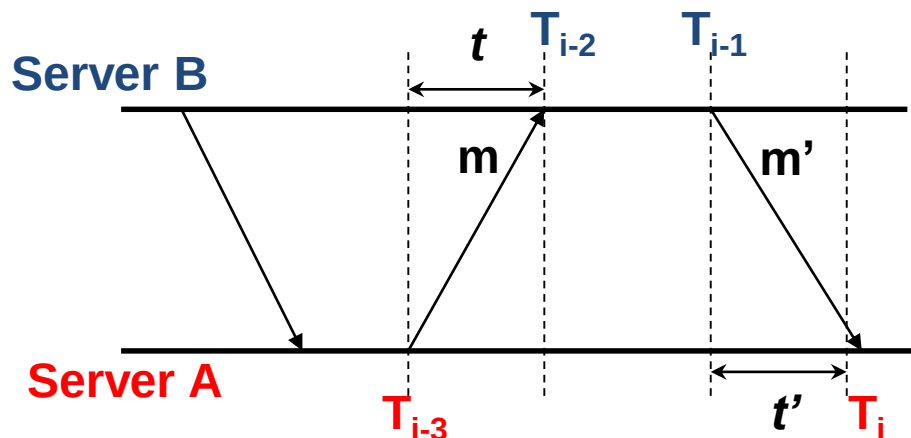


Modo simétrico

Informações adicionais:

- Utiliza filtragem de dados, e seleção de peers
- Precisão da sincronização
 - Na Internet: algumas dezenas de ms
 - Em LANs: 1ms

- Utilizado quando maior precisão é necessária
 - Níveis mais altos na hierarquia (com número de stratum baixo)
 - Associação entre par de servidores
 - Dados de tempo são retidos para melhorar a precisão com o tempo
- Cada mensagem possui timestamps de eventos de msgs recentes:
 - Tempo local do momento da emissão da mensagem anterior
 - Tempo local do momento da recepção da mensagem anterior
 - Tempo local do momento da emissão da mensagem corrente



Para cada par de mensagens m e m' , NTP calcula:

o_i = offset estimado de B em relação a A

d_i = delay no envio das msgs (t e t')

$$T_{i-2} = T_{i-3} + t + o \quad \text{e} \quad T_i = T_{i-1} + t' - o$$

$$d_i = t + t' = T_{i-2} - T_{i-3} + T_i - T_{i-1}$$

$$o = o_i + (t' - t)/2, \quad \text{onde} \quad o_i = (T_{i-2} - T_{i-3} + T_{i-1} - T_i)/2$$

Leituras adicionais

- Coulouris, G.; Dollimore, J.; Kindberg, T. and Blair, G.—
“*Distributed Systems: Concepts and Design*” (5th edition),
Addison Wesley, 2012
 - Capítulo 14 (seções 14.1, 14.2 e 14.3)

INF01151 – Sistemas Operacionais II

Aula 20 - Relógios lógicos & Algoritmos de eleição

Prof. Alberto Egon Schaeffer Filho

Introdução

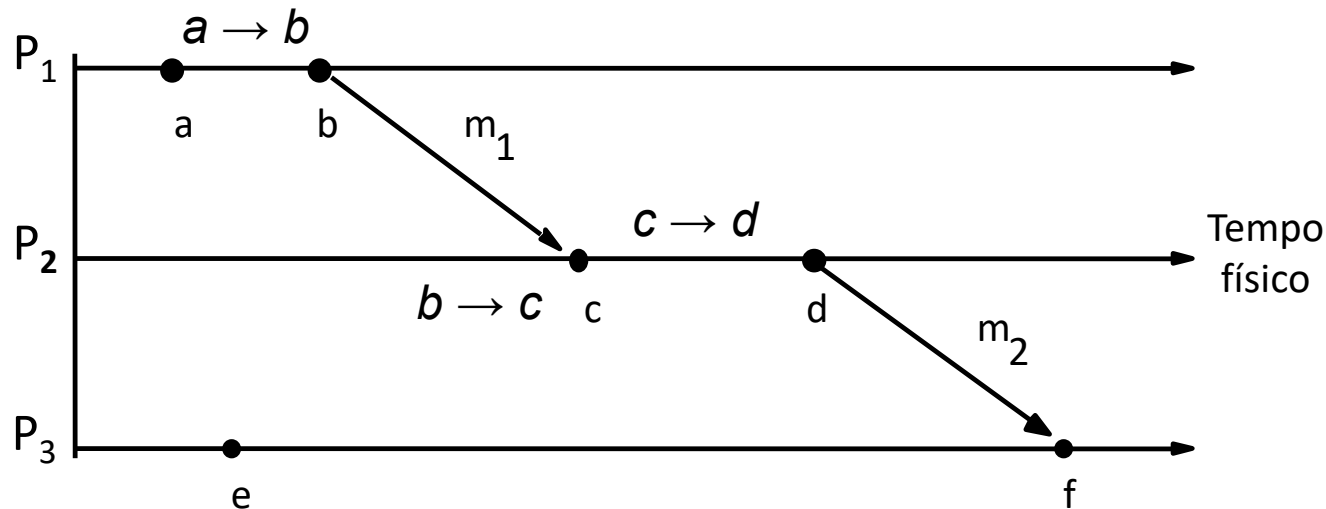
- Como visto na aula passada...
 - Em um único processador, eventos são ordenados pelo relógio local
 - Impossível sincronizar perfeitamente relógios em um sistema distribuído
 - Nem sempre se pode usar o tempo físico para ordenar pares de eventos!
- Solução é utilizar abordagem (intuitiva) de causa-consequência
 - Se dois eventos acontecerem no mesmo processo P_i , então eles ocorreram na ordem que o processo as observou ($\rightarrow_i$)
 - Se uma mensagem é enviada entre processos, o evento de envio da mensagem ocorreu antes do evento de recepção da mensagem
- Ordenação parcial de eventos pode ser obtida através da generalização desses dois pontos acima
 - Relação *happened-before* (by Leslie Lamport)

Notação e definições

- Notação
 - $e_1 \rightarrow_i e_2$
 - Indica que o evento e_1 precede o evento e_2 , no processo P_i
 - $e_1 \rightarrow e_2$
 - Indica que o evento e_1 precede o evento e_2
- Ordenação de eventos
 - Organizar os eventos em uma sequência de tal forma que um preceda o outro
 - Ordem *total*: todos os eventos do sistema estão ordenados
 - Ordem *parcial*: alguns eventos do sistema estão ordenados

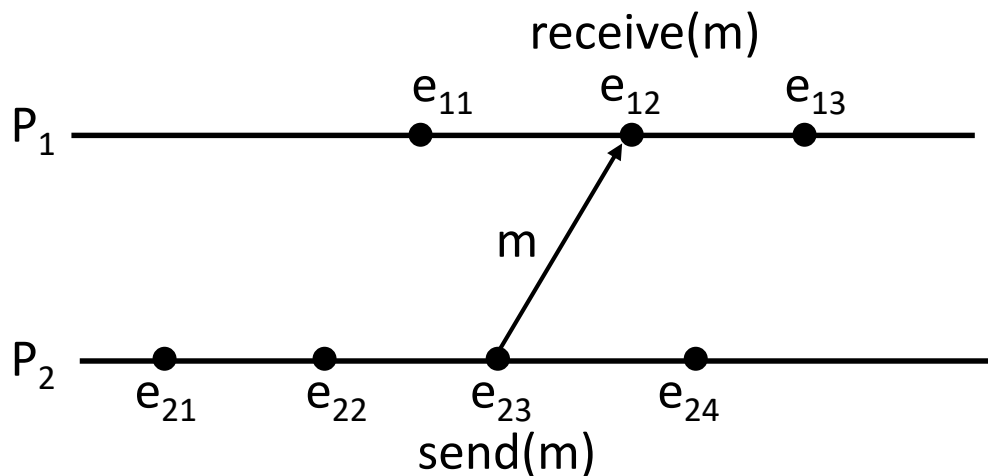
Relação *happened-before*

- Definição da relação *happened-before* ($\rightarrow$)
 - HB1**: se $\exists$ processo P_i : $e \rightarrow_i e'$, então $e \rightarrow e'$
 - HB2**: para qualquer mensagem m , $send(m) \rightarrow receive(m)$
 - HB3**: se e, e' e e'' são eventos tais que $e \rightarrow e'$ e $e' \rightarrow e''$, então $e \rightarrow e''$



Eventos concorrentes: $a \parallel e$ ou seja, não há relação $\rightarrow$ entre eles

Relação *happened-before*



$e_{23} \checkmark \rightarrow e_{12}$ $e_{22} \checkmark \rightarrow e_{12}$ $e_{21} \checkmark \rightarrow e_{12}$ $e_{22} \checkmark \rightarrow e_{13}$ $e_{21} \checkmark \rightarrow e_{13}$

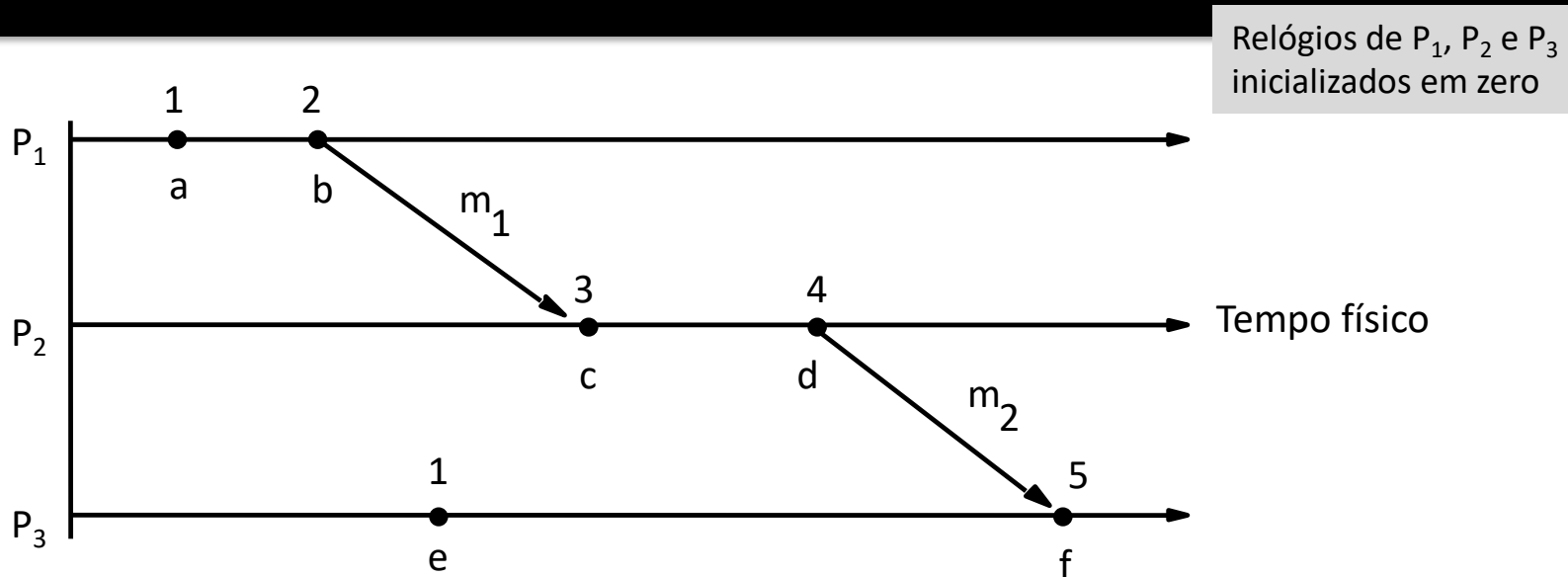
$e_{11} \times \rightarrow e_{21}$ $e_{11} \times \rightarrow e_{22}$ $e_{11} \times \rightarrow e_{23}$ $e_{11} \times \rightarrow e_{24}$

e_{11} é concorrente com e_{21} , e_{22} , e_{23} , e_{24}

Relógio lógico de Lamport

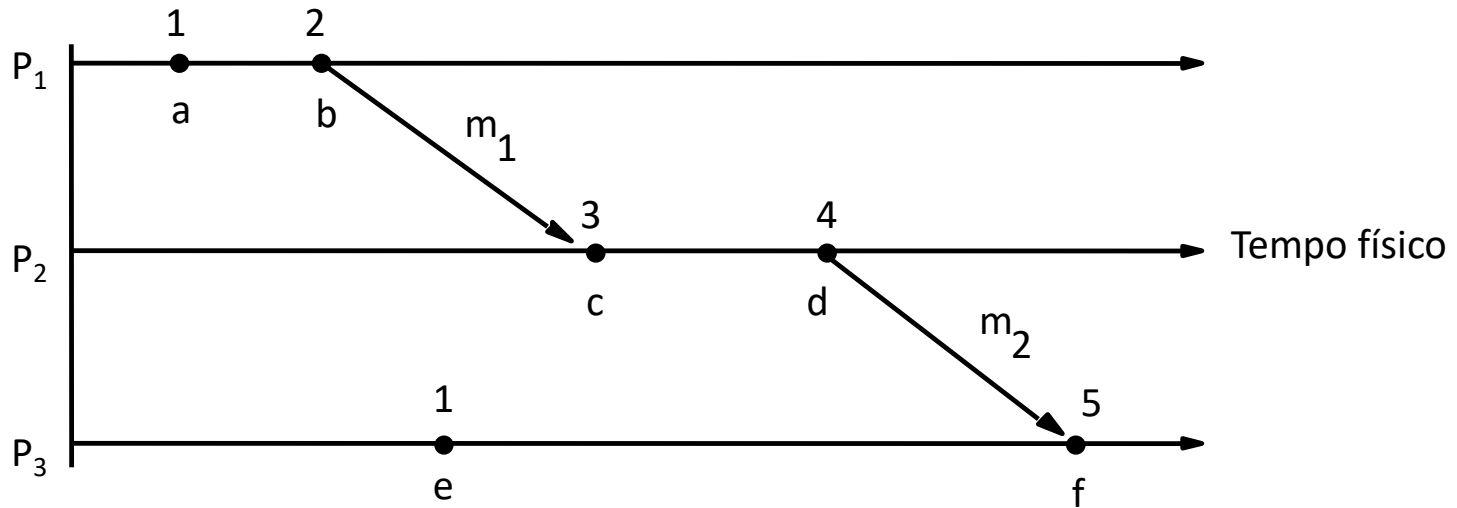
- Forma de “capturar” numericamente relação *happened-before*
 - Contador em software que aumenta a contagem monotonicamente
 - Valor do contador não tem qualquer relação com o relógio físico
- Condições:
 - L_i é o relógio lógico no processo P_i
 - Se $a \rightarrow b$ no processo P_i , $L_i(a) < L_i(b)$
 - Se a é o envio da mensagem m em P_i e b é a recepção da mensagem m em P_j ; então $L_i(a) < L_j(b)$
- Regras para atualização de relógios:
 - **LC1:** L_i é incrementado antes de cada evento ser processado em P_i : $L_i := L_i + 1$
 - **LC2:** **(a)** quando P_i envia mensagem m , ele envia junto valor $t = L_i$
(b) quando P_j recebe (m, t) , ele calcula $L_j := \max(L_j, t)$ e então aplica LC1 para definir o timestamp de $\text{receive}(m)$

Relógio lógico de Lamport: exemplo



- Como obter ordenação total? Eventos distintos em diferentes processos podem ter a mesma indicação de tempo lógico
- Solução é criar uma ordem global (usando algum critério)
 - Incluir identificadores dos processos onde os eventos aconteceram
 - $(T_i, i) < (T_j, j)$, se e somente se $(T_i < T_j)$ ou $(T_i = T_j \text{ e } i < j)$
 - Sem relação temporal real, mas fornece um critério de ordem...

Relógio lógico de Lamport: deficiência

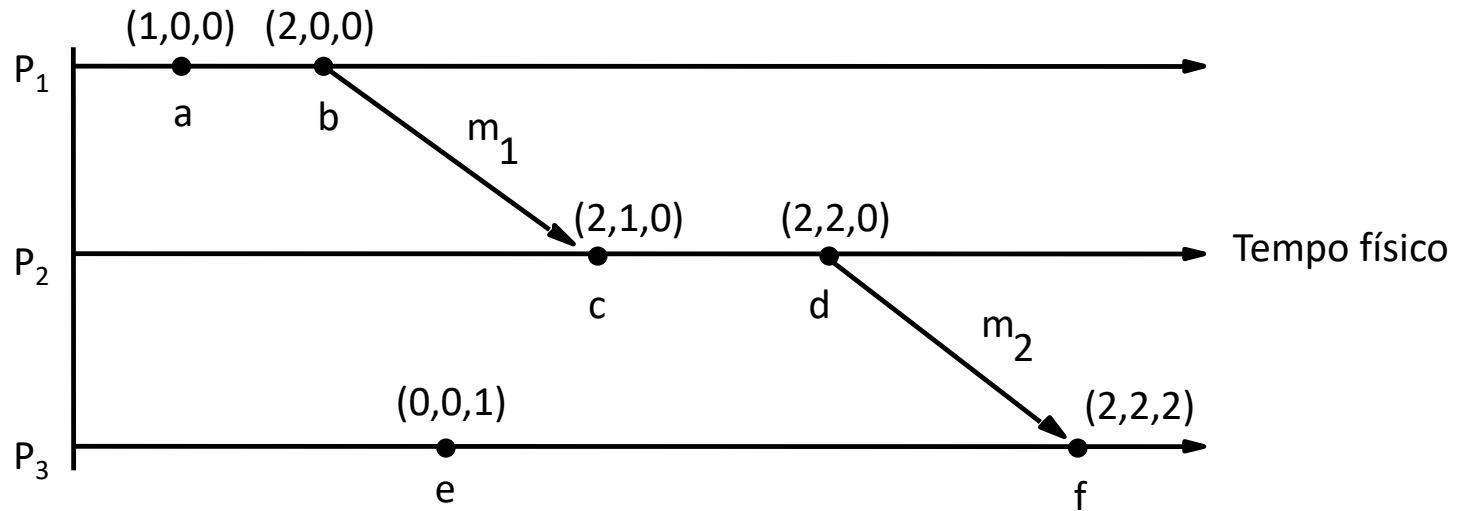


- Deficiência:
 - se $e_i \rightarrow e_j$ então $L(e_i) < L(e_j)$, **MAS o contrário não é verdadeiro**, ou seja, e_i pode não preceder e_j mesmo que $L(e_i) < L(e_j)$
 - Exemplo: $L(e) < L(b)$, mas $e \parallel b$
 - Acontece sempre quando dois processos x e y não tiverem seus relógios lógicos sincronizados
 - Falta de mensagens entre x e y
 - Mais eventos ocorrendo em um processo que em outro

Relógios vetoriais (Mattern e Fidge)

- Proposta para suprir deficiência do relógio de Lamport
 - O fato de que a partir de $L(e) < L(e')$ não concluir $e \rightarrow e'$
- Relógio vetorial para cada um dos N processos é um vetor de N inteiros
 - Cada processo P_i mantém seu relógio vetorial V_i , usado para fazer timestamp de eventos locais
- Regras para atualização de relógios:
 - **VC1**: inicialmente, $V_i[j] = 0$, para $i, j = 1, 2, \dots, N$;
 - **VC2**: imediatamente antes de P_i processar um evento, ele marca o timestamp atualizando $V_i[i] = V_i[i] + 1$
 - **VC3**: P_i inclui o valor $t = V_i$ em cada mensagem que envia
 - **VC4**: quando P_i recebe timestamp t em uma mensagem, ele atualiza $V_i[j] = \max(V_i[j], t[j])$, para $j = 1, 2, \dots, N$, e aplica VC2

Relógios vetoriais: exemplo



- Comparação de relógios vetoriais
 - $V = V'$ se e somente se $V[j] = V'[j]$, para todo $j = 1, 2, \dots, N$
 - $V \leq V'$ se e somente se $V[j] \leq V'[j]$, para todo $j = 1, 2, \dots, N$
 - $V < V'$ se e somente se $V \leq V'$ e $V \neq V'$

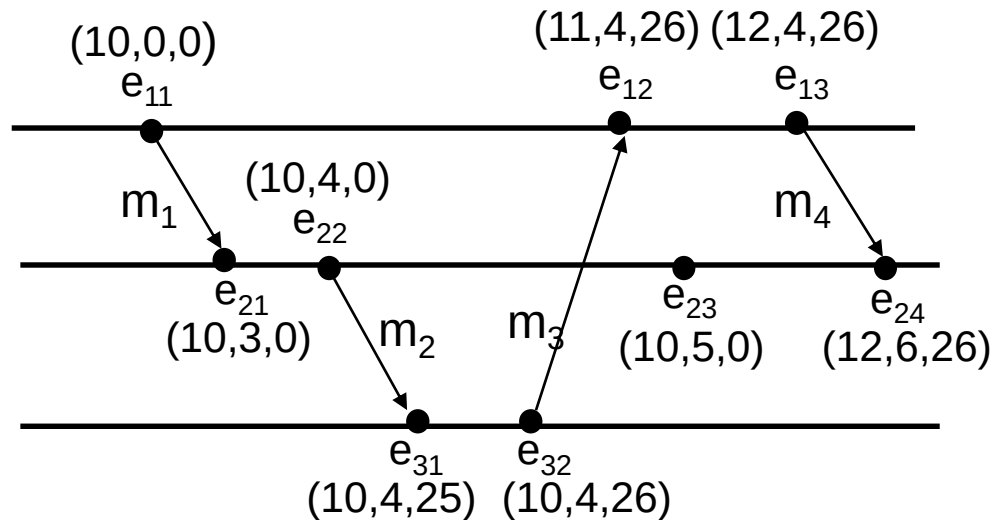
Relógios vetoriais: exemplo

Valores iniciais
(exemplo)

$L_1=(9,0,0)$ P_1

$L_2=(0,2,0)$ P_2

$L_3=(0,0,24)$ P_3



maior
 $e_{23} (10, 5, 0)$
 $e_{32} (10, 4, 26)$
 igual
 maior
 Logo, são concorrentes!

- Relógio vetorial também fornece apenas ordenamento parcial
 - e_{23} vem antes ou depois de e_{32} ?

Indo adiante em sistemas distribuídos...

- Como em um sistema centralizado, também é necessário efetuar atividades de controle
 - Exclusão mútua
 - Tratamento de deadlock
 - Detecção de término
- Realizadas por algoritmos de controle (para sistemas distribuídos)
 - Podem usar um processo coordenador ou serem totalmente distribuídos (sem coordenador)
 - Coordenador é eleito entre vários processos
 - >> Algoritmos de eleição

Algoritmos de eleição

- Algoritmos para escolher um processo único para desempenhar algum papel em particular (e.g., coordenador, time-server, etc)
 - Sempre que houver desconfiança da falha do coordenador
 - Necessário que todos os processos concordem na escolha
- Definições:
 - Processo inicia eleição quando ele detecta que coordenador atual falhou
 - Um processo não pode iniciar mais de uma eleição ao mesmo tempo
 - Porém, N processos poderiam iniciar N eleições ao mesmo tempo
 - Em um dado momento processo P_i pode ser participante ou não-participante
- Processo eleito deve ser único, e deve ser aquele com maior **id**
 - **id** = Qualquer valor “útil”, desde que seja único e totalmente ordenado
 - E.g., eleger processo com a menor carga: $id = \langle 1/load, i \rangle$
 - E.g., eleger processo com maior prioridade ou maior pid, etc

Algoritmos de eleição

- Algoritmos clássicos (para sistemas distribuídos):
 - Algoritmo do anel
 - Algoritmo valentão (bully)
- Em ambos
 - Cada processo P_i ($i = 1, 2, \dots, N$) tem variável $elected_i$
 - Quando P_i se torna participante, ele seta $elected_i = \perp$
 - Ao final, deve satisfazer duas propriedades
 - Liveness
 - Algoritmo termina quando todos os processos tem $elected_i \neq \perp$, ou
 - Sofreram colapso
 - Safety
 - Ao final, todos os processos que não sofreram colapso setaram $elected_i = P$, onde P é o processo com maior id e que não tenha sofrido colapso

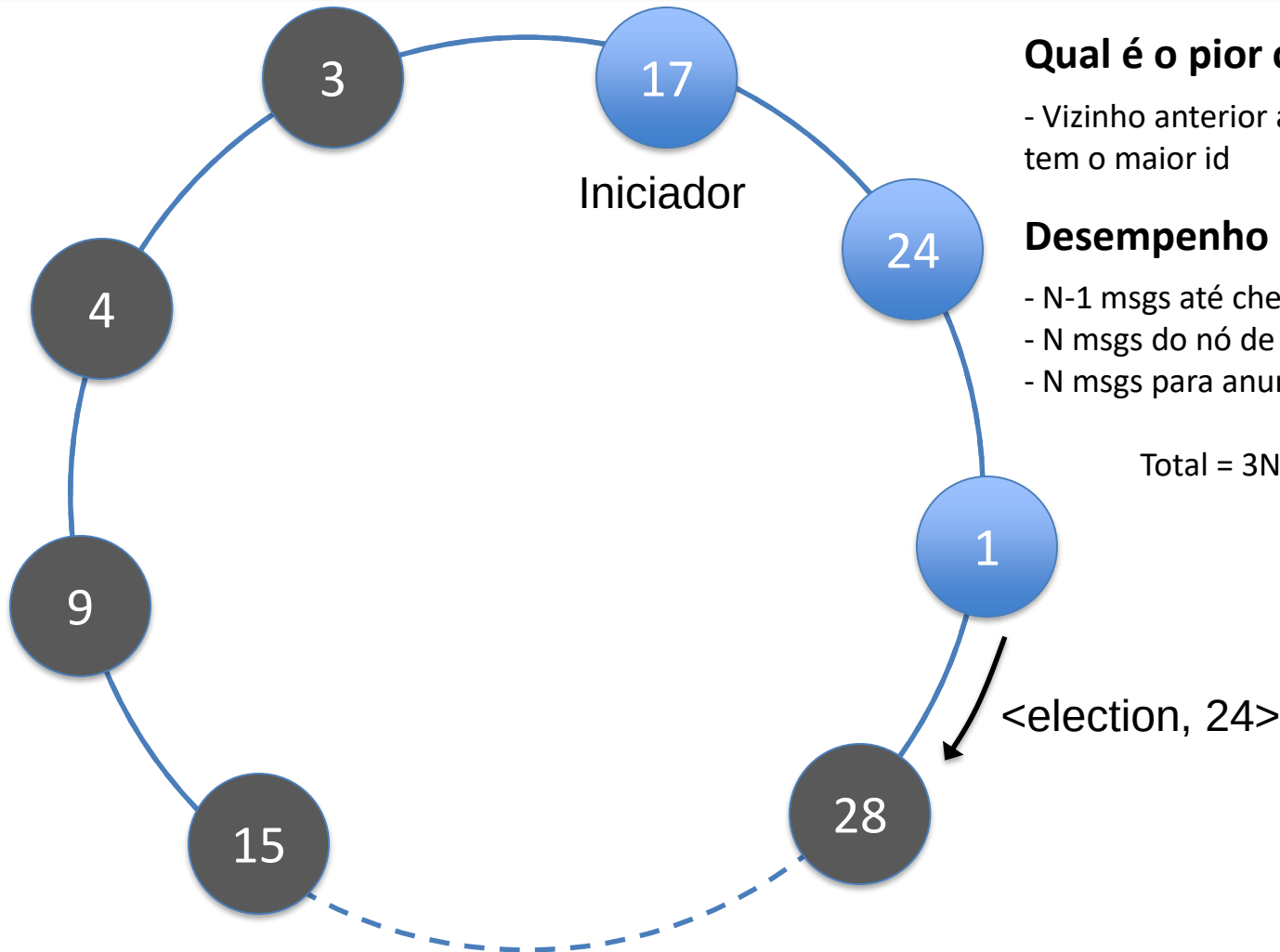
Algoritmo do anel [Chang & Roberts, 1979]

- Processo são organizados em um anel lógico (unidirecional)
 - Cada processo P_i se comunica apenas com o próximo processo $P_{(i+1) \bmod N}$
 - Mensagens enviadas no sentido horário, não tolera falhas
- Funcionamento:
 - Qualquer P_i pode iniciar eleição: marca P_i como participante, e envia mensagem m com seu id para o sucessor no anel: $m = \langle \text{election}, \text{id} \rangle$
 - Quando algum processo P_j recebe $m = \langle \text{election}, \text{id} \rangle$:
 - Compara id na mensagem com seu próprio
 - Se id na mensagem for maior, repassa a mensagem para o próximo vizinho
 - Se id na mensagem for menor e processo não-participante, substitui id e repassa
 - Porém, P_j não repassa se este já for participante
 - Se algum processo recebe na mensagem seu próprio id, então é o maior
 - Coordenador: processo se marca como não-participante, e envia mensagem $m = \langle \text{elected}, \text{id} \rangle$ para o vizinho (repassada até fazer outra volta)

P_j também se
marca como
participante



Algoritmo do anel [Chang & Roberts, 1979]



Qual é o pior caso?

- Vizinho anterior ao iniciador é aquele que tem o maior id

Desempenho no pior caso?

- $N-1$ msgs até chegar ao nó de maior id
- N msgs do nó de maior id p/ chegar nele mesmo
- N msgs para anunciar que é o coordenador

Total = $3N-1$ mensagens

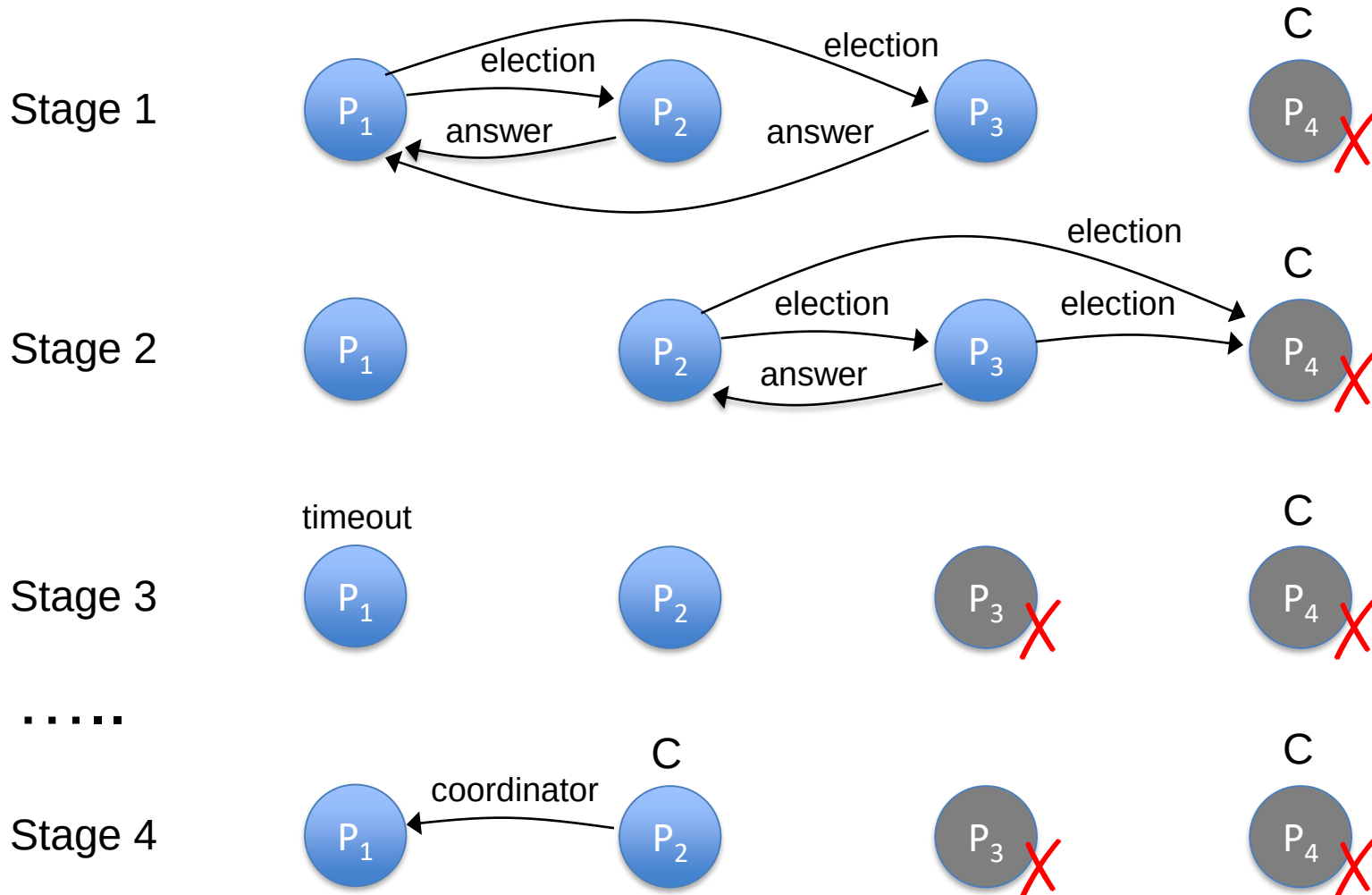
Algoritmo *bully* (valentão)

- Proposto por Garcia-Molina, 1982
- Diferenças em relação ao algoritmo do anel:
 - Permite falha de processos, mas assume entrega confiável de mensagens
 - Assume que sistema é síncrono (utiliza timeouts p/ detectar falhas)
 - Assume que cada processo sabe quais outros processos tem id maior
- Tipos de mensagens
 - `election`: usada para iniciar uma eleição
 - `answer`: usada em resposta a uma mensagem do tipo `election`
 - `coordinator`: usada para anunciar o id do processo eleito
- Eleição começa quando P_i determina que coordenador falhou
 - Síncrono: T_{trans} (max. transmissão), e $T_{process}$ (max. processamento)
 - Detector de falhas: usa timeout $T = 2T_{trans} + T_{process}$

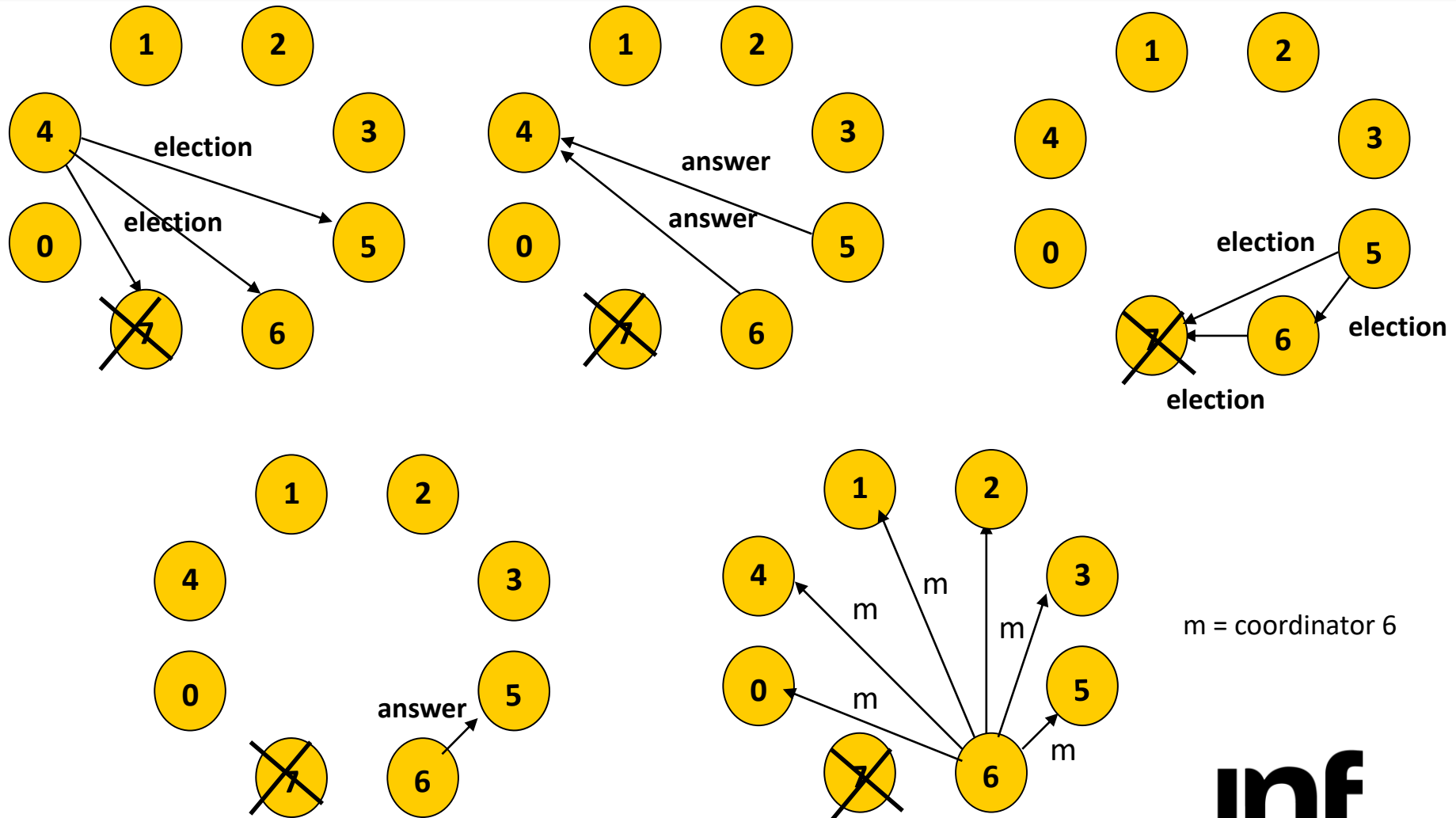
Algoritmo *bully* (valentão)

- Funcionamento:
 - P_i não obtendo resposta do coordenador envia *mensagem* `election` para todos *processos com maior id que o seu*
 - Se não receber nenhuma mensagem do tipo `answer` dentro de um *time-out* T , P_i se auto-define como novo coordenador
 - Se processo P_i recebe mensagem do tipo `answer`, isso significa que tem um processo com id superior
 - Espera por mais T' para receber mensagem `coordinator`. **E se não chegar?**
 - Processo P_j que recebe uma mensagem `election`, envia `answer` p/ emissor, e inicia sua própria eleição – a menos que já tenha feito isso!
 - Processo vencedor envia uma mensagem `coordinator` todos os outros processos (obviamente, eles terão id menor)
- Na recuperação de um processo P_k ele reinicia a eleição
 - Se possui maior id ele se tornará o coordenador e anunciará
 - Mesmo que o coordenador atual esteja ok → Bully!!!

Algoritmo *bully* (valentão)



Exemplo: Algoritmo *bully* (valentão)



Leituras adicionais

- Coulouris, G.; Dollimore, J.; Kindberg, T. and Blair, G.—
“Distributed Systems: Concepts and Design” (5th edition),
Addison Wesley, 2012
 - Capítulo 14 (seção 14.4)
 - Capítulo 15 (seção 15.3)
- Lamport, L. *“Time, clocks and the ordering of events in a distributed system”*. Communications of the ACM, Vol. 21, No. 7, pp. 558-65, 1978.

INF01151 – Sistemas Operacionais II

Aula 21 - Comunicação em grupo

Prof. Alberto Egon Schaeffer Filho

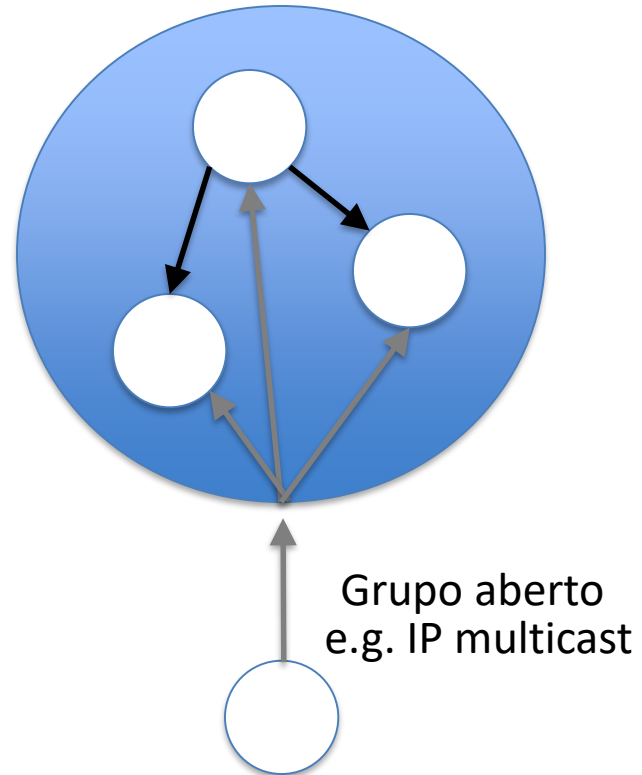
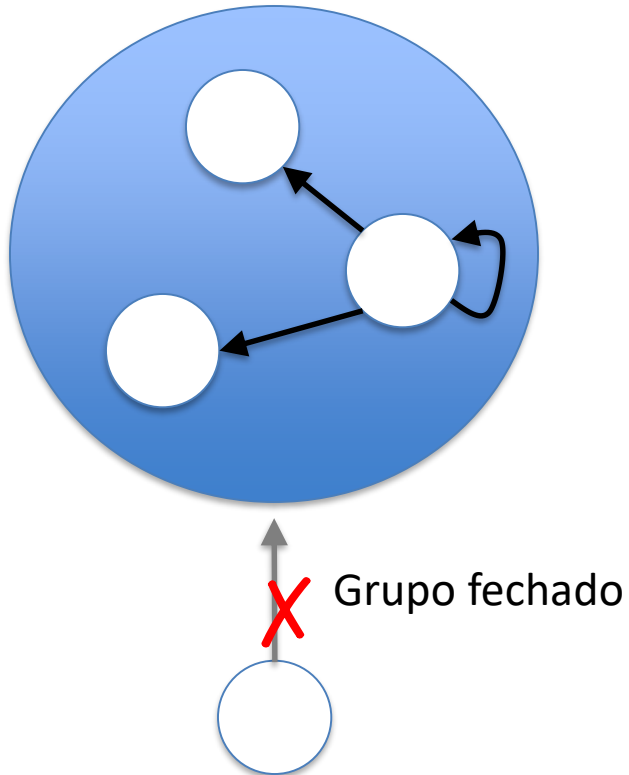
Introdução

- Forma elementar de comunicação entre processos é um-para-um
 - Um único emissor para um único receptor
 - Também denominada de ponto-a-ponto ou *unicast*
- Em muitos casos, comunicação em grupo é mais apropriada
 - Modelo de comunicação indireta
 - Mensagem enviada para o grupo deve ser entregue a todos os membros
 - Membership geralmente é transparente
- Algumas situações exigem comunicação entre grupo de processos
 1. Tolerância a falhas utilizando servidores replicados
 2. Localização de serviços
 3. Propagação de notificação de eventos

Comunicação em grupo

- Comunicação *um-para-muitos* (ou *multicast*)
 - Se processo é membro do grupo, ele pode receber mensagens enviadas para aquele grupo
 - Muito útil para encontrar recursos e divulgar informações
- Noção de grupo
 - Conjunto de processos que recebem as informações
 - Podem ser:
 - **Fechado**: apenas membros do grupo podem enviar mensagens
 - **Aberto**: qualquer processo pode enviar mensagens para o grupo
 - Questão de gerência
 - **Criação do grupo**: grupos permanentes existem mesmo quando não há membros
 - **Membership**: grupos tipicamente são dinâmicos, processos podem entrar e sair
 - **Overlapping ou não**: processos podem participar de número arbitrário de grupos

Grupo fechado vs. grupo aberto



Pontos genéricos de implementação

- Forma de implementar o *multicast*
 - Mensagem é enviada ao grupo, e entregue a todos os membros do grupo
 - Endereço *multicast* para identificar processos destino (ou melhor, grupo)
 - Emissor envia uma única mensagem para o grupo (ao invés de n operações de send)
 - Não somente uma conveniência para o programador, mas também mais eficiente
- *IP multicast* é um mecanismo de comunicação assíncrono
 - Em sua forma mais simples, não há como o emissor esperar por todos destinos estarem prontos para receber a mensagem
 - Emissor pode desconhecer quantos membros tem no grupo
 - Lado receptor:
 - Não bufferizado: destino perde o *multicast* se não está pronto para receber
 - Bufferizado: o destino receberá o *multicast* quando estiver pronto
- Outros aspectos: Ordenamento? Confiável?
 - Serão vistos na aula de hoje....

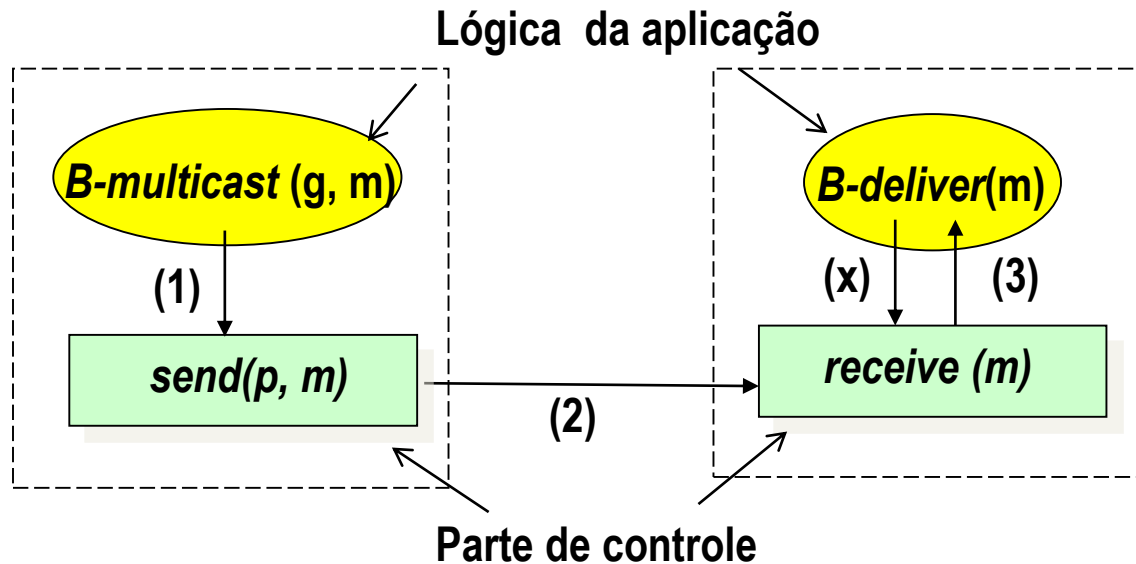
Confiabilidade em *multicast*

- Confiabilidade definida em termos de:
 - **Integridade**: mensagem recebida é a mesma que a enviada, s/ duplicações
 - **Validade**: mensagem enviada será eventualmente entregue
 - **Concordância**: se mensagem é entregue a um processo, então é entregue a todos do grupo
 - Propriedade “tudo ou nada”
 - Ou todos os processos do grupo recebem o *multicast* ou é como se nenhum tivesse recebido
 - Problema quando se prevê falhas emissor, receptor ou na rede
- Confirmações de entrega da mensagem *multicast*
 - **0-confiável**: emissor não espera nenhuma confirmação (*e.g.*, *IP multicast*)
 - **1-confiável**: emissor recebe a confirmação de um destino
 - **m-de-n confiável**: emissor espera a resposta de *m* processos de um grupo formado por *n* processos ($1 < m < n$)
 - **Confiável**: emissor espera confirmação de todos membros

Multicast básico

- Primitiva `multicast(g, m)` que garante que a mensagem será entregue ao destino
- Primitivas:
 - `B-multicast(g, m)`: para cada processo $p \in g$, `send(p, m)`
 - `B-deliver(m)`: *em* receive entrega m para $p \in g$

Confiável?



Multicast confiável sobre IP multicast

- Propriedades de multicast confiável
 - **Integridade**: processo correto $p \in g$ entrega mensagem m no máximo uma vez, tendo m sido enviada usando operação $multicast(g,m)$
 - **Validade**: se processo correto faz $multicast(g,m)$, então processo eventualmente entregará m (self-delivery)
 - **Concordância**: se algum processo correto entrega m , então todos os processos corretos $p \in g$ entregarão m em algum momento
 - *não é suportada pelo B-multicast do slide anterior
 - Relacionado à atomicidade (ou tudo ou nada)
- Multicast confiável pode ser implementado sobre IP multicast
 - **Assumption**: grupos fechados, e sem overlapping

Multicast confiável sobre IP multicast

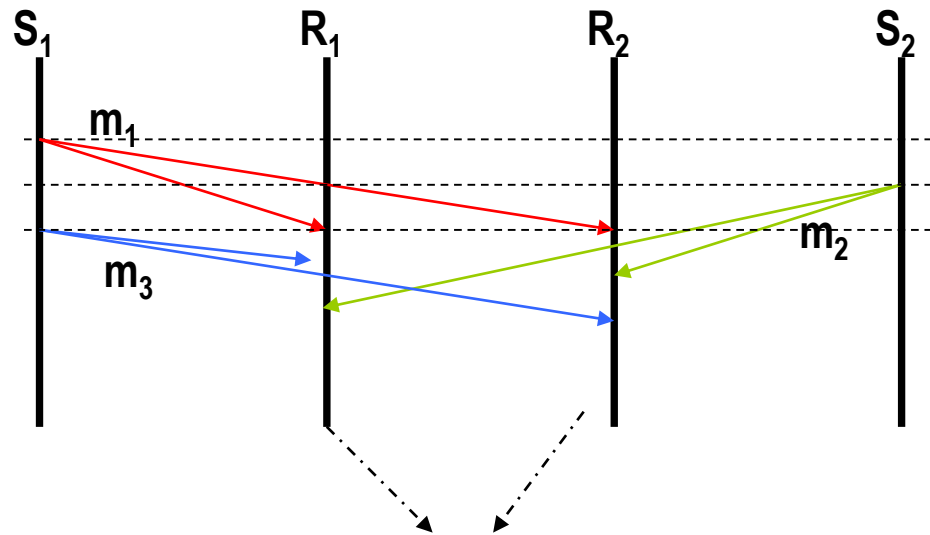
- Tipos de mensagens
 - **Piggybacked acks**: acks enviados junto com mensagem a ser transmitida
 - **Negative acks**: indicam a falta de uma mensagem esperada
- Cada processo $p \in g$:
 - S_g^p : número de mensagens enviadas por p
 - $\{R_g^q\}$: número de sequência da última mensagem de q para grupo g
- Quando p faz $R\text{-multicast}(g, m)$:
 - Envia junto S_g^p e acks $\langle q, R_g^q \rangle$ e depois incrementa S_g^p
 - Ack indicará o número de sequência da última mensagem enviada por q (vista por p)
- Quando processo recebe mensagem de q com número seq. S :
 - Se $S = R_g^q + 1$, processo faz $R\text{-deliver}(g, m)$
 - Se $S \leq R_g^q$, processo já entregou essa mensagem, então descarta
 - Se $S > R_g^q + 1$, ou se $ack(q) > R_g^q$, guarda m para depois ou solicita

Entrega ordenada de mensagens

- Multicast básico entrega mensagens em uma ordem arbitrária
 - Adequado para algumas aplicações: e.g., notificações no Facebook
 - Não satisfatório para muitas aplicações: e.g., eventos em usina nuclear
- Multicast ordenado
 - Todas as mensagens são entregues para todos os destinos em uma ordem que seja apropriada para a aplicação - semânticas possíveis:
 - **Ordenação FIFO:** preserva a ordem da perspectiva do emissor, ou seja, se processo envia uma m antes de m' , m será entregue antes de m' em todos os processos do grupo
 - **Ordenação causal:** se uma mensagem “*happens before*” outra mensagem, essa relação será preservada na entrega
 - **Ordenação total (consistente):** se m for entregue antes m' em um processo, então a mesma ordem será preservada em todos os processos

Ordenação FIFO

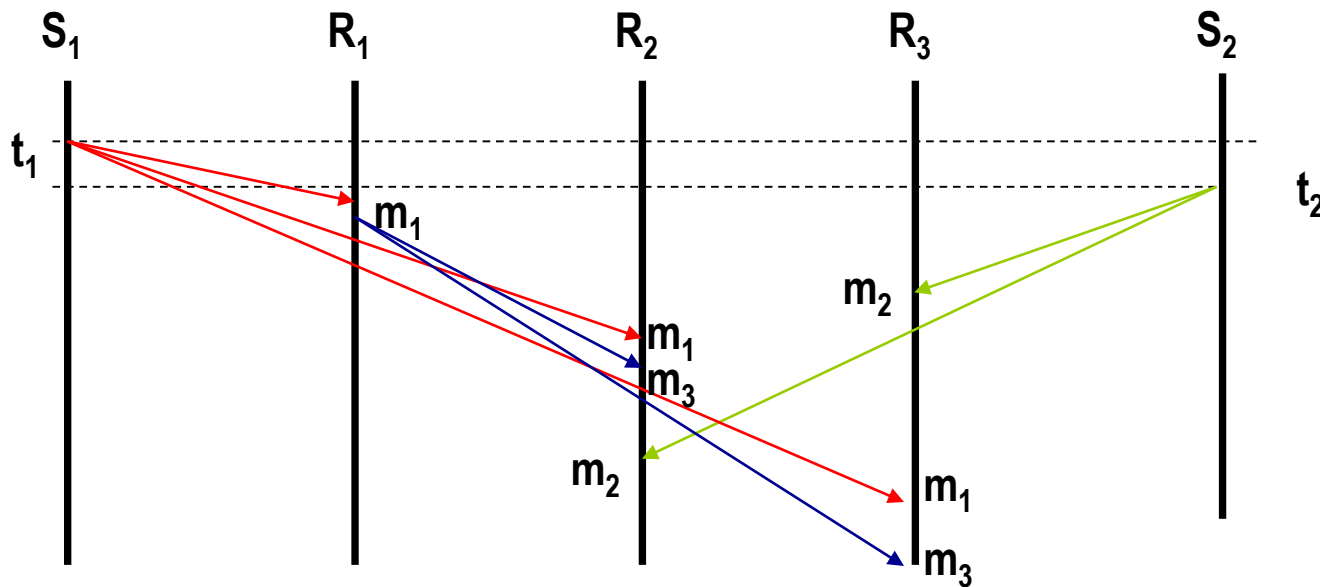
- Se um processo executa $multicast(g, m)$ e após $multicast(g, m')$, cada processo $q \in g$ realizará $deliver(m')$ após ter executado $deliver(m)$
 - Mensagens enviadas por um processo são entregues ao destino na ordem que foram enviadas



m_3 deve ser entregue depois de m_1 , mas ordem de m_2 não importa

Ordenação causal

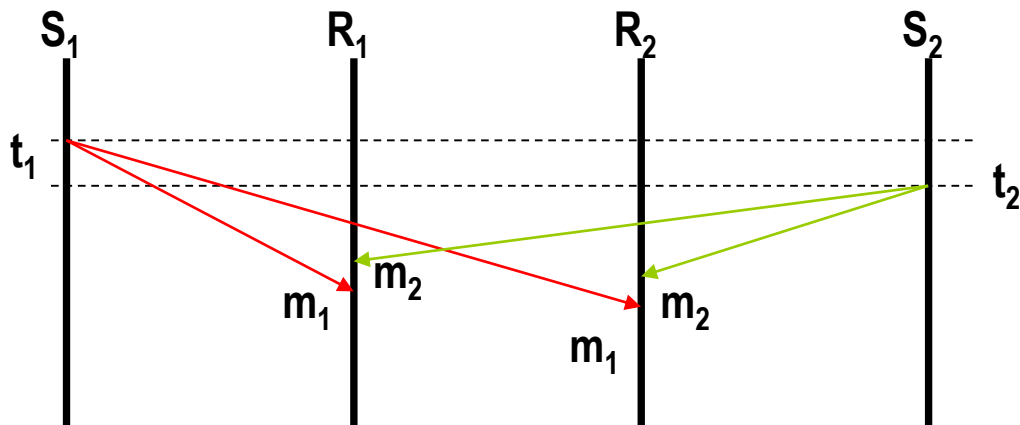
- Se $\text{multicast}(g, m) \rightarrow \text{multicast}(g, m')$ então $\text{deliver}(m')$ será executado após $\text{deliver}(m)$



Por causa da recepção de m_1 , R_1 envia a mensagem m_3 para R_2 e R_3 .
A ordem m_1 e m_3 deve ser mantida nesses dois destinos.

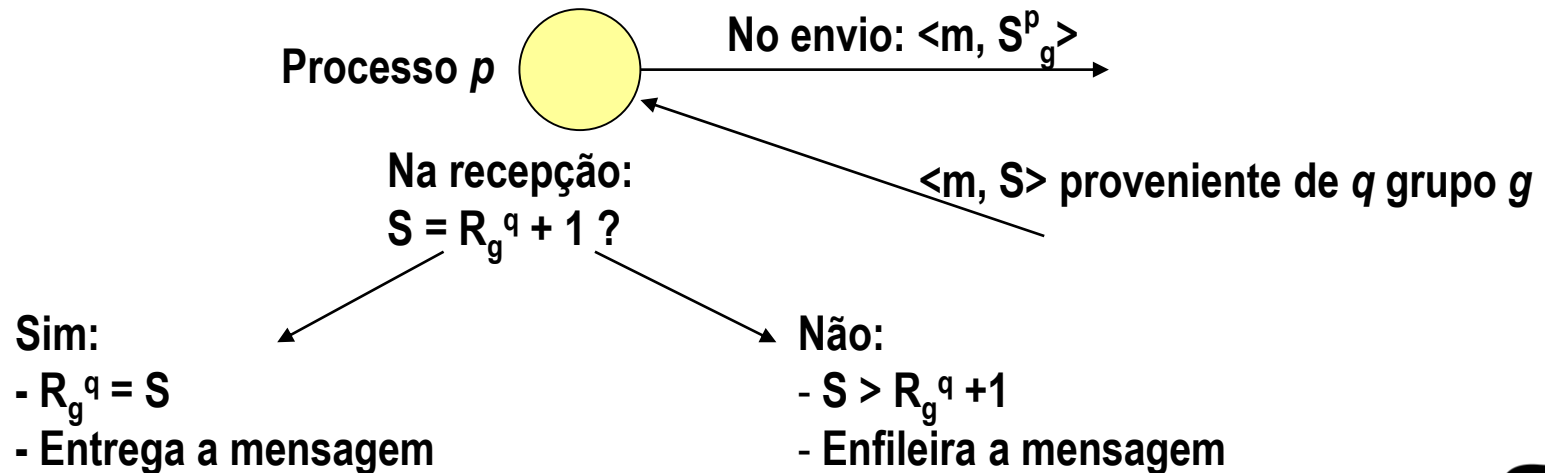
Ordenação total (consistente)

- Se um processo $p \in g$ executa $deliver(m)$ antes de $deliver(m')$, então qualquer outro processo $q \in g$ entregará m' após m
 - Garantia que todas as mensagens são entregues na mesma ordem em todos os processos mesmo que difira da ordem de emissão



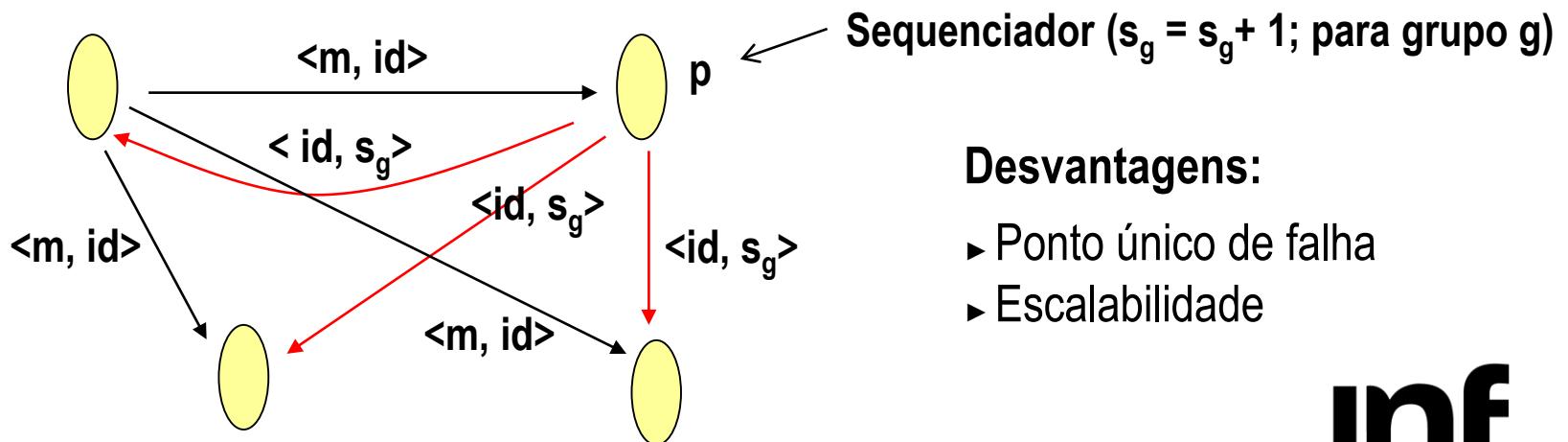
Algoritmo para ordenação FIFO

- Todas as msgs de um emissor são entregues na mesma ordem
- Baseado em números de seqüência
- Dois contadores em cada processo:
 - S_g^p : número de mensagens que processo p enviou a g
 - $\{R_g^q\}$: número de seqüência da última mensagem recebida do processo q



Algoritmo ordenação total (centralizado)

- Idéia básica: atribuir identificadores totalmente ordenados a mensagens de multicast
 - Para que cada processo tome a mesma decisão quanto ao ordenamento
- Definição de um processo $p \in g$ como **sequenciador**
 - Número de sequência por grupo, e não por processo
 - Recebe mensagem de um processo q e a retransmite a todos os membros com um identificador único sequencial



Algoritmo ordenação total (centralizado)

1. Algorithm for group member p

On initialization: $r_g := 0$;

To TO-multicast message m to group g

$B\text{-multicast}(g \cup \{\text{sequencer}(g)\}, \langle m, i \rangle)$;

On B-deliver($\langle m, i \rangle$) *with* $g = \text{group}(m)$

Place $\langle m, i \rangle$ in hold-back queue;

On B-deliver($m_{\text{order}} = \langle \text{"order"}, i, S \rangle$) *with* $g = \text{group}(m_{\text{order}})$

wait until $\langle m, i \rangle$ in hold-back queue and $S = r_g$;

$TO\text{-deliver } m$; // (after deleting it from the hold-back queue)

$r_g = S + 1$;

2. Algorithm for sequencer of g

On initialization: $s_g := 0$;

On B-deliver($\langle m, i \rangle$) *with* $g = \text{group}(m)$

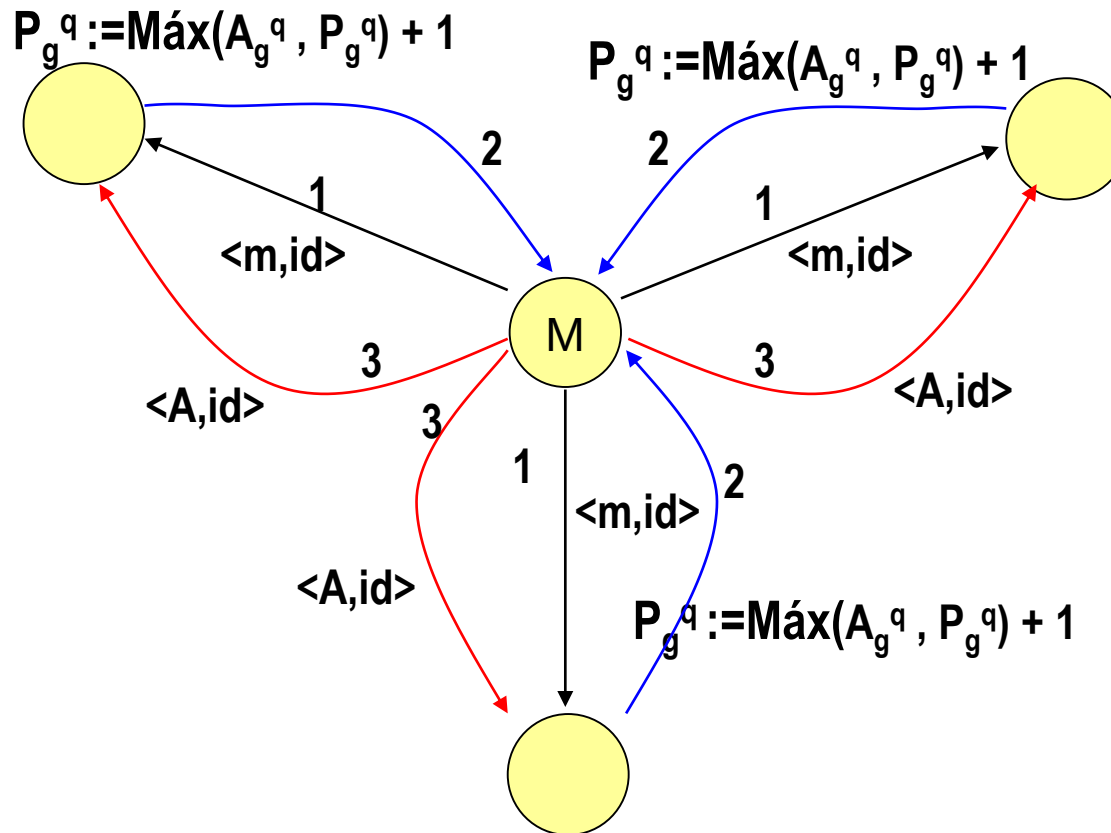
$B\text{-multicast}(g, \langle \text{"order"}, i, s_g \rangle)$;

$s_g := s_g + 1$;

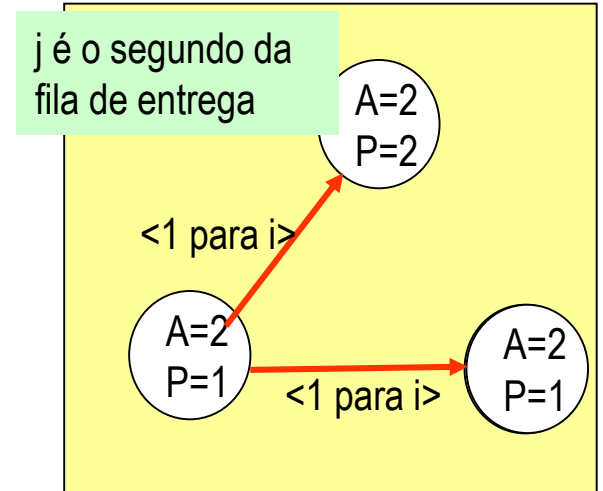
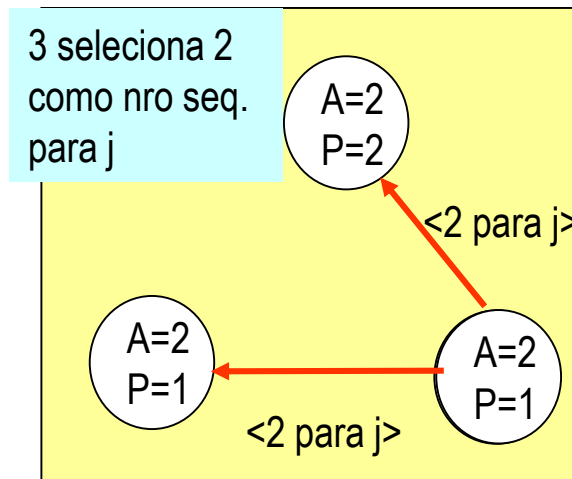
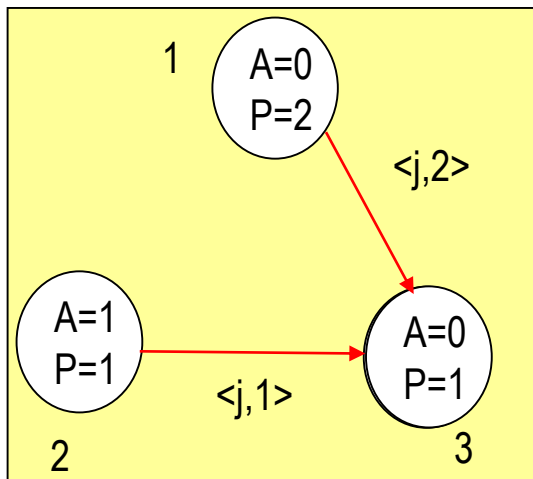
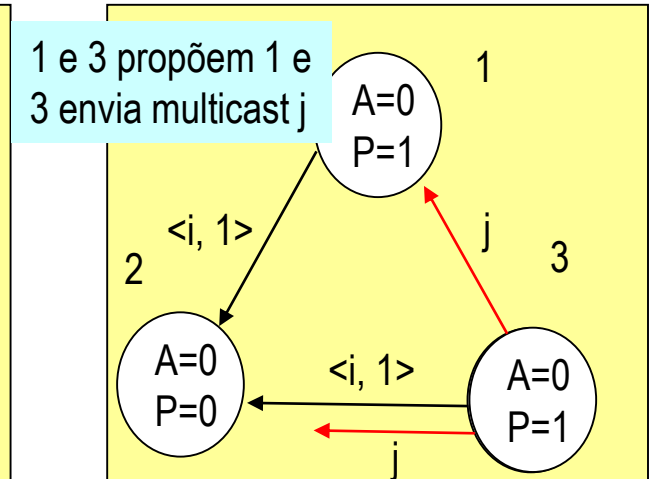
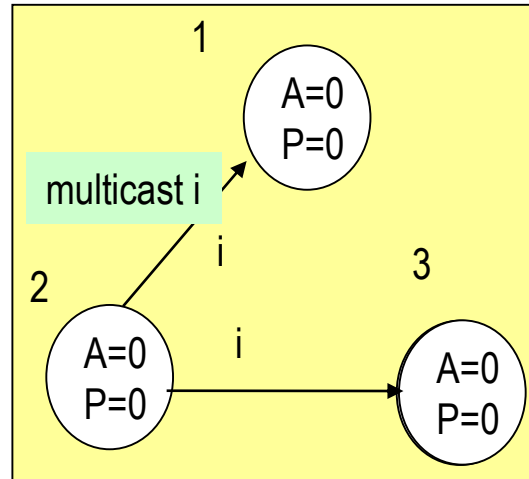
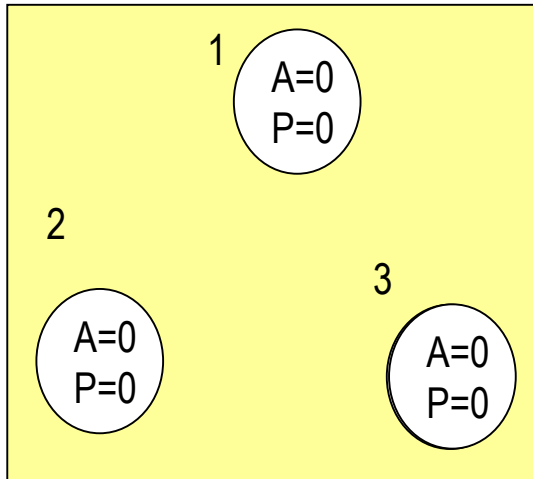
Algoritmo ordenação total (distribuído)

- Ideia básica:
 - Processos devem coletivamente concordarem na atribuição de números de sequência
 - Ao receber uma mensagem, cada processo *propõe* um número de seq.
 - Cada processo q no grupo g armazena o maior número de sequência acordado até o momento (A^q_g), e seu maior número proposto P^q_g
- Funcionamento:
 - Processo p envia $\langle m, id \rangle$ para g (id é um identificador de m)
 - Cada processo q responde a p propondo um número de sequência $P^q_g := \max(A^q_g, P^q_g) + 1$, que é enviado para o emissor
 - Emissor seleciona o maior número de sequência proposto pelos membros e envia uma nova mensagem $\langle A, id \rangle$ com esse número de sequência
 - Membros associam número de sequência a mensagem id
 - Membros realizam entrega baseado no num. de sequência

Algoritmo ordenação total (distribuído)



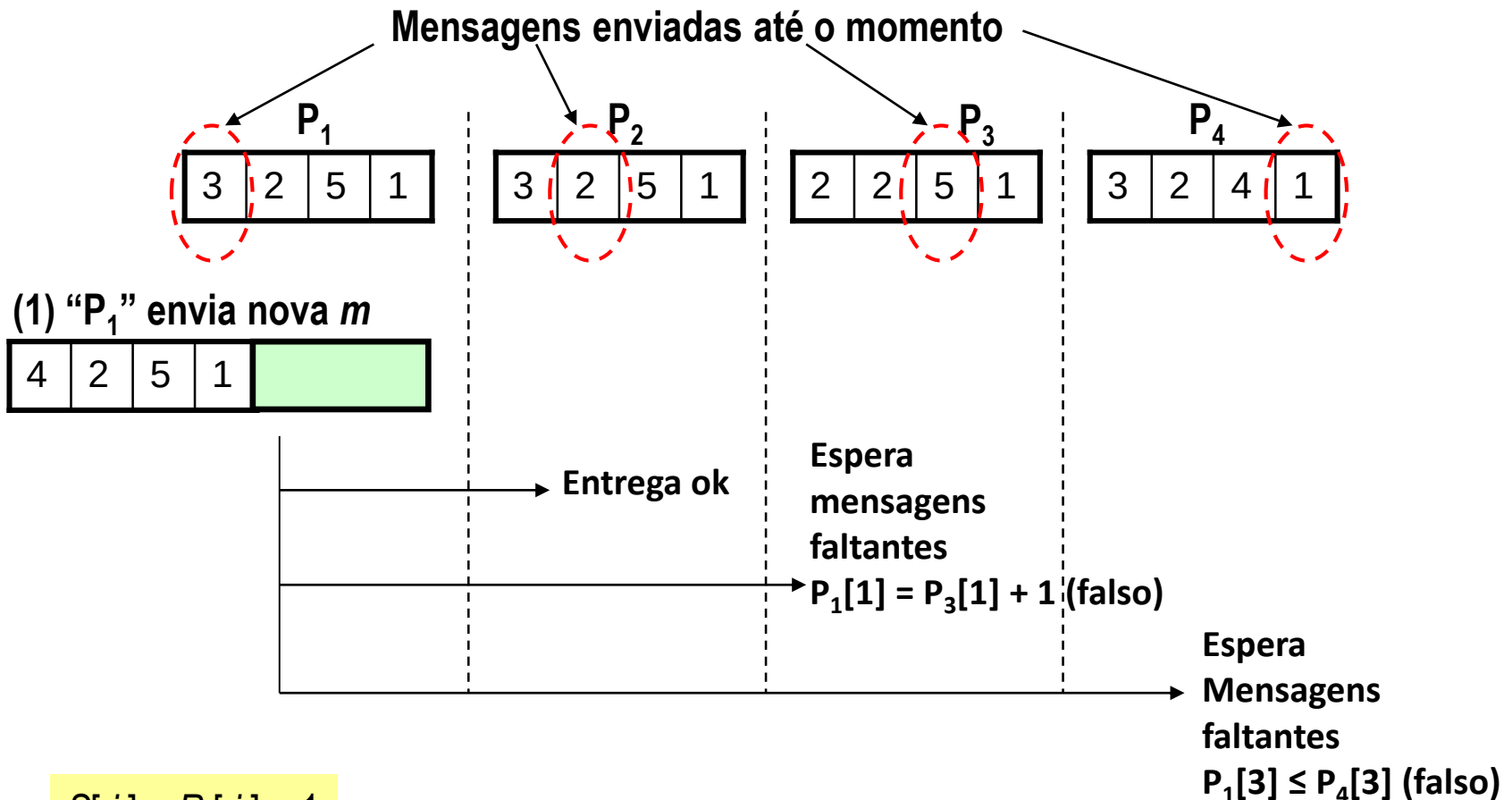
Exemplo: ordenação total



Algoritmo ordenação causal

- Baseado em relógios vetoriais (lógicos)
 - Relação causal apenas em termos de multicast
 - Cada entrada contém n° de multicasts que “*happened before*” próximo multicast
- Funcionamento:
 - Cada membro mantém um vetor de n elementos
 - Membro i corresponde a i -ésima entrada do vetor
 - Valor da entrada é o n° de sequência da última mensagem recebida de i
 - Ao enviar uma mensagem, cada membro incrementa sua própria entrada no vetor e envia o vetor junto com a mensagem
 - Entrega de mensagem enviada por P_i é feita no destino quando (simultaneamente):
 1. $S[i] = R[i] + 1 \rightarrow$ destino não perdeu nenhuma mensagem do remetente
 2. $S[j] \leq R[j]$ para todo $j \neq i \rightarrow$ destino já entregou qualquer mensagem que remetente tenha entregue quando ele fez o multicast
 - onde S é vetor recebido junto a mensagem, R é o vetor local

Algoritmo ordenação causal (*cont.*)



$$S[i] = R[i] + 1$$
$$S[j] \leq R[j]$$

Algoritmo ordenação causal

Algorithm for group member p_i ($i = 1, 2, \dots, N$)

On initialization

$V_i^g[j] := 0$ ($j = 1, 2, \dots, N$);

To CO-multicast message m to group g

$V_i^g[i] := V_i^g[i] + 1$;

$B\text{-multicast}(g, \langle V_i^g, m \rangle)$;

On $B\text{-deliver}(\langle V_j^g, m \rangle)$ from p_j , with $g = \text{group}(m)$

place $\langle V_j^g, m \rangle$ in hold-back queue;

wait until $V_j^g[j] = V_i^g[j] + 1$ and $V_j^g[k] \leq V_i^g[k]$ ($k \neq j$);

$CO\text{-deliver } m$; // after removing it from the hold-back queue

$V_i^g[j] := V_i^g[j] + 1$;

Comentário final: *multicast* na Internet

- Existe apenas na camada de rede
 - Endereço IP classe D (224.0.0.0 a 239.255.255.255)
 - IP realiza entrega computador a computador
 - Portas identificam processos origem e destino
- Noção de grupo é dinâmica
 - Membros entram e saem sem aviso prévio
 - Não há garantias de ordem, nem de entrega
 - Multicast 0-confiável
 - Podem participar de vários grupos simultaneamente
 - Grupo aberto
- Disponível apenas para sockets UDP (DGRAM)

 Reforça a necessidade dos algoritmos e das semânticas estudadas

Leituras adicionais

- Coulouris, G.; Dollimore, J.; Kindberg, T. and Blair, G.—
“*Distributed Systems: Concepts and Design*” (5th edition),
Addison Wesley, 2012
 - Capítulo 4 (seção 4.4)
 - Capítulo 6 (seção 6.2)
 - Capítulo 15 (seção 15.4)

INF01151 – Sistemas Operacionais II

Aula 22 - Exclusão mútua (distribuída)

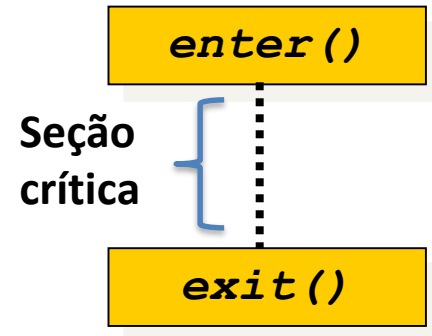
Prof. Alberto Egon Schaeffer Filho

Introdução

- Sincronização de acesso a dados e de controle são problemas clássicos em sistemas operacionais
 - Oriundo da necessidade de processos compartilharem recursos
 - Ocorre também em sistemas distribuídos... dessa forma são necessários mecanismos para:
 - Exclusão mútua em sistemas distribuídos
 - Evitar interferência e garantir consistência
- Em sistemas centralizados
 - Resolvido com emprego de mutexes, variáveis de condição e semáforos (existência de memória comum entre processos)
- Em sistemas distribuídos
 - Problema deve ser obrigatoriamente resolvido através de troca de mensagens (não há memória comum entre processos)
 - Caso trivial: recurso é gerenciado por um servidor que provê exclusão mútua
 - **Caso geral: necessita de um mecanismo separado para exclusão mútua**

Algoritmos para exclusão mútua

- Baseado em duas primitivas:
 - ***enter()***: entra na SC; se recurso não está disponível, bloqueia
 - ***exit()***: saída da SC; libera recurso o que provoca a eventual autorização para outro processo ingressar (desbloquear)
- Propriedades:
 - ***Safety***: no máximo um processo por vez na SC
 - ***Liveness***: os pedidos para entrada e saída da SC devem ter sucesso
 - Sem deadlock e starvation
 - ***Ordering***: se um pedido para entrar na SC aconteceu antes (*happened-before*) de outro, então a entrada na SC deve respeitar essa ordem



Suposições:

- Processos não falham por colapso ("pendurar")
- Mensagens não são perdidas (sem falhas por omissão)



Senão, adicionar
mecanismos de
tolerância a
falhas

Critérios para avaliação de algoritmos

- ① Atendimento as propriedades *safety*, *liveness* e *ordering*
- ② Largura de banda consumida
 - Proporcional ao número de mensagens trocadas para realização do protocolo `enter()` e `exit()`
- ③ Cliente delay
 - Sofrido por cada processo a cada operação de `enter()` e `exit()`
- ④ Synchronization delay
 - Devido a comunicação necessária entre entradas sucessivas
 - Tempo entre um processo sair da SC e o próximo entrar na SC
- ⑤ Tolerância a falhas

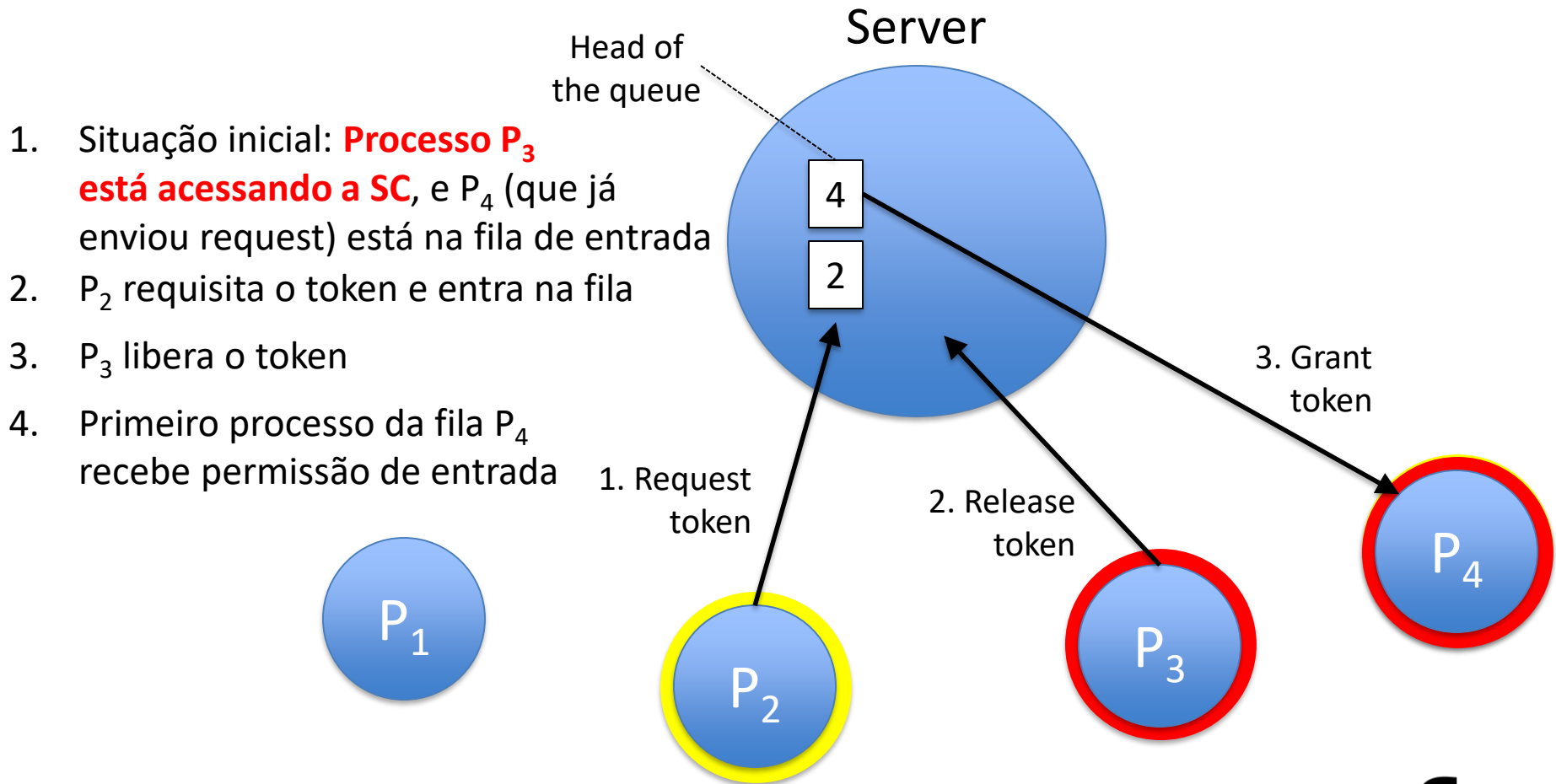
Algoritmos para exclusão mútua em sistemas distribuídos

- Algoritmo centralizado
 - Baseado em um modelo cliente-servidor (monitor)
- Algoritmos distribuídos
 - Baseado em token de permissão (anel)
 - Baseado em fila distribuída (Ricart-Agrawala)

Algoritmo centralizado

- Utiliza um controlador/servidor que concede permissões para que processos entrem na SC
 - Um processo é eleito para ser o controlador
 - Utilizando algum dos algoritmos (p. ex. *bully*) vistos em aulas anteriores...
 - Processo controlador decide sobre entrada na seção crítica
 - Mecanismo mais simples para alcançar exclusão mútua
- Princípio de funcionamento:
 - Baseado em três mensagens: ***request***, ***grant*** e ***release***
 - Processos pedem permissão para entrar na seção crítica (*request*):
 - Se **livre**: recebem autorização do controlador (*grant*)
 - Se **ocupado**: ficam bloqueados a espera da mensagem de *grant*
 - Processo ao liberar a SC, avisa o controlador (*release*)

Exemplo: algoritmo centralizado



Algoritmo centralizado: vantagens e desvantagens

- Baseado na troca de apenas três mensagens
 - Duas para o protocolo de entrada: ***request, grant***
 - Uma para o protocolo de saída: ***release***
- Propriedades:
 - *Safety*: apenas processo com token é autorizado a entrar na seção crítica
 - *Liveness*: obtida através de fila de atendimento de requisições
 - *Ordering*: satisfaz relação *happened-before*? **NÃO**
- Client delay?
 - Entrada na SC: request/grant round-trip
 - Saída da SC: caso release seja assíncrono, não há delay
- Synchronization delay?
 - Round-trip: mensagem de release para servidor, seguido de mensagem de grant
- Ponto único de falha: processo controlador
 - Recuperação exige eleição de novo controlador

Algoritmo baseado em anel

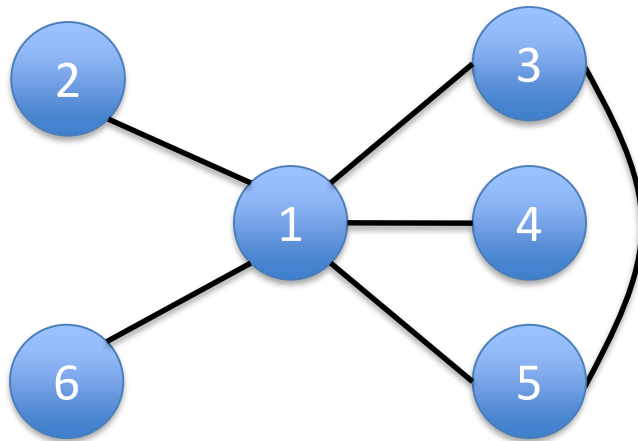
- Permite exclusão mútua entre N processos sem a necessidade de processo adicional
- Processos são logicamente organizados em anel unidirecional
 - Cada processo P_i passa um **token** para seu vizinho $P_{(i+1) \bmod N}$
 - Ideia geral: exclusão mútua é alcançada quando processo obtém um token (ficha) que é passado de processo para processo ao redor do anel
 - Token dá ao detentor (e apenas ele) o direito de acesso à seção crítica (*safety*)
 - Sendo unidirecional elimina *deadlock*, *starvation* e é justo
- Funcionamento básico:
 - Processo que deseja entrar na seção crítica espera receber o token
 - Ao **receber o token**, entra na seção crítica
 - Ao **sair da seção crítica**, **repassa o token** adiante para o vizinho
 - Se processo **não** deseja entrar na seção crítica
 - Ao receber o token, repassa adiante para o vizinho

Etapas do algoritmo em anel

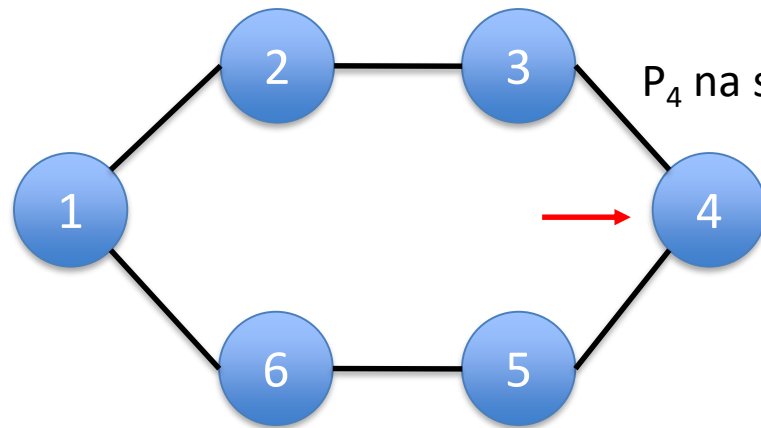
1. P_i deseja entrar na seção crítica
 - 1.1. Bloqueia a espera do token
2. P_j recebe token
 - 2.1. Se P_j deseja entrar na seção crítica, entrada é permitida e P_j segura o token
 - 2.1.1. Quando terminar de acessar a SC, repassa o token para o vizinho
 - 2.2. Se P_j não deseja usar a seção crítica, repassa o token para o vizinho

Algoritmo em anel

- Token é uma mensagem especial que serve para obter e colher permissão para entrar na seção crítica
 - Processos esperam receber o token para entrar na seção crítica



Relação entre os processos



Lógica do anel (controle)

Problema: O que acontece quando nenhum processo deseja entrar na SC?

Algoritmo baseado em anel

- Problema: algoritmo continuamente *consome bandwidth*
- Propriedades:
 - *Safety*: existe apenas um token, portanto, apenas um processo pode entrar na seção crítica
 - *Liveness*: garante que quem deseja entrar na seção crítica obterá o token em tempo finito
 - Por construção do anel, processo em algum momento receberá o token de seu vizinho
 - *Ordering*: pode não respeitar a ordem de solicitação
 - Processos podem trocar outras mensagens além do token sendo passado
- Client delay?
 - Entrada na SC: entre 0 (se acabou de receber o token) e N mensagens
 - Saída da SC: somente 1 mensagem
- Synchronization delay (entre saída de processo e entrada do próx.)?
 - Entre 1 e N mensagens

Algoritmo Ricart-Agrawala [1981]

- Implementa exclusão mútua entre N processos
 - Baseado em multicast e relógios lógicos
 - Garante o acesso à seção crítica em uma ordem FCFS
 - Implementa uma fila distribuída
- Idéia básica:
 - Cada processo está em um de três estados possíveis:
 - **RELEASED**: processo não está na seção crítica e não deseja entrar
 - **WANTED**: processo deseja entrar na seção crítica
 - **HELD**: processo está na seção crítica
 - Processo que deseja entrar na seção crítica envia mensagem aos demais e espera que estes (todos!) autorizem sua entrada
 - Total de mensagens: $2*(n-1)$
 - $n-1$ requisições de entrada e $n-1$ respostas (autorizações)
 - Multicast nativo: n mensagens (1 de requisição e $n-1$ autorizações)

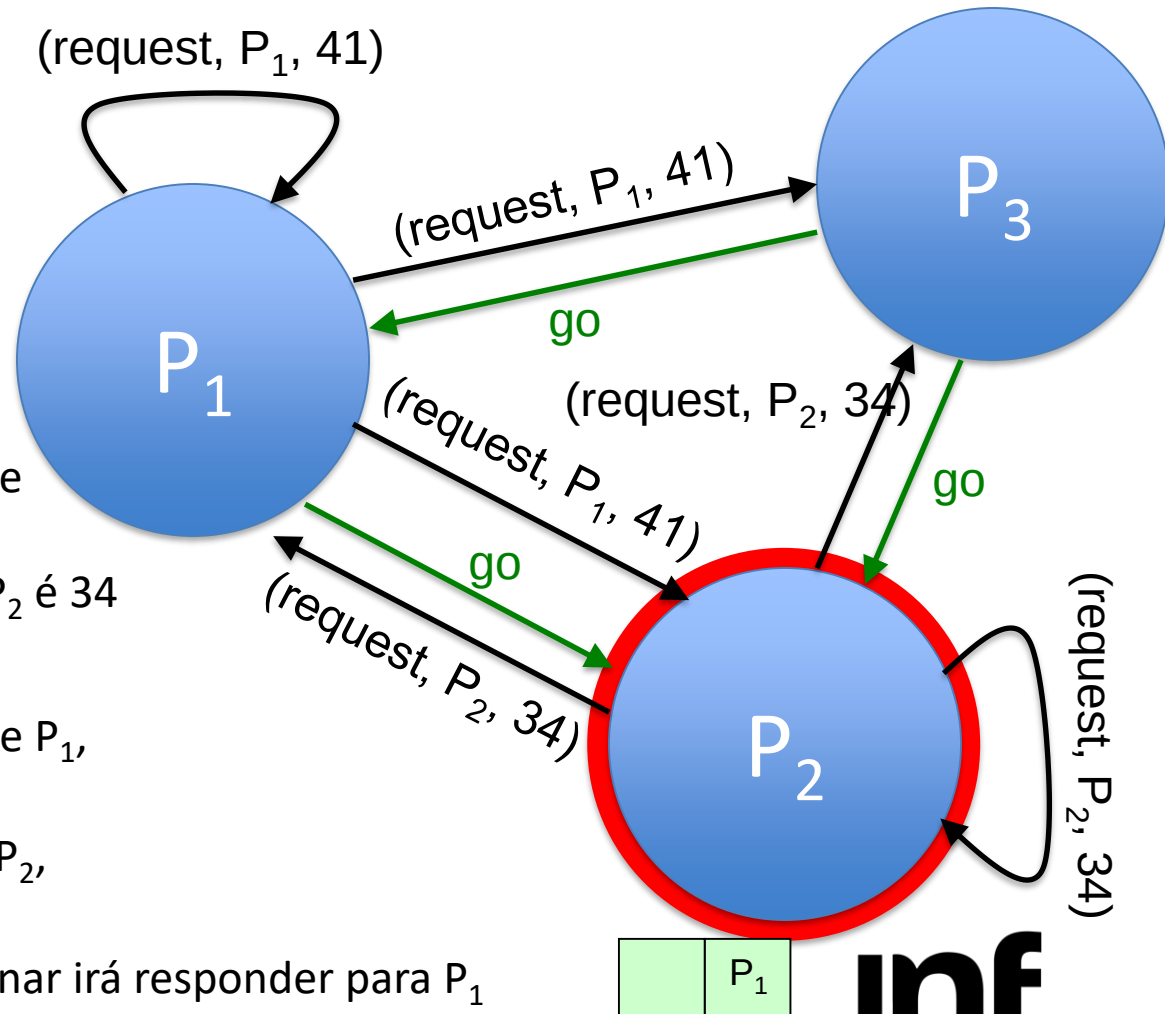
Etapas do algoritmo Ricart-Agrawala

1. P_r **deseja entrar** na seção crítica
 - 1.1. **Envia** mensagem (multicast) de **requisição de entrada** (request, P_r , ts)
 - 1.2. **Bloqueia** a espera das **autorizações** (TODAS!!)
2. P_j **recebe mensagem** de requisição de entrada de P_r
 - 2.1. Se P_j **não deseja entrar na seção crítica (RELEASED)**, envia “go” para P_r
 - 2.2. Se P_j **deseja entrar na SC (WANTED)**, envia “go” para P_r apenas se timestamp for menor do que o de sua própria requisição, senão, enfileira a requisição de P_r
 - 2.3. Se P_j **está usando a seção crítica (HELD)**, enfileira a requisição de P_r
3. P_r **recebe n-1 replies** do tipo “go”: **entra na seção crítica**
4. P_r **ao sair** da seção crítica **envia “go”** para cada processo que está na sua lista de requisição pendente

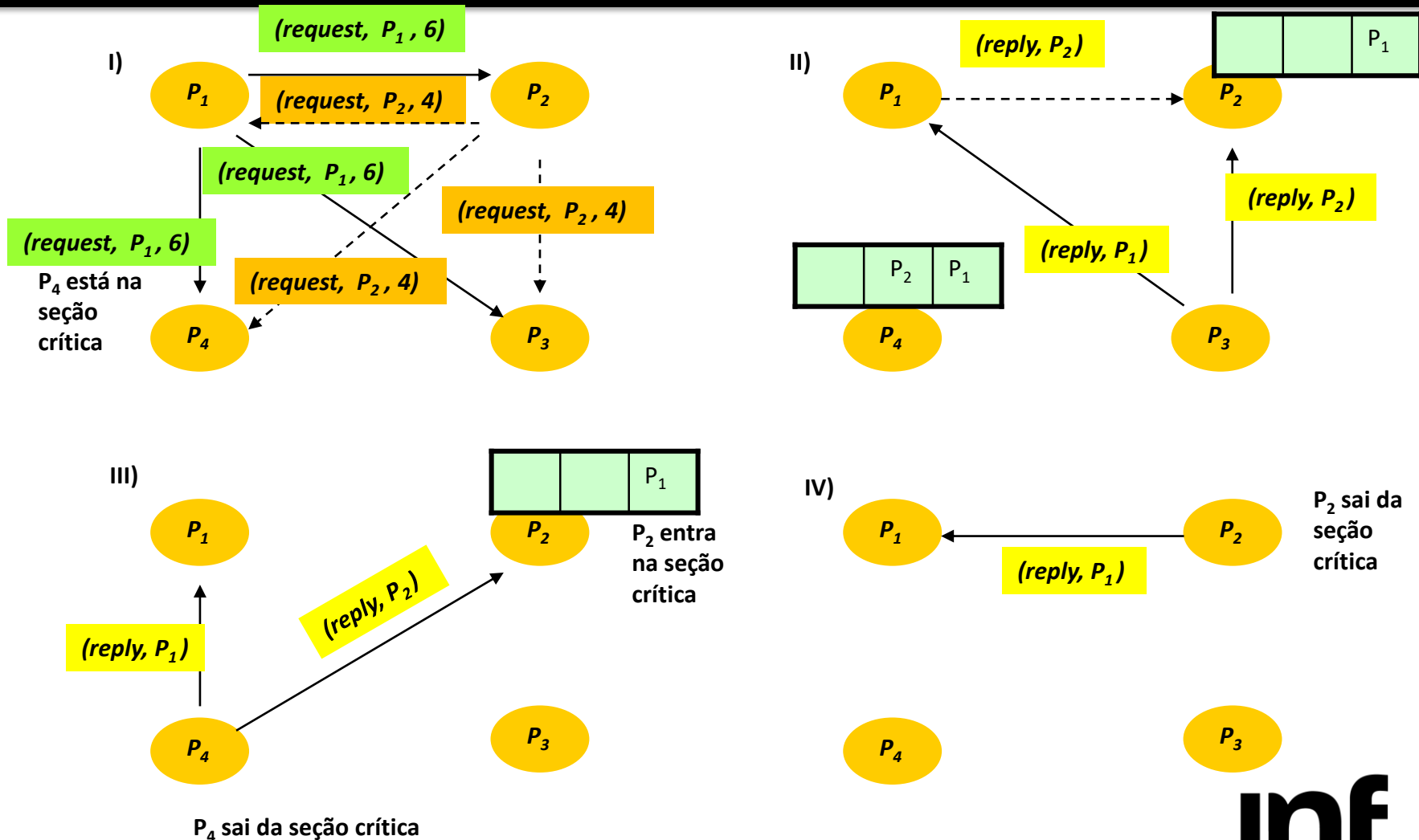
Se dois processos requisitarem acesso à SC “ao mesmo tempo”, quem irá entrar? E se os dois tiverem o mesmo timestamp?

Exemplo: algoritmo de Ricart-Agrawala

1. Situação inicial: P_3 não deseja entrar na SC
 - P_1 e P_2 requisitam entrada de forma concorrente
 - Timestamp de P_1 é 41 e de P_2 é 34
2. P_3 responde imediatamente
3. Quando P_2 recebe requisição de P_1 , não responde e enfileira
4. Quando P_1 recebe requisição de P_2 , responde imediatamente
5. P_2 entra na SC, e quando terminar irá responder para P_1



Exemplo: algoritmo de Ricart-Agrawala



Algoritmo Ricart-Agrawala

On initialization

state := **RELEASED**;

To enter the critical section

state := **WANTED**;

Multicast request to all processes;

T := request's timestamp;

Wait until (number of replies received = (N - 1));

state := **HELD**;

} request processing
deferred here

On receipt of a request $\langle T_i, p_i \rangle$ at p_j ($i \neq j$)

if (state = **HELD** or (state = **WANTED** and $(T, p_j) < (T_i, p_i)$))

then

 queue request from p_i without replying;

else

 reply immediately to p_i ;

end if

To exit the critical section

state := **RELEASED**;

reply to any queued requests;

Algoritmo Ricart-Agrawala [1981]

- Propriedades:
 - *Safety*: apenas um processo receberá as $n-1$ autorizações
 - *Liveness*: atendimento conforme relógios lógicos, portanto, não há postergação infinita
 - *Ordering*: garantido através de relógios lógicos
- Número de mensagens?
 - Entrar na SC: $2(N-1)$
- Client delay?
 - Round trip time
- Synchronization delay?
 - Tempo de transmissão de apenas uma mensagem

Comentários gerais sobre algoritmos exclusão mútua em relação a tolerância a falhas

- Algoritmo centralizado
 - Perda do controlador → algoritmo de eleição
- Algoritmos baseado em anel (token)
 - Erros possíveis
 - Falhas de processos ou rede “quebram” anel
 - Possibilidade do token ser perdido
 - Processo de monitoração do anel (algoritmo de eleição)
- Algoritmos de fila distribuída (Ricart-Agrawala)
 - Possível receber a autorização apenas de um sub-conjunto de processos (Algoritmos de Maekawa, Tanenbaum, Lamport...)
- Não endereçam todos os problemas
 - Questão do estado dos recursos no momento da falha
 - Onde estava o token?

Leituras adicionais

- Coulouris, G.; Dollimore, J.; Kindberg, T. and Blair, G.—
“*Distributed Systems: Concepts and Design*” (5th edition),
Addison Wesley, 2012
 - Capítulo 15 (seção 15.2)

Exercícios

- ① No **algoritmo centralizado** para exclusão mútua, descreva uma situação/exemplo onde duas requisições não são processadas em ordem happened-before.
- ② Apresente um exemplo de execução do **algoritmo baseado em anel** que mostre que a entrada de processos na seção crítica não acontece, necessariamente, em ordem happened-before. *Lembre-se: a topologia de anel é apenas utilizada para controle, e processos podem se comunicar através de alguma outra interconexão física.*
- ③ Em alguns sistemas, cada processo pode tipicamente utilizar a seção crítica diversas vezes antes que outro processo requisiite acesso. Explique porque o **algoritmo de Ricart e Agrawala** é ineficiente para este caso, e descreva como melhorar a performance do algoritmo.

INF01151 – Sistemas Operacionais II

Aula 23 - Deadlocks

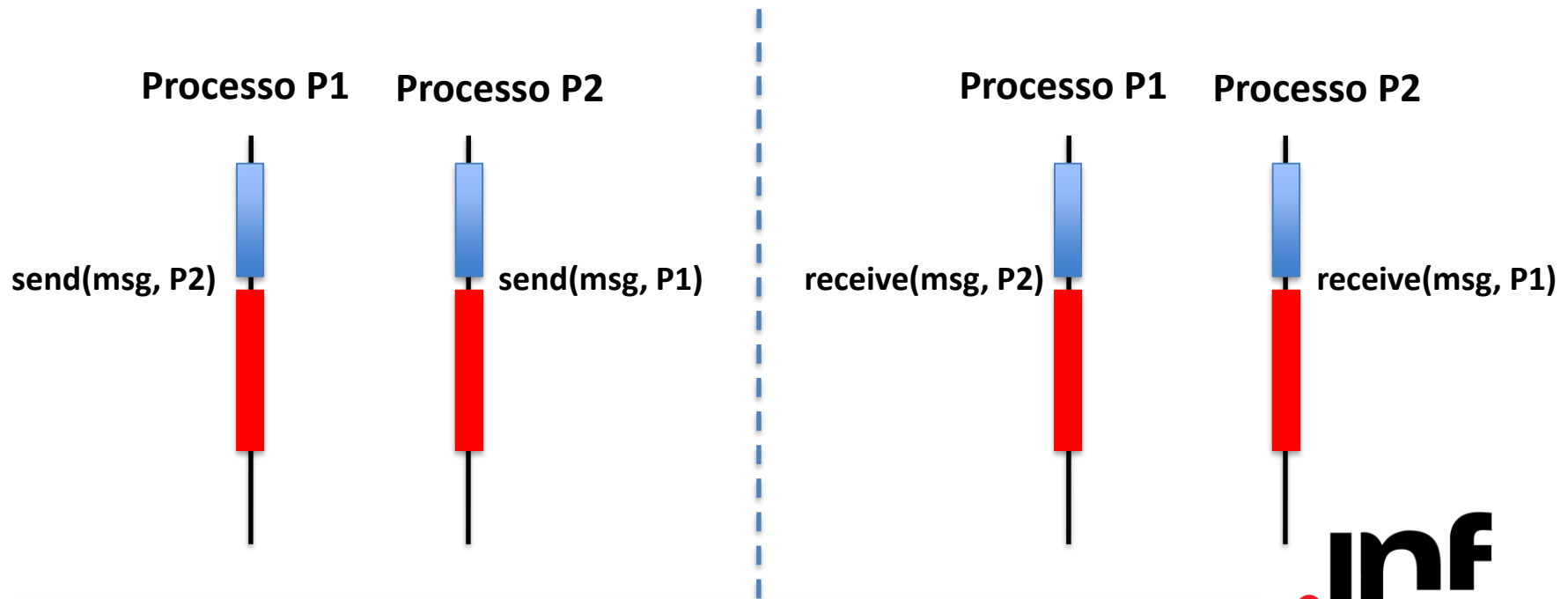
Prof. Alberto Egon Schaeffer Filho

Deadlock em sistemas distribuídos

- Deadlock é a espera por um evento que não ocorrerá
 - Situação em que processos/threads ficam bloqueados permanentemente esperando pela liberação de um recurso
- Deadlock distribuido
 - Ocorre quando processos dispersos por diferentes computadores de uma rede aguardam por eventos que não ocorrerão
 - Problema similar a sistemas centralizados
 - Dificuldade é a coleta e a manutenção de informações
- Certos autores dividem em:
 - *Deadlocks* de comunicação
 - *Deadlocks* na alocação de recursos (exemplo clássico!)
 - *Deadlocks* fantasmas

Deadlock de comunicação

- Devido à semântica bloqueante de send / receive
 - Uma espera circular por comunicação
 - P_1 bloqueia no envio de mensagem a P_2 , que bloqueia no envio de mensagem a P_3 , que bloqueia no envio de mensagem a P_1
 - P_1 espera uma resposta de P_2 , que espera uma resposta de P_3 , que espera uma resposta de P_1



Condições para ocorrência de deadlock

- Quatro situações devem ser satisfeitas simultaneamente:

1. Exclusão mútua

- Somente um processo pode usar o recurso em um determinado instante

2. Segura/espera

- Um processo, de posse de um recurso, espera para alocar outro recurso

3. Recurso não-preemptível

- Uma vez alocado um recurso, ele não pode ser retirado de um processo

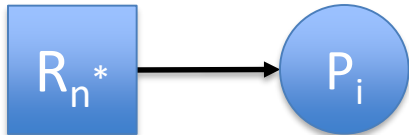
4. Espera circular

- Uma sequência de processos onde um processo mantém ao menos um recurso que é necessário ao próximo processo

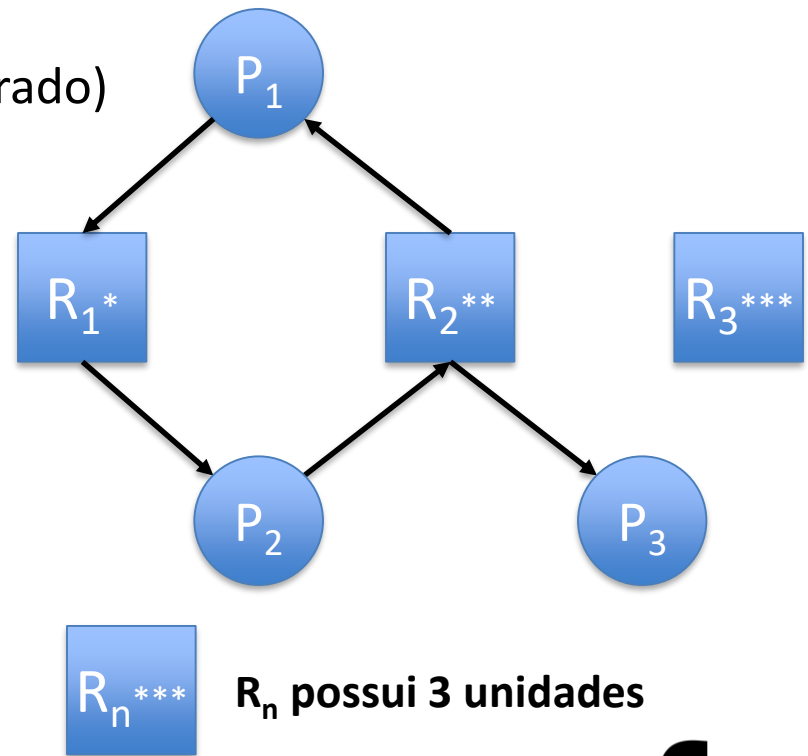
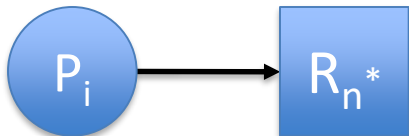
Se uma for quebrada, então *deadlock* não acontece no sistema!

Modelagem de deadlocks

- Grafo direcionado
 - Grafo de alocação e requisição de recursos
- Dois tipos de nós
 - Processos (círculo) e recursos (quadrado)
- Dois tipos de arestas:
 - Alocação (recurso $\rightarrow$ processo): processo detém um recurso



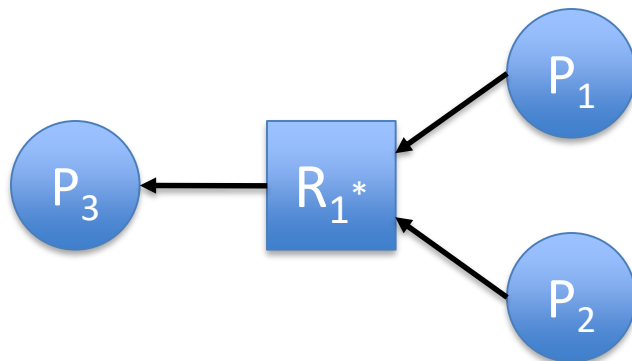
- Requisição (processo $\rightarrow$ recurso): processo deseja um recurso



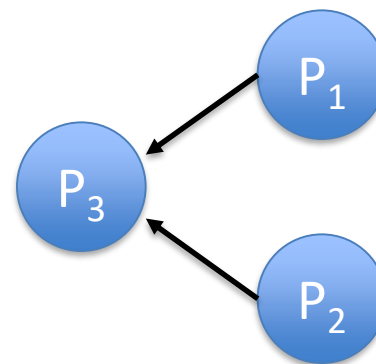
R_n possui 3 unidades

Grafos RRAG e WFG

- Resource Request and Allocation Graph (RRAG)
 - Dois tipos de nós: processos e recursos
- Wait-For-Graph (WFG)
 - Caso particular de um RRAG onde há apenas uma unidade de um determinado recurso
 - Simplificação, com apenas um tipo de nó: processos



RRAG



WFG

Tratamento de deadlock

- Ignorar
 - Algoritmo da “avestruz”
- Prevenir (*prevention*)
 - Condicionar o sistema a afastar qualquer possibilidade de deadlocks
 - “Negar” uma das quatro condições necessárias como no caso centralizado
- Evitar (*avoidance*)
 - Permitir que possibilidade exista, mas na iminência de acontecer, deadlock é cuidadosamente desviado
 - Alocação criteriosa de recursos (algoritmo do banqueiro como no caso centralizado)
- Detectar & recuperar
 - Deixar o deadlock ocorrer, e ser capaz de determinar sua existência
 - Depois, limpar os deadlocks do sistemas
 - Mecanismos de *checkpoint*, transação e suspensão/retomada



Prevenção de deadlock

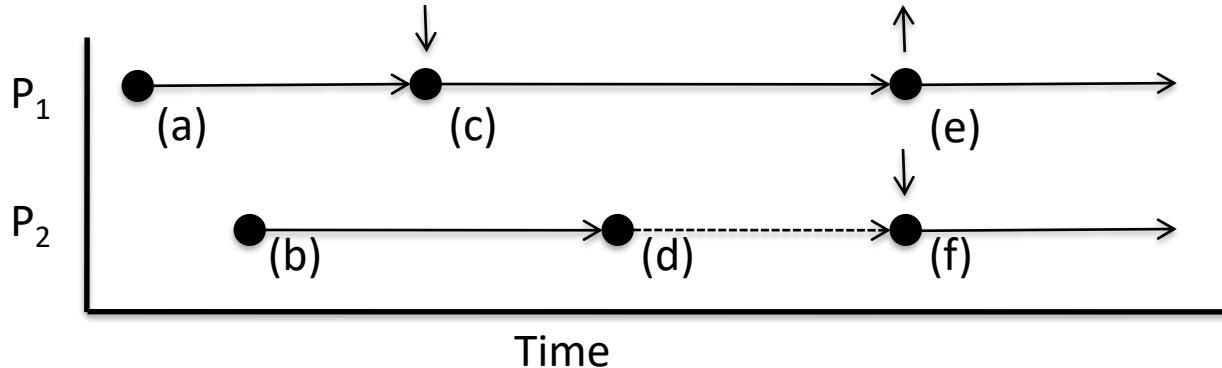
- Objetivo:
 - Eliminar uma das quatro condições necessárias
- Duas estratégias básicas para prevenir deadlocks [Rosenkrantz, 1978]
 - *Wound-wait* (ferir-esperar)
 - *Wait-die* (esperar-morrer)
- Ambos os algoritmos dependem da ordenação de processos com base no momento em que cada processo foi iniciado
 - Processos ordenados por idade
 - Processos na criação recebem timestamps
 - $\langle \text{tempo_local}, \text{id_nó} \rangle$

Estratégias *wound-wait* e *wait-die*

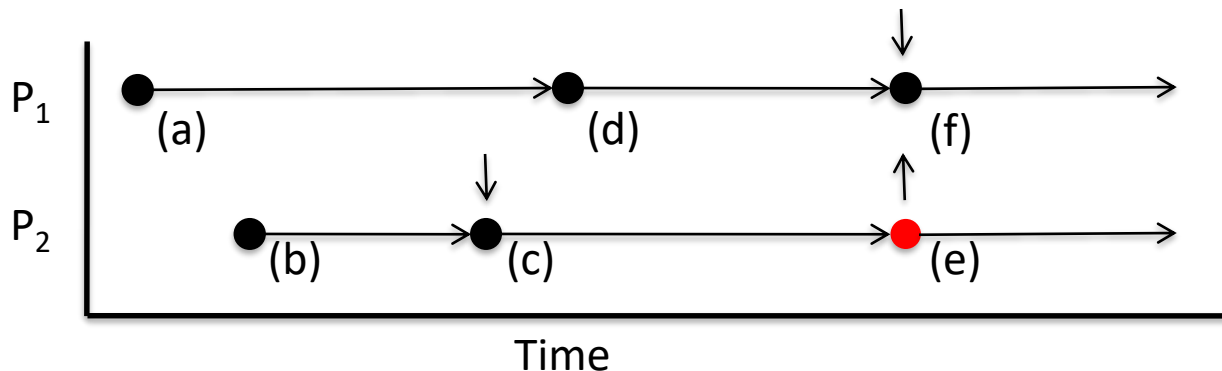
- *Wound-wait*

- Processo mais novo (maior *timestamp*) pode esperar pelo mais velho, mas o mais velho não espera pelo mais novo

- Nega condição de recurso não-preemptível



- (a) P_1 starts
- (b) P_2 starts
- (c) P_1 acquires R_1
- (d) P_2 requests R_1 , waits
- (e) P_1 releases R_1
- (f) P_2 acquires R_1



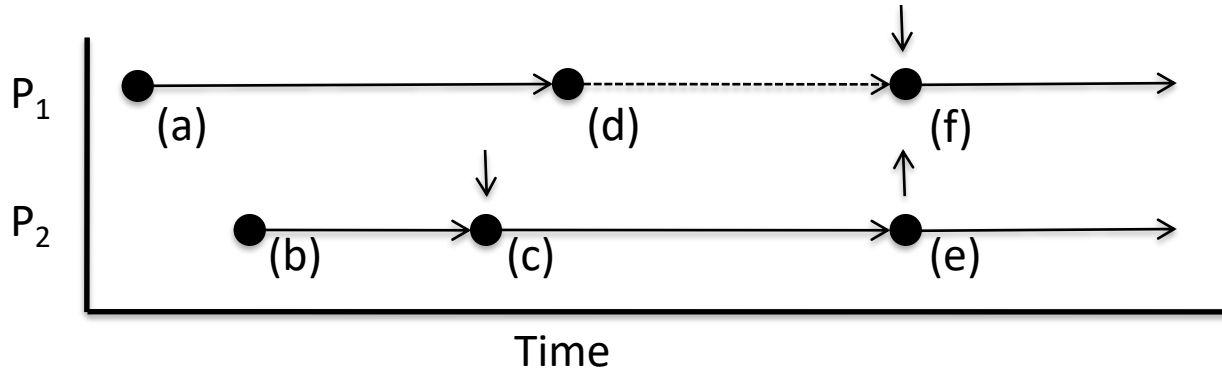
- (a) P_1 starts
- (b) P_2 starts
- (c) P_2 acquires R_1
- (d) P_1 requests R_1 , wounds P_2
- (e) P_2 is restarted, releases R_1
- (f) P_1 acquires R_1

Estratégias *wound-wait* e *wait-die*

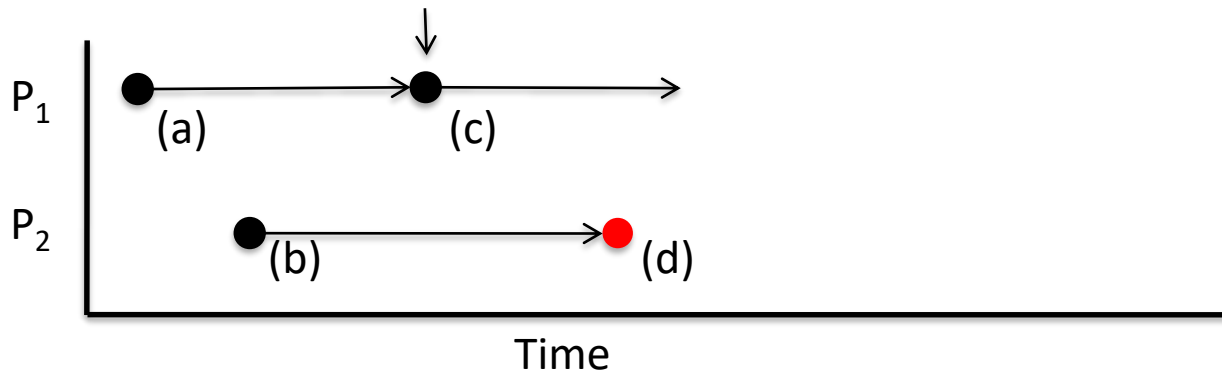
- *Wait-die*

- Processo mais velho (menor *timestamp*) pode esperar pelo mais novo, mas o mais novo não espera pelo mais velho

- Nega condição de segura/espera



- (a) P_1 starts
- (b) P_2 starts
- (c) P_2 acquires R_1
- (d) P_1 requests R_1 , waits
- (e) P_2 releases R_1
- (f) P_1 acquires R_1



- (a) P_1 starts
- (b) P_2 starts
- (c) P_1 acquires R_1
- (d) P_2 requests R_1 , dies

Questão do aborto de processo

- As estratégias abortam, se necessário, o processo mais novo
 - O que realizou menos tarefas (redução de prejuízo)
 - Wound-wait:
 - Processo novos são reiniciados se recurso for requisitado por processo mais velho
 - Processos novos devem esperar por processos mais antigos
 - Conforme envelhece, esperam com menos frequência
 - Wait-die:
 - Processos novos irão morrer ao invés de esperar
 - Conforme processos envelhecem, podem ser forçados a esperar com mais frequência
 - Cuidados: preemptar recursos alocados antes de abortar o processo

Situação	<i>Wait-die</i>	<i>Wound-wait</i>
P_{velho} precisa recurso do P_{novo}	P_{velho} espera	P_{novo} aborta
P_{novo} precisa recurso do P_{velho}	P_{novo} aborta	P_{novo} espera

Detecção de deadlocks

- Deixar o deadlock ocorrer, e ser capaz de determinar sua existência
 - Consiste em detectar ciclos em grafos de alocação e removê-los
 - Quando detectado, selecionar um processo para ser abortado para quebrar o ciclo
 - Quem escolher para abortar?
- Técnicas similares ao caso centralizado
 - Porém, no caso centralizado existe um único grafo de alocação com uma representação “global”
- Algoritmos para sistemas distribuídos:
 - Centralizado
 - Distribuído (Chandy-Misra-Hass)

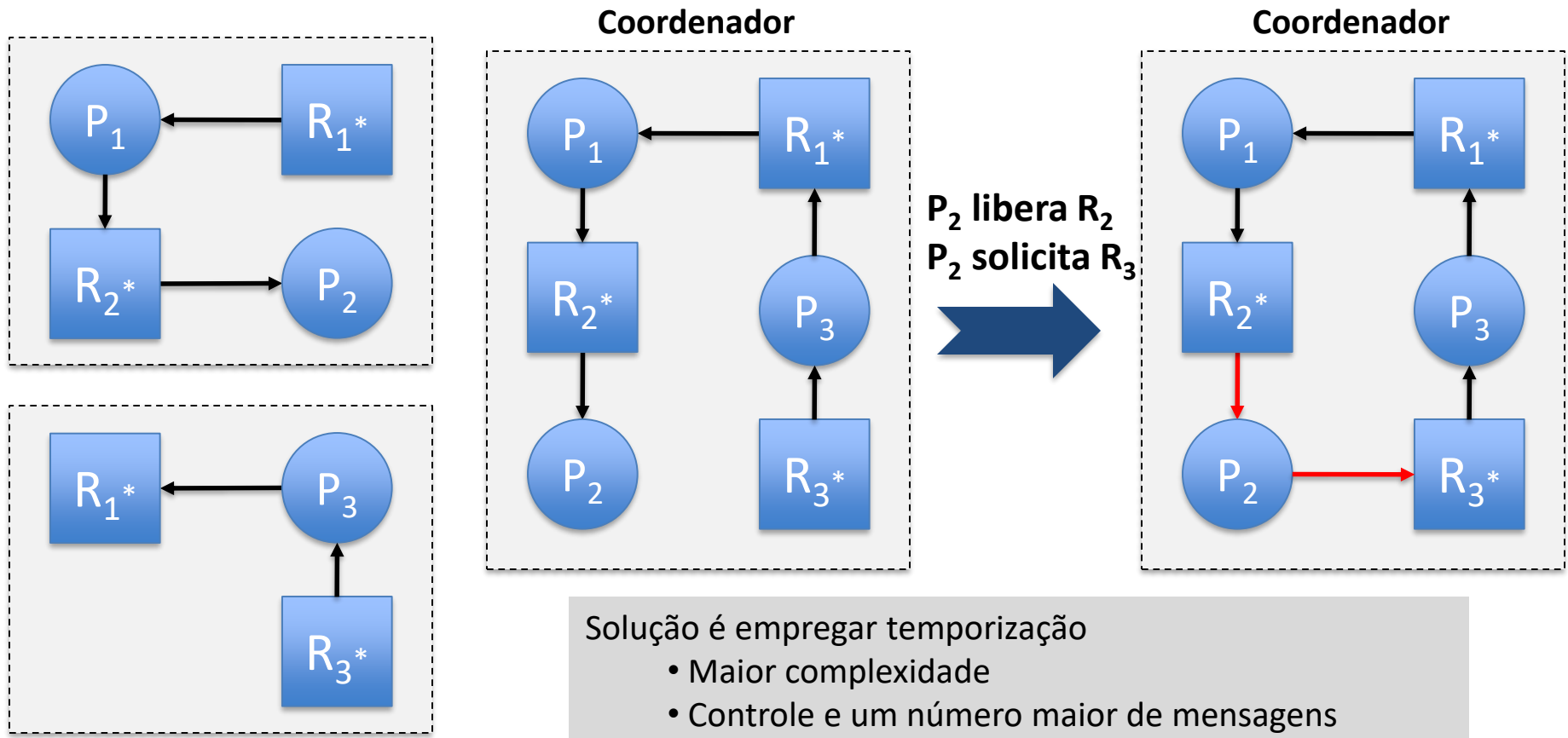
Algoritmo centralizado

- Cada computador mantém grafo local de alocação de recursos
- Existe um computador central (coordenador) que monitora o sistema inteiro
 - Outros processos notificam o coordenador quanto a alocações
 - Coordenador constrói um grafo de alocação “global”
 - Quando um ciclo for detectado
 - Coordenador implementa algum algoritmo de recuperação (e.g., matar um processo)
- Opções para construção do grafo global:
 - ① Mensagens de atualização para cada adição ou remoção de arestas
 - ② Periodicamente, com envio de lista de arestas adicionadas/removidas
 - ③ Sob demanda do coordenador

Problemas usuais: baixa escalabilidade, ponto unico de falhas, trade-off sobre a frequência de updates, ...

Problema: deadlock fantasma

- Devido a atrasos associados à computação distribuída, é possível que um algoritmo detecte um deadlock que não exista
 - Deadlock fantasma: é “detectado” mas não é na verdade um deadlock



Algoritmo Chandy-Misra-Haas

- Algoritmo distribuído (edge chasing, path pushing)
 - Grafo global de alocação não é contruído
 - Mensagens de probe enviadas através do grafo por todo o sistema distr.
- Princípio de funcionamento:
 - Mensagens especiais (*probe*) são enviadas pelo grafo de dependência
 - Ao receber um *probe*, o processo envia a seu sucessor
 - Se um *probe* retorna ao seu originador: → *deadlock*

Probe é uma tupla (*i*, *j*, *k*)

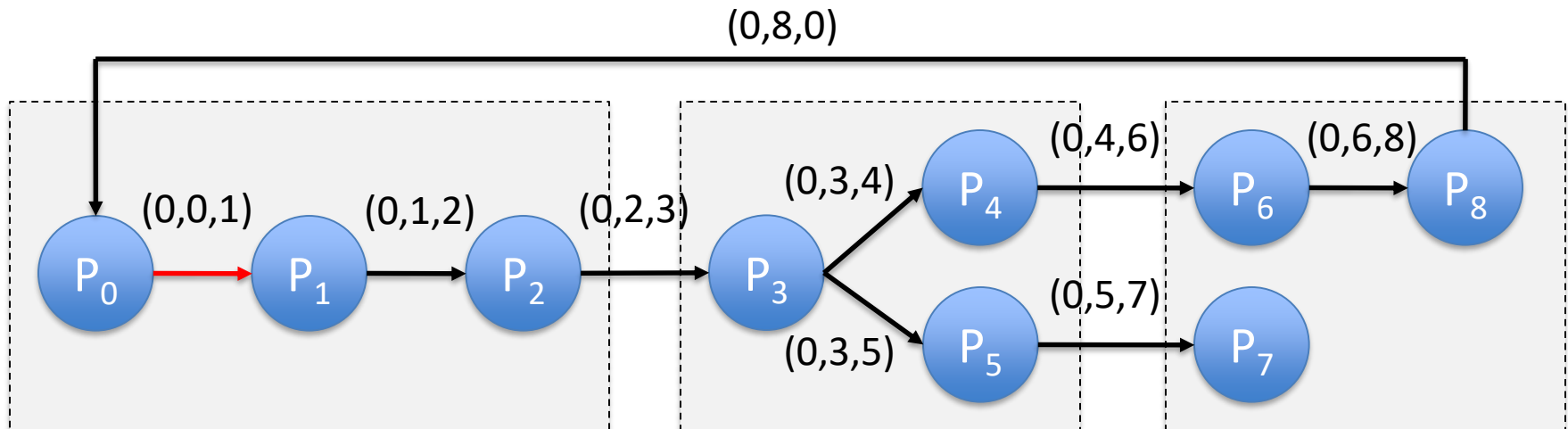
i: iniciador do processo de detecção de *deadlock*

j: emissor do *probe*

k: destinatário do *probe*

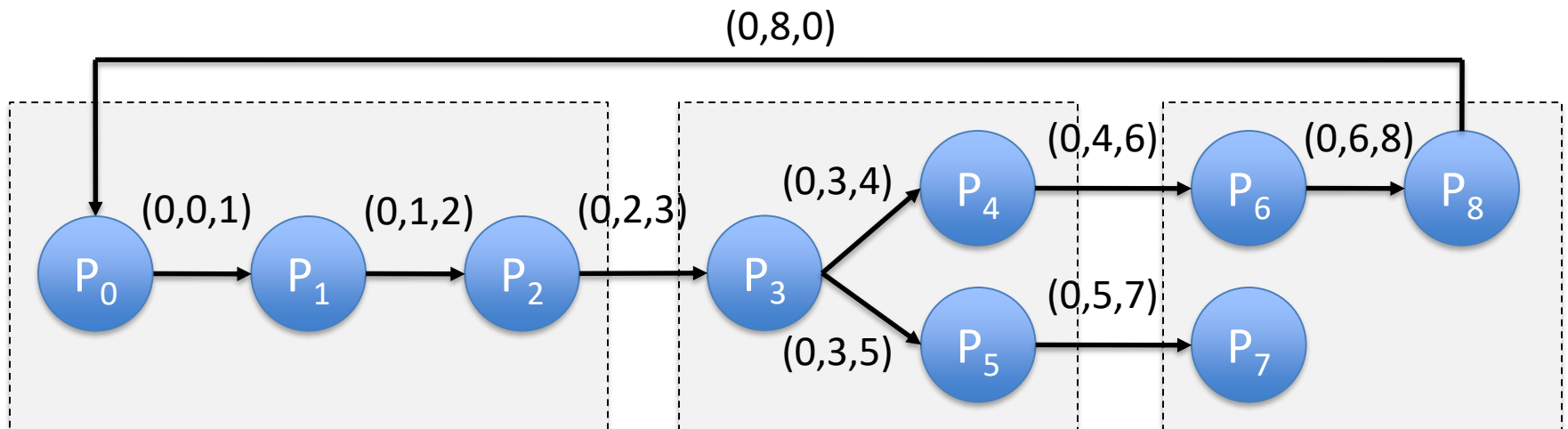
Algoritmo Chandy-Misra-Haas

- Funcionamento do algoritmo
 - Sempre que um processo P_i será bloqueado devido a outro processo (P_j), ele envia uma mensagem com o *probe*(i, i, j)
 - Se P_j recebe *probe*(i,i,j) e P_j está bloqueado a espera de um processo P_k , então ele envia uma mensagem *probe*(i,j,k) para P_k
 - Se P_j não está bloqueado, nenhuma mensagem é enviada
 - Se o *probe* retornar à origem (P_i) é porque existe um *deadlock*



Algoritmo Chandy-Misra-Haas

- Como quebrar o *deadlock*?
 - Processo que iniciou a seqüência de mensagens se suicida
 - Desvantagem (risco): suicídio em demasia devido ao tempo da mensagem circular
 - E.g.: P_6 e P_0 iniciam o *probe* ao mesmo tempo
 - Incluir na mensagem a identidade de quem a envia
 - O emissor original recebe uma lista dos processos envolvidos no ciclo
 - Eliminar o de mais alto (baixo) identificador/prioridade/timestamp



Recuperação de deadlock

- Geralmente realizada pela retirada forçada de um processo do sistema e retomada de seus recursos
 - A idéia é “voltar” o sistema para uma situação onde não ocorra o *deadlock*
 - Envolve suspensão, morte ou reinicialização de processo(s)
 - Pode implicar em perder o trabalho que o processo retirado realizou
 - Como determinar qual processo retirar?
- Técnicas básicas (mesma de centralizados)
 - Suspensão/retomada (preempção temporária dos seus recursos)
 - Quando for seguro, retomar o processo retido sem perda de trabalho
 - Checkpoint
 - Limita a perda de trabalho até o momento do último checkpoint
 - Transações
 - Mudanças permanentes se transação for concluída com sucesso

Leituras adicionais

- Silberschatz, A. and Galvin, P. – *“Operating Systems Concepts”*. Wiley, 8th edition, 2009.
 - Capítulo 7
 - Capítulo 18 (seção 18.5)
- Deitel, H.; Deitel, P.; Choffnes, D. – *Sistemas Operacionais*, 3ª edição, Pearson Prentice Hall, 2005.
 - Capítulo 7
 - Capítulo 17 (seção 17.6)
- Coulouris, G.; Dollimore, J.; Kindberg, T. and Blair, G.– *“Distributed Systems: Concepts and Design”* (5th edition), Addison Wesley, 2012.
 - Capítulo 16 (seção 16.4)
 - Capítulo 17 (seção 17.5)

INF01151 – Sistemas Operacionais II

Aula 24 - Transações

Prof. Alberto Egon Schaeffer Filho

Introdução

- Sequência de operações em objetos que é executada **de forma atômica** pelos servidores que gerenciam tais objetos
- Necessidade de manter objetos (recursos) de um servidor em um **estado consistente** na presença de
 - Falhas por colapso do servidor/cliente
 - Servidor deve garantir que ou a transação executa completamente (commit) ou seus resultados parciais são apagados (abort/rollback)
 - Acessos concorrentes por múltiplos clientes
 - Um cliente não pode observar efeitos parciais de operações feitas por outro cliente (**isolamento**)

Motivação para transações

- Qual a diferença entre transações e mecanismos de sincronização vistos em aula (como mutex, semáforos e monitores)?
 - Mecanismos vistos atendem somente o problema de acessos concorrentes
- Transações atendem 3 problemas:
 - Tratamento de acessos concorrentes (múltiplos clientes)
 - Tratamento de falhas lógicas em caso de objetos persistentes
 - Tratamento de falhas de hardware
- Exemplo clássico (para falhas)
 - Operação bancária para remessa de fundos
 - Envolve 2 sub-operações distintas
 - Retirada da conta de origem
 - Depósito na conta de destino
 - Implementação distribuída deve prever
 - Falha em uma ou outra operação
 - Se falhar, tudo deve ser cancelado; se a 1ª já foi confirmada, desfazer

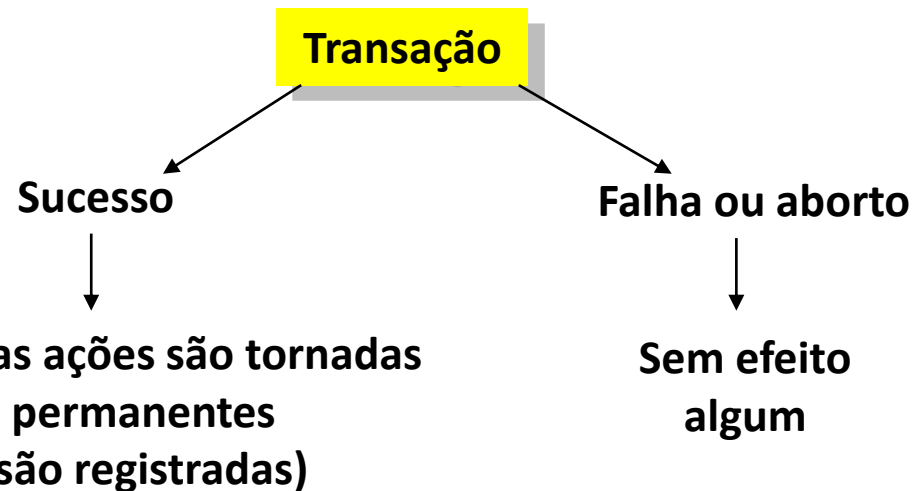
Transação

- Em algumas situações, clientes necessitam que uma sequência de requisições independentes sejam **atômicas**
 - Não sofrem interferência de operações executadas por outros processos
 - Ou todas as operações completam com sucesso, ou elas não devem produzir efeito algum na presença de falhas
 - Pode envolver vários servidores → transações distribuídas
 - Pode ser implementada por framework, componente, serviço, middleware, ...
- Propriedades de uma transação
 - **Atômica** (noção de “tudo ou nada”)
 - Todas operações devem ser concluídas com êxito, ou não devem ter nenhum efeito na presença de falhas (ou cancelamento)
 - **Durabilidade**: se transação completar com sucesso, seus “efeitos” são salvos em armazenamento permanente (e.g., disco)
 - **Isolamento**: uma transação não pode observar os efeitos parciais de outras transações (transações disparadas por clientes)

Propriedade A.C.I.D.

Propriedades A.C.I.D. da transação

- Como obter Atomicidade, Isolamento e Durabilidade?
 - Atomicidade e durabilidade → objetos recuperáveis
 - Recuperar estado dos objetos após uma falha do servidor
 - Armazenamento permanente de forma a refletir o efeito de tudo ou nada
 - Isolamento → sincronizar as operações
 - Noção de equivalência serial
 - Transações devem poder executar de forma concorrente desde que gerem o mesmo resultado de execução serial
 - Objetivo é maximizar a concorrência
- E sobre o “C” de “ACID”? **Todas as ações são tornadas permanentes (são registradas)**
 - Consistência
 - Garantir invariantes do sistema
 - Responsabilidade da lógica da transação



Qual o “valor agregado” de transações?

- O que agregam em relação ao já visto (mutex, semáforos, monitores)?
 - Recuperação
 - Na presença de falhas, restaura o estado dos objetos a um valor prévio consistente
 - Maximização de concorrência
 - Transações devem poder executar de forma concorrente

Coordenação de transações

- Transações são criadas e gerenciadas por um coordenador
 - Implementado por um framework, componente, serviço, middleware, etc
- Criação e gerenciamento de cada transação
 - Tem um identificador único (*transaction ID* - TID)
 - Primitivas básicas: *open*, *close* e *abort*
- Implementação:
 - Ao abrir uma transação se obtém um *TID* que deve ser informado a cada operação
 - Transação completa quando cliente chama `closeTransation()` ... ou `abortTransation()`

- `openTransation( )` → TID
 `ação_1(TID, argumentos)`

 `ação_n(TID, argumentos)`
- `closeTransaction(TID)`

Opcionalmente:

- `abortTransaction(TID)`

Importante: servidor também pode decidir abortar a transação

Há a noção de efetivação (*commit*) associada ao `closeTransaction`

Exemplos de transação

Operação normal

```
TID = openTransaction( );  
op_1(TID, argumentos);  
op_2(TID, argumentos);  
...  
op_n(TID, argumentos);  
closeTransaction(TID);
```

Abortada pelo cliente

```
TID = openTransaction( );  
op_1(TID, argumentos);  
op_2(TID, argumentos);  
...  
op_n(TID, argumentos);  
abortTransaction(TID);
```

Abortada pelo servidor

```
TID = openTransaction( );  
op_1(TID, argumentos);  
op_2(TID, argumentos);  
...  
op_n(TID, argumentos);  
→ ERRO na operação  
→ Notifica o cliente
```

Informa ao cliente que deve tomar
uma ação apropriada

Fases de uma transação

Fase 1:

Operações são realizadas e registradas de forma a permitir que sejam confirmadas ou desfeitas.

Fase 2:

Modificações decorrentes das operações são **concretizadas** ou **desfeitas**.

openTransaction()



Execução de operações que fazem parte da transação



closeTransaction() OU

abortTransaction()



Commit()

Modificações realizadas são tornadas permanentes



Abort()

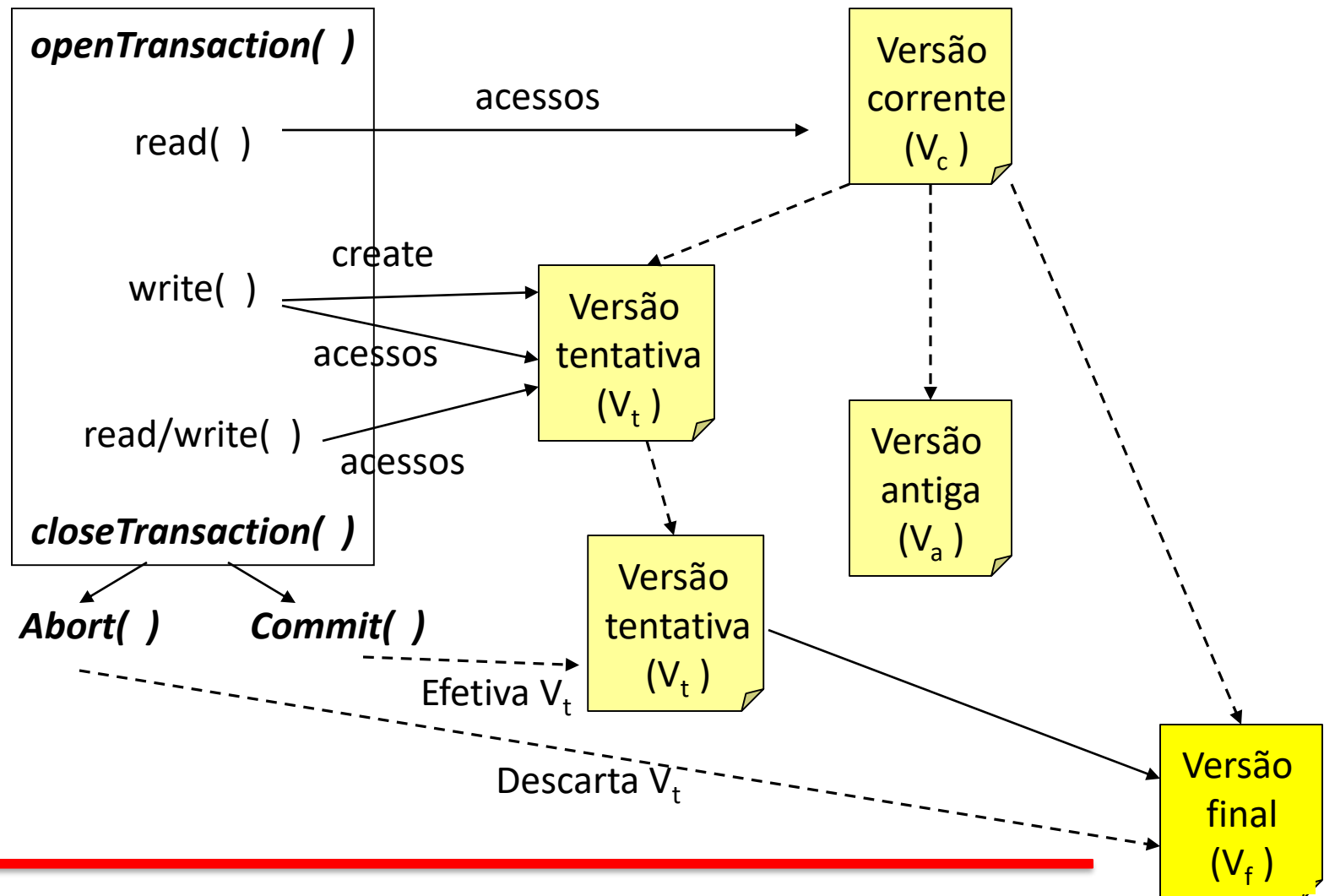
Modificações realizadas não são concretizadas

Abort pode ser feito pelo cliente ou pelo servidor!

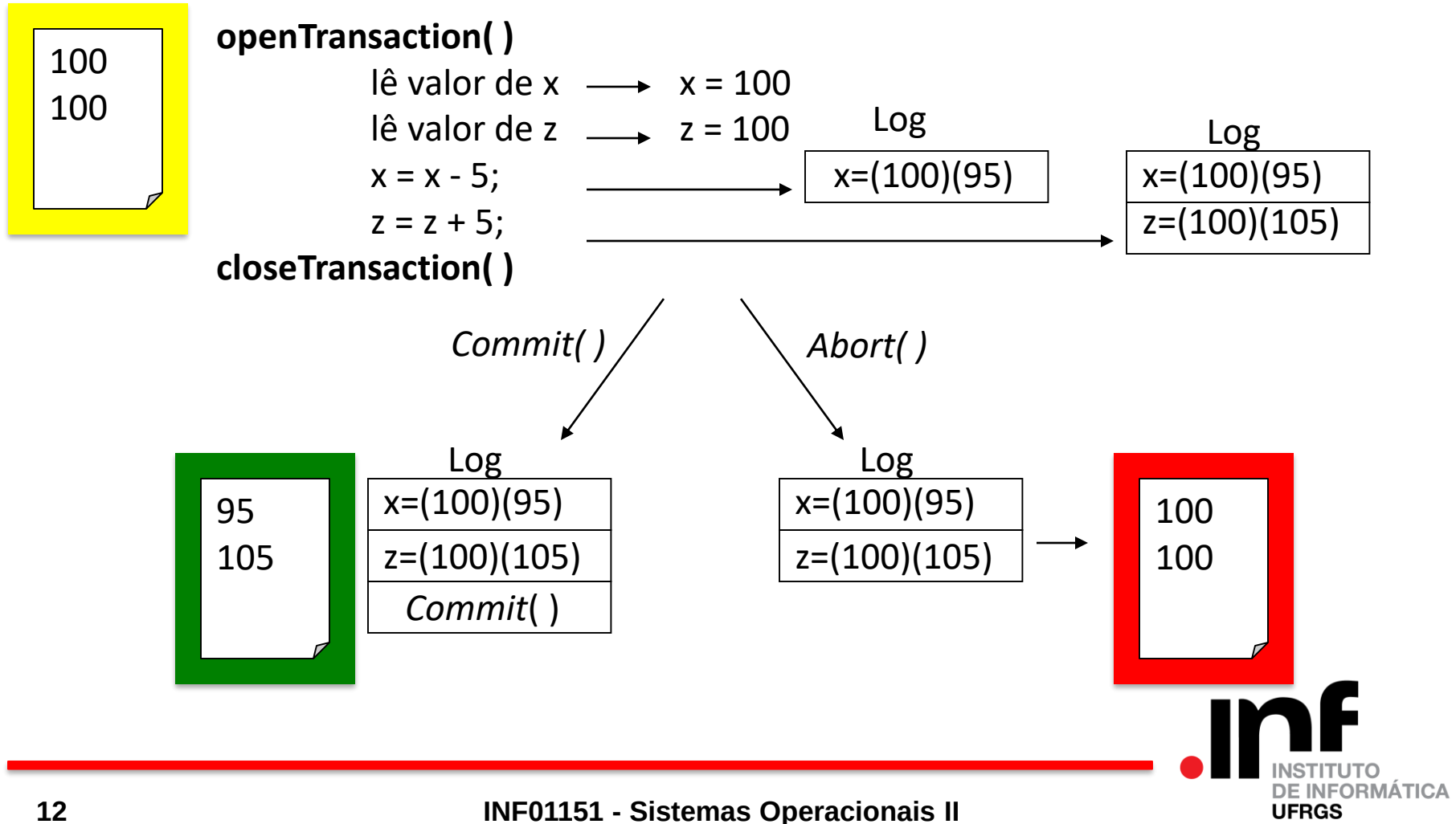
Aspectos de implementação de transações

- Mecanismos de cópias privadas (versionamento)
- Técnica de journalização (logs)

Cópias privadas (versionamento)



Técnica de journalização (*logs*)



Falhas durante transações

- Processo servidor falha
 - Substituição por um novo processo servidor
 - Aborta transações não confirmadas (ou seja, não “committed”)
 - Restaura o estado dos objetos de transações “committed”
 - Processos clientes recebem exceção por timeout ou são avisados que o TID da transação não é mais válido
 - Cliente abandona ou refaz a transação (geralmente com envolvimento humano)
- Processo cliente falha
 - Transações iniciadas no servidor tem tempo de expiração
 - Transações expiradas são abortadas

Controle da concorrência

- Dois problemas bem conhecidos em transações concorrentes:
 - **Atualizações perdidas:**
 - Ocorre quando duas (ou mais) transações lêem o valor antigo de uma variável e depois o utilizam para calcular um novo valor
 - Similar a problema de incremento concorrente duplo em variável compartilhada
 - ✓ Causa: leitura concorrente de valor anterior
 - **Consulta inconsistente**
 - Quando uma transação de consulta (leitura) é executada concorrentemente com uma transação de atualização (escrita)
 - ✓ Causa: série de leituras concorrentes a escritas

Exemplo: transações bancárias (API)

- Operações da interface *conta* (*account*)
 - **deposit(amount)**
 - Deposita **amount** na conta
 - **withdraw(amount)**
 - Retira **amount** da conta
 - **getBalance()** -> **amount**
 - Retorna o saldo da conta: **amount**
 - **setBalance(amount)**
 - Altera o saldo da conta; novo valor = **amount**

Exemplo: transações bancárias (API)

- Operações da interface *agência* (*branch*)
 - **create(name) -> account**
 - Cria nova conta com nome **name**
 - **lookUp(name) -> account**
 - Retorna uma referência a uma conta de nome **name**
 - **branchTotal() -> amount**
 - Retorna o saldo acumulado de todas as contas da agência

Problema da atualização perdida

Condição inicial: conta A = \$100; Conta B = \$200 e Conta C = \$300

Problema: realizar saques nas contas A e C de forma a creditar 10% da conta B na própria (valor final de B = \$ 242,00; valor final de A = \$ 80; valor final de C=\$ 278,00)

Transaction <i>T</i> :	Transaction <i>U</i> :
<i>balance = b.getBalance();</i> <i>b.setBalance(balance*1.1);</i> <i>a.withdraw(balance/10)</i>	<i>balance = b.getBalance();</i> <i>b.setBalance(balance*1.1);</i> <i>c.withdraw(balance/10)</i>
<i>balance = b.getBalance();</i> \$200	
	<i>balance = b.getBalance();</i> \$200
	<i>b.setBalance(balance*1.1);</i> \$220
<i>b.setBalance(balance*1.1);</i> \$220	
<i>a.withdraw(balance/10)</i> \$80	
	<i>c.withdraw(balance/10)</i> \$280

Problema de consulta inconsistente

Condição inicial: conta A = \$200; Conta B = \$200 e Total = \$400

Situação: realizar uma transferência de \$100 da conta A para a conta B e obter o valor total depositado na agência

Transaction V:

a.withdraw(100)

b.deposit(100)

a.withdraw(100); \$100

b.deposit(100) \$300

Transaction W:

aBranch.branchTotal()

total = a.getBalance() \$100

total = total+b.getBalance() \$300

total = total+c.getBalance()

•

Equivalência serial

- O que é:
 - Critério para execução concorrente correta, de forma a **evitar os problemas** de atualizações perdidas e recuperações inconsistentes
- Princípio:
 - Se for sabido que cada uma das transações tem o efeito correto quando executada sozinha,
 - Então é possível inferir que se essas transações forem executadas uma por vez, em alguma ordem, o efeito combinado também será correto
 - Intercalação de operações na qual a combinação dos efeitos é o mesmo que como as transações tivessem sido executadas uma de cada vez **em uma determinada ordem**

Resultado final de duas ou mais transações simultâneas deve ser igual ao obtido pela execução sequencial delas (qualquer ordem)

Equivalência serial entre T e U

Transaction T:	Transaction U:
<i>balance = b.getBalance()</i> <i>b.setBalance(balance*1.1)</i> <i>a.withdraw(balance/10)</i>	<i>balance = b.getBalance()</i> <i>b.setBalance(balance*1.1)</i> <i>c.withdraw(balance/10)</i>

balance = b.getBalance() \$200

*b.setBalance(balance*1.1)* \$220

a.withdraw(balance/10) \$80

balance = b.getBalance() \$220

*b.setBalance(balance*1.1)* \$242

c.withdraw(balance/10) \$278

Equivalência serial entre V e W

Transaction V:	Transaction W:
<i>a.withdraw(100);</i> <i>b.deposit(100)</i>	<i>aBranch.branchTotal()</i>

a.withdraw(100); \$100

b.deposit(100) \$300

total = a.getBalance() \$100

total = total+b.getBalance() \$400

total = total+c.getBalance()

...

Operações conflitantes

- Duas operações são ditas conflitantes quando o efeito combinado delas depende da ordem em que são executadas

Operações		Conflito	Motivo
read	read	não	Duas operações de read independem da ordem em que são executadas
read	write	sim	O valor do read depende se é feito antes ou depois de uma escrita
write	write	sim	O valor final depende da ordem em que os writes são feitos

Para equivalência serial é necessário e suficiente que:

Todos os pares de operações conflitantes entre duas transações sejam executados na mesma ordem em todos os objetos que ambas acessam

Operações conflitantes

- Considere duas transações T e U:
 - T: `x=read(i); y=read(j); write(k,x+y);`
 - U: `write(j,10); write(i,20);`
- Os pares de operações conflitantes são os seguintes:
 - `x=read(i)` e `write(i,20);`
 - `y=read(j)` e `write(j,10);`

T	U
	<code>write(j, 10);</code>
<code>x = read(i);</code>	
	<code>write(i, 20);</code>
<code>y = read(j);</code>	
<code>write(k,x+y)</code>	

NÃO é serialmente equivalente

T	U
<code>x = read(i);</code>	
<code>y = read(j);</code>	
	<code>write(j, 10);</code>
	<code>write(i, 20);</code>
<code>write(k,x+y)</code>	

É serialmente equivalente

Estratégias para controle de concorrência

- Travas (*locks*) e bloqueio
 - Transação (TID) obtém direito de acesso exclusivo sobre um objeto
 - Outras transações devem esperar (ou em alguns casos, compartilhar o lock)
 - É a idéia da exclusão mútua considerando ocorrência de falhas
- Controle otimista
 - Transação executa sempre até que necessite ser efetivada (committed)
 - Verifica se ela executou operações conflitantes, em caso afirmativo, cancela a transação
- Indicação de tempo (*timestamp*)
 - Registra o tempo de leitura e de escrita mais recentes de cada objeto
 - Para cada operação, timestamp da transação é comparado com timestamp do objeto
 - para determinar se operação é efetivada, retardada ou rejeitada (cancelada)

**Basicamente todas forçam, de alguma maneira,
a equivalência serial das operações**

Locks

- Princípio básico:
 - Proibir o acesso (read/write) de diferentes processos através de *locks* no recurso
 - Idéia é que seja feito automaticamente pelo sistema de transações
 - Transações precisam adquirir determinado lock antes de prosseguir
 - Por exemplo, lock para “account B”
- Melhorias:
 - Distinguir os *lock* de escrita daqueles de leitura
 - *Read lock* autoriza outros *read locks*
 - *Write lock* proíbe qualquer outro tipo de *lock* (bloqueios)

Problemas com locks

- Locks por demanda podem levar a *deadlocks* ou inconsistências
 - Para prevenir deadlocks
 - Adquirir locks em uma determinada ordem
 - Adquirir todos os locks necessários de uma só vez
 - Para detectar deadlocks
 - Modelados utilizando grafos de alocação de recursos
 - Representação de transações esperando por um determinado lock
 - Identificação de ciclos → Abortar transação!
- Granularidade do *lock*
 - Lock limita severamente a concorrência
 - Arquivo vs. registro vs. objeto
- Overhead
 - Mesmo transações read-only precisam em geral utilizar locks
 - Garantir que dado lido não está sendo modificado por outra transação

Controle otimista

- Utilizado quando chance de problemas de conflito é pequena
 - Não introduz controles para evitar, mas sim para resolver
 - Quando conflito acontece, alguma transação é geralmente abortada
- Baseado em três etapas:
 - **Trabalho:** cada transação executa suas operações sobre uma cópia privada – *tentative version* – dos objetos (versionamento)
 - Quando houver múltiplas transações concorrentes, podem existir múltiplas cópias
 - **Validação:** verifica se modificações ocorridas no objeto são conflitantes com operações feitas por outras transações
 - Se não, transação é válida. Se sim, transação é abortada
 - **Atualização:** corresponde ao *commit*
 - Alterações na versão tentativa são tornadas permanentes

✓ Vantagens (em relação a locks):

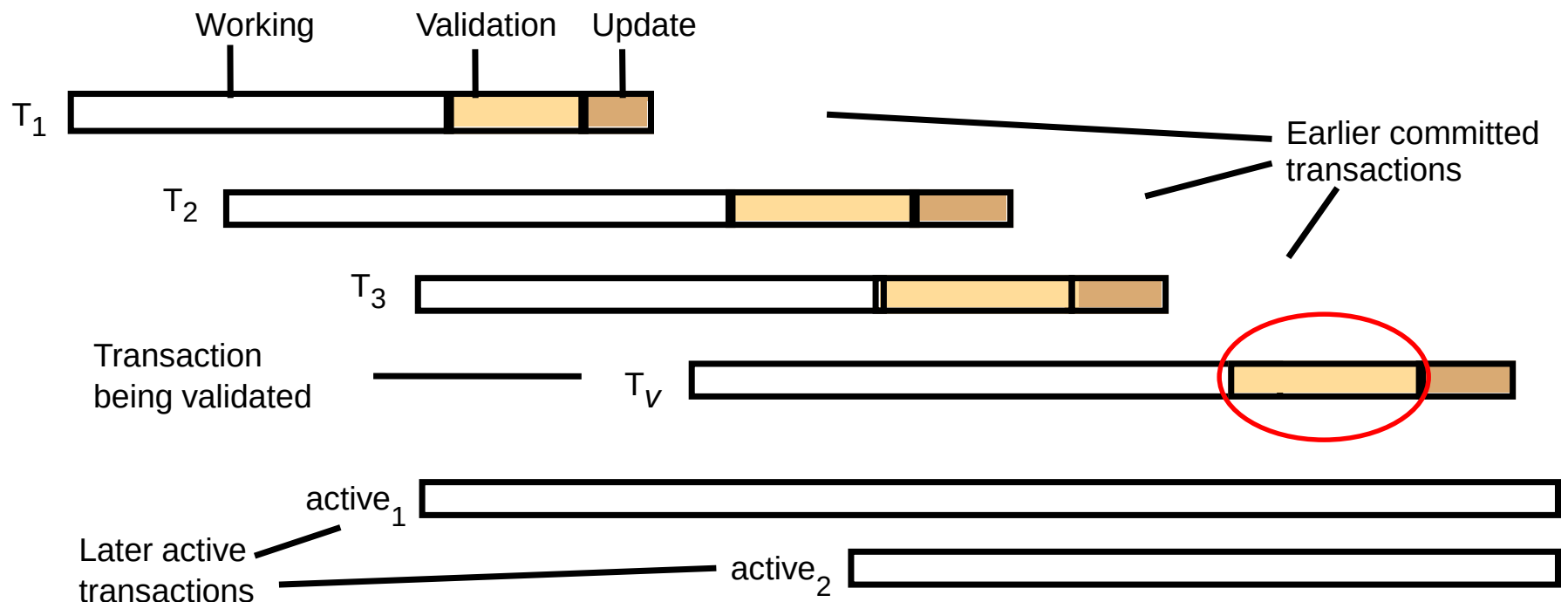
- Livre de deadlock
- Mais paralelismo, menos overhead

✓ Desvantagem:

- “Desfazer” operações em caso de conflito

Controle otimista: validação

- Backward validation vs. Forward validation



Controle otimista: validação

- Backward validation
 - Nenhum dos *reads* de transações mais antigas (e sobrepostas) podem ser afetados
 - Apenas compara *reads* de T_v com *writes* de transações passadas (sobrepostas)

Backward validation of transaction T_v

```
boolean valid = true;
for (int  $T_i$  = startTn+1;  $T_i$  <= finishTn;  $T_i$ ++) {
    if (read set of  $T_v$  intersects write set of  $T_i$ ) valid = false;
}
```

- Forward validation
 - Apenas compara os *writes* de T_v com os *reads* de transações ativas (sobrepostas)
 - Se T_v for transação read-only, sempre passará por essa validação

Forward validation of transaction T_v

```
boolean valid = true;
for (int  $T_{id}$  = active1;  $T_{id}$  <= activeN;  $T_{id}$ ++) {
    if (write set of  $T_v$  intersects read set of  $T_{id}$ ) valid = false;
}
```

Carimbo de tempo (*timestamp*)

- Cada openTransaction recebe um *timestamp*
 - Relógio de Lamport (valores únicos)
 - Define sua posição no tempo na sequência de transações
- Objetos dentro da transação
 - São rastreados com um `read_timestamp` e um `write_timestamp`
 - Cada operação é validada no momento de sua execução
- As transações mais antigas tem preferência sobre as mais novas
 - Em caso de conflito a mais nova é abortada
 - As alterações são desfeitas
 - E usuário eventualmente deve ser avisado

Regra para ordenamento: requisição de escrita em um objeto é válida somente se o objeto foi lido e escrito apenas por transações mais antigas, e requisição para ler um objeto é válida apenas se o objeto foi escrito por transações mais antigas.

Conflito de operações para ordenação de timestamp

Regra	T_c	T_i	
1	escrita	leitura	T_c não deve <i>escrever</i> um objeto que tenha sido <i>lido</i> por qualquer T_i onde $T_i > T_c$; isso exige que $T_c \geq$ a indicação de tempo de leitura máxima do objeto.
2	escrita	escrita	T_c não deve <i>escrever</i> um objeto que tenha sido <i>modificado</i> por qualquer T_i onde $T_i > T_c$; isso exige que $T_c >$ indicação de tempo de escrita do objeto committed.
3	leitura	escrita	T_c não deve <i>ler</i> um objeto que tenha sido <i>modificado</i> por qualquer T_i onde $T_i > T_c$; isso exige que $T_c >$ indicação de tempo de escrita do objeto committed.

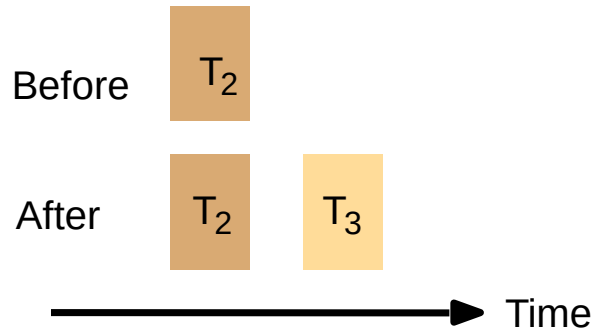
Regra de escrita

Operação de escrita requisitada por transação T_c em objeto D :

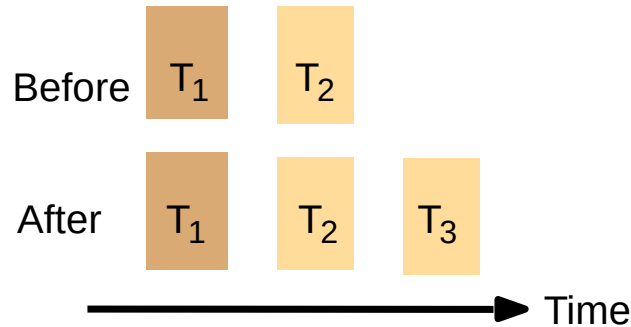
```
if ( $T_c \geq$  maximum read_timestamp on  $D$  &&  
     $T_c >$  write_timestamp on committed version of  $D$ )  
    perform write operation on tentative version  
    of  $D$  with write_timestamp  $T_c$   
else  
    /* write is too late */  
    Abort transaction  $T_c$ 
```

Exemplo de escrita: com $T_3 \geq T_{\text{leitura}}$

(a) T_3 write



(b) T_3 write



Key:

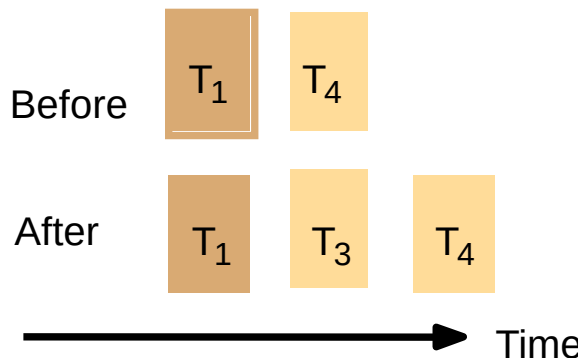


Committed

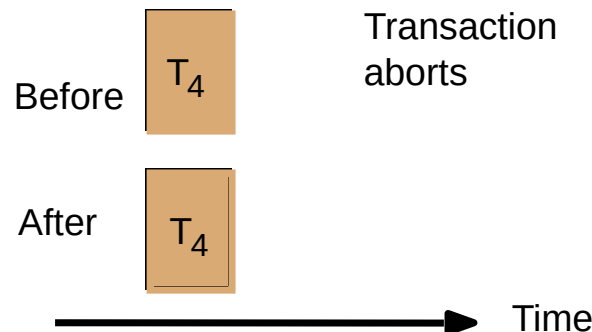


Tentative

(c) T_3 write



(d) T_3 write



object produced
by transaction T_i
(with write timestamp T_i)

$$T_1 < T_2 < T_3 < T_4$$

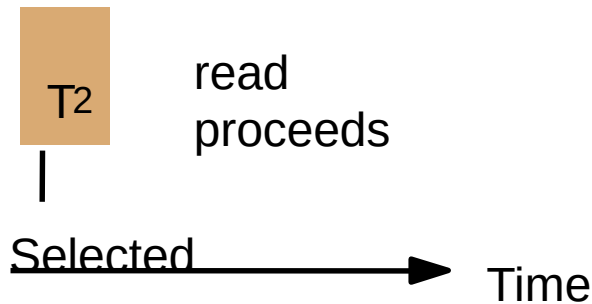
Regra de leitura

Operação de leitura requisitada por transação T_c em objeto D :

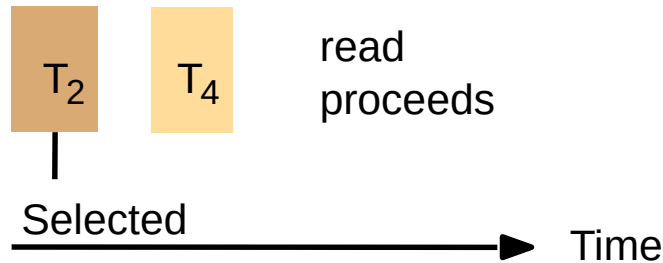
```
if ( $T_c > \text{write\_timestamp}$  on committed version of  $D$ )
{
    let  $D_{\text{selected}}$  be the version of  $D$  with the maximum  $\text{write\_timestamp} \leq T_c$ 
    if ( $D_{\text{selected}}$  is committed)
        perform read operation on the version  $D_{\text{selected}}$ 
    else
        Wait until the transaction that made version  $D_{\text{selected}}$  commits
        or aborts then reapply the read rule
}
else
    Abort transaction  $T_c$ 
```


Exemplo de leitura

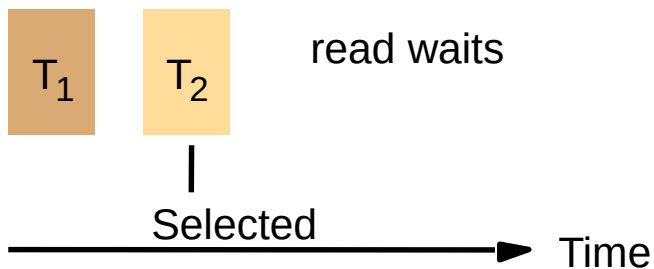
(a) T_3 read



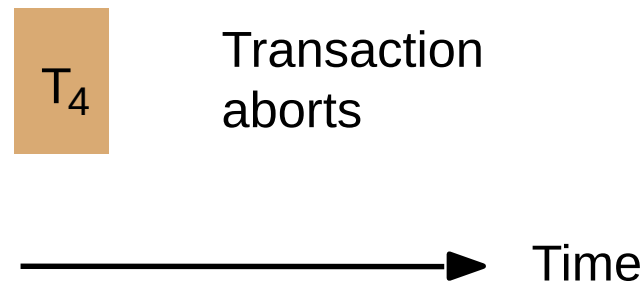
(b) T_3 read



(c) T_3 read



(d) T_3 read



Key:



Committed



Tentative

object produced
by transaction T_i
(with write timestamp T_i)
 $T_1 < T_2 < T_3 < T_4$

Leituras adicionais

- Coulouris, G.; Dollimore, J.; Kindberg, T. and Blair, G.—
“*Distributed Systems: Concepts and Design*” (5th edition),
Addison Wesley, 2012.
 - Capítulo 16

INF01151 – Sistemas Operacionais II

Aula 25 - Replicação

Prof. Alberto Egon Schaeffer Filho

Introdução

- Replicação é a manutenção de várias cópias (réplicas) dos dados ou serviços em vários computadores distribuídos
 - Implica réplica de servidores utilizadas para “melhorar” um serviço
- Vantagens da replicação
 - Melhor desempenho
 - Aumento da disponibilidade
 - Tolerância a falhas
- Como replicação é feita?
 - Replicação passiva, replicação ativa, ...

① Replicação para desempenho

- Princípio de funcionamento
 - Cópias extras podem permitir dividir workload entre múltiplos servidores
 - Cópias extras permitem que dados fiquem disponíveis mais perto dos clientes
- Replicação de dados
 - Imutáveis
 - Trivial, pois não há necessidade de atualizações
 - Aumenta o desempenho com custo relativamente baixo
 - Dinâmicos
 - Necessidade de mecanismos (protocolos) para garantir que os clientes recebam dados atualizados
- Tipicamente é boa quando há mais *reads*
 - *Read* pode ser feito a partir da melhor cópia
 - *Writes* devem ir para todas as cópias

② Replicação para disponibilidade

- Usuários necessitam ter sistemas com alta disponibilidade
 - Proporção de tempo no qual um serviço (dados) é disponível a usuários dentro de um tempo razoável (ideal é 100%!!)
- Além do tempo gasto no controle de concorrência (ex., devido a locks), outros aspectos afetam a disponibilidade
 - Particionamento da rede e desconexões
 - Falha no servidor
- Múltiplas cópias podem garantir que falhas não impossibilitem utilização do sistema
 - Possível aumentar a disponibilidade através de replicação
 - Ex.: n servidores, probabilidade de falha no servidor igual a p , portanto, a disponibilidade do serviço é dada por $1-p^n$
 - com $p=5\%$, então 1 servidor fornece 95%, 2 servidores 99,75%, etc

③ Replicação para tolerância a falhas

- Alta disponibilidade é interessante, mas não implica em dados necessariamente corretos
 - Desatualização
 - Atualizações conflitantes em caso de particionamento da rede
 - Usuários em lados opostos de uma rede particionada fazem atualizações conflitantes que precisam ser resolvidas
 - Servidores podem fazer coisas erradas, proteção contra falhas bizantinas
- Serviço tolerante a falhas
 - Garante um comportamento correto mesmo na presença de um certo tipo e número de falhas
 - Se f em $f+1$ servidores falham (colapso), então em princípio, pelo menos uma cópia permanece para atender o serviço
 - Se f falham de forma bizantina, $2f+1$ servidores oferecem serviço correto, se os servidores corretos concordarem na resposta (votação)

Modelo de replicação: requisitos

- Requisitos comuns de um modelo de replicação de dados
 - **Transparência**: clientes não vêem que existem múltiplas cópias físicas
 - Mesmo que existam múltiplas cópias físicas, cliente só conhece um único objeto lógico
 - Clientes apenas requisitam operações a esse objeto lógico
 - Clientes esperam por apenas um resultado (mesmo que operação tenha executado em mais de uma cópia)
 - **Consistência**: múltiplas cópias devem estar coerentes
 - Dependente da aplicação (pode ser mais restrito ou menos restrito)
 - Relação custo x benefício
 - Inconsistência temporária é uma possibilidade
- Objetivo do sistema distribuído
 - Distribuir updates para todas as réplicas
 - Direcionar requisições para a réplica mais “conveniente”

Modelo de replicação

- Modelo básico:
 - Dados são conjuntos de itens chamados de objetos (arquivo, objeto Java, etc)
 - Cada objeto lógico é formado por um conjunto de cópias físicas (réplicas)
 - Réplicas não são necessariamente idênticas/consistentes
 - Algumas podem ter sido atualizadas enquanto outras não
 - Gerenciadores (replica managers) aplicam operações em réplicas
 - De forma recuperável
 - De forma atômica
 - Resultado depende
 - Do estado da réplica
 - Do ordenamento das operações
- Replicação pode ser utilizada em combinação com **transações**

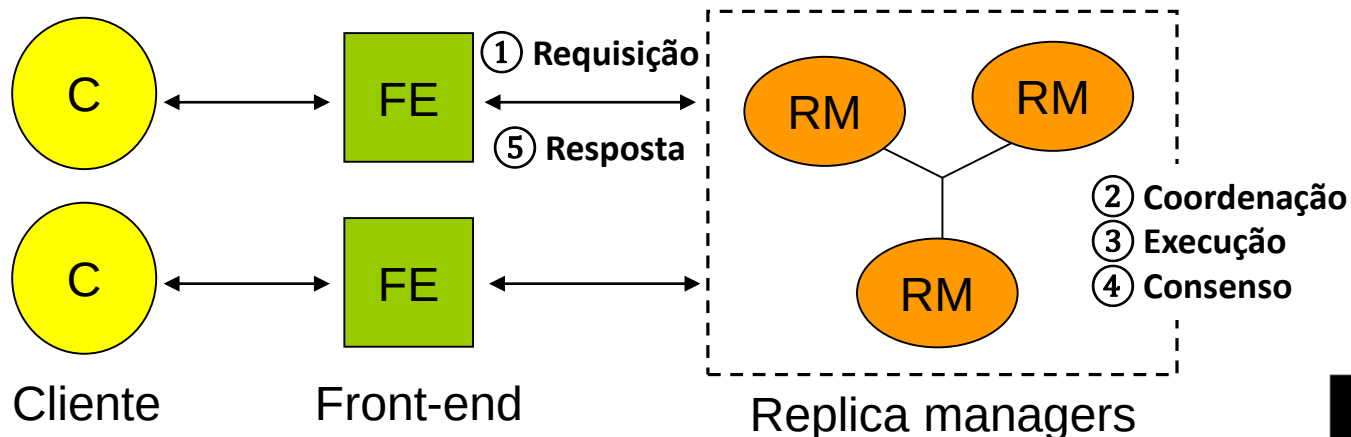
Modelo (genérico) de replicação

- Arquitetura básica:

Podem ser satisfeitas por qualquer réplica

Precisam ser propagadas para todas as réplicas

- **Clientes (C):** fazem requisições (*read-only* ou *update requests*)
- **Front-end (FE):** fornece transparência aos clientes encaminhando requisições aos RMs
- **Replica managers (RMs):** executam as operações nas réplicas de forma atômica
 - Se aplicado em ambiente cliente-servidor, cada RM é um servidor
 - Considera um número estático ou dinâmico de RMs



Cinco etapas (genérico)

1. **Requisição**: envio da requisição do FE a um ou mais RMs
 - Para um RM primário, que se comunicará com os demais; ou
 - Em *multicast* para os RMs
2. **Coordenação**: RMs definem se a requisição deve ser processada, e a ordem de execução de requisições entre si
 - Ordem FIFO: baseado na ordem dos requests em um dado FE
 - Ordem causal: baseado na ordenação happened-before
 - Ordem total: baseado na mesma ordem em todos os RMs
3. **Execução**: realiza requisição de forma *tentativa*, de modo que essa possa ser desfeita mais tarde se necessário (*transação*)
4. **Consenso**: RMs efetivam (*commit*) ou abortam (*abort*) a execução
5. **Resposta**: um ou mais RMs respondem ao FE
 - Primeira que chega
 - Resposta da maioria (falha bizantina)

Noções de comunicação em grupo

Implementação da replicação

- Serviço baseado em replicação: ***visão idealista***
 - Serviço continua respondendo apesar de eventuais falhas
 - Clientes não conseguem distinguir se serviço é obtido de implementação com dados replicados ou de uma única instância
- Considerando uma solução trivial (e ingênua)
 - Dois gerenciadores de réplica A e B que mantêm cópias de duas contas bancárias x e y
 - Clientes lêem e atualizam as contas em um gerenciador de réplica e tentam o outro em caso de problemas
 - Gerenciadores de réplica propagam as informações de atualização em background, após responder para os clientes
 - As contas inicialmente tem saldo zero

Situação exemplo

- Cliente 1
 - Atualiza o saldo da conta x para \$1 no gerenciador de réplica B (que falha logo em seguida)
 - Atualiza o saldo da conta y para \$2, mas gerenciador de réplica B não responde
 - Executa então no gerenciador de réplica A
- Cliente 2
 - Lê o saldo das contas x e y no gerenciador de réplica A

Cliente 1	Cliente 2
setBalance _B (x, 1)	
setBalance _A (y, 2)	
	getBalance _A (y) → 2
	getBalance _A (x) → 0

Comportamento anômalo não teria acontecido se as contas de banco tivessem sido implementadas em um servidor único

Linearizability vs. Sequential consistency

- Linearizability

- Sistema distribuído que atua exatamente como um único servidor gerenciando uma única cópia é *linearizable*
- Deve satisfazer dois critérios para qualquer execução
 1. Série de operações intercaladas atende a especificação de uma única cópia correta
 2. Intercalação é consistente com os tempos reais em que as operações ocorreram na execução
- Inclui noção de tempo real, o que é difícil de obter no mundo real

- Sequential consistency

- Garantir que as operações ocorrem de forma correta e em determinada ordem
 - Sem uso de relógios globais
- Deve satisfazer dois critérios para qualquer execução
 1. Série de operações intercaladas atende a especificação de uma única cópia correta
 2. Intercalação é consistente com a ordem do programa executado por cada cliente em específico (intercalações podem embaralhar sequencia de operações de um conjunto de clientes em qualquer ordem, desde que a ordem das operações de cada cliente seja mantida)
- Cada cliente vê as operações na ordem em que ele fez elas

Linearizability vs. Sequential consistency

Linearizable

Cliente 1	Cliente 2
	$\text{getBalance}_A(y) \rightarrow 0$
	$\text{getBalance}_A(x) \rightarrow 0$
$\text{setBalance}_B(x, 1)$	
$\text{setBalance}_A(y, 2)$	

Linearizable $\Rightarrow$ sequentially consistent

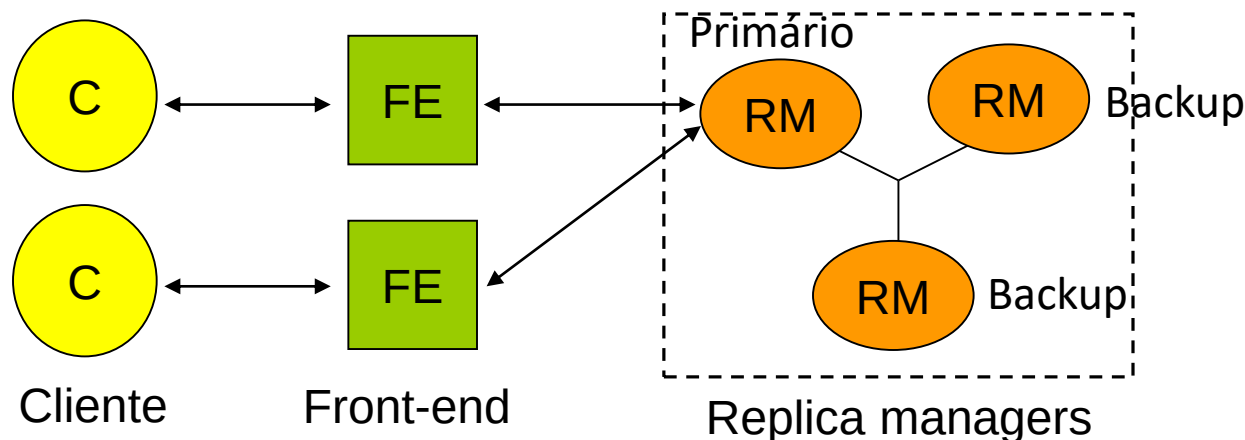
Sequentially consistent, mas não linearizable

Cliente 1	Cliente 2
$\text{setBalance}_B(x, 1)$	
	$\text{getBalance}_A(y) \rightarrow 0$
	$\text{getBalance}_A(x) \rightarrow 0$
$\text{setBalance}_A(y, 2)$	

- Valor de x consultado depois que ele foi alterado para 1!
- Sequentially consistent pois:
 - Estado que cliente 2 vê realmente existiu antes do cliente 1 iniciar sua operação
 - Cliente 2 "poderia" ter executado suas operações antes do cliente 1 ter iniciado

Replicação passiva

- Replicação passiva ou *backup*-primário
 - Um único gerenciador de replica (RM) primário
 - Um ou mais gerenciadores de réplicas secundários (backup ou escravos)
- FE comunica somente com o RM primário para executar serviço
 - RM primário executa as operações e envia atualizações aos secundários
 - Se o primário falha, um secundário assume (eleição)



Replicação passiva: etapas (1-5)

- **Requisição:**
 - FE envia a requisição, contendo um identificador único, ao RM primário
- **Coordenação:**
 - Primário trata requisições de forma atômica, na ordem em que as recebe
 - Verifica o identificador único, em caso de já ter executado, simplesmente reenvia a resposta
- **Execução:**
 - Realiza a requisição e armazena o resultado
- **Consenso:**
 - Se a requisição atualizar algum dado, o primário envia o estado atualizado, a resposta e o identificador único para todos os secundários
 - Espera confirmação dos secundários
- **Resposta:**
 - Primário envia resposta ao FE, que a encaminha ao cliente

Replicação passiva: linearizability

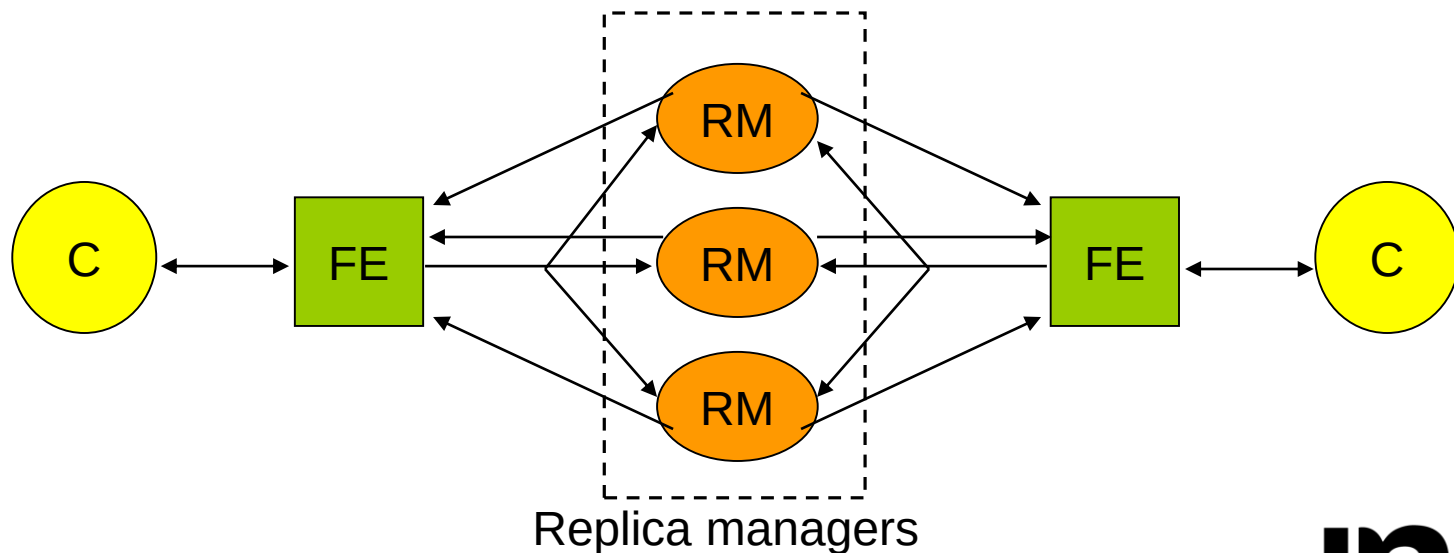
- Claramente satisfaz linearizability se não houver falhas
 - Primário realiza operações em “tempo real” e em ordem
 - Não há preocupação em atualizar em múltiplos lugares
- E se houver falhas?
 - Ainda satisfaz linearizability se:
 - Um único backup se tornar o novo primário
 - Os outros replica managers sobreviventes concordam em quais operações haviam sido realizadas antes da falha
- Possível utilizar características da comunicação em grupo para satisfazer essas características

Replicação passiva: comentários

- Aspectos de tolerância a falhas
 - Necessita $f+1$ réplicas para sobreviver f falhas por colapso
 - FE apenas precisa ser capaz de localizar novo primário
 - Não tolera falhas bizantinas!
 - Falha bizantina no primário causa caos em todo o sistema
 - Replicação passiva exige comportamento correto do primário

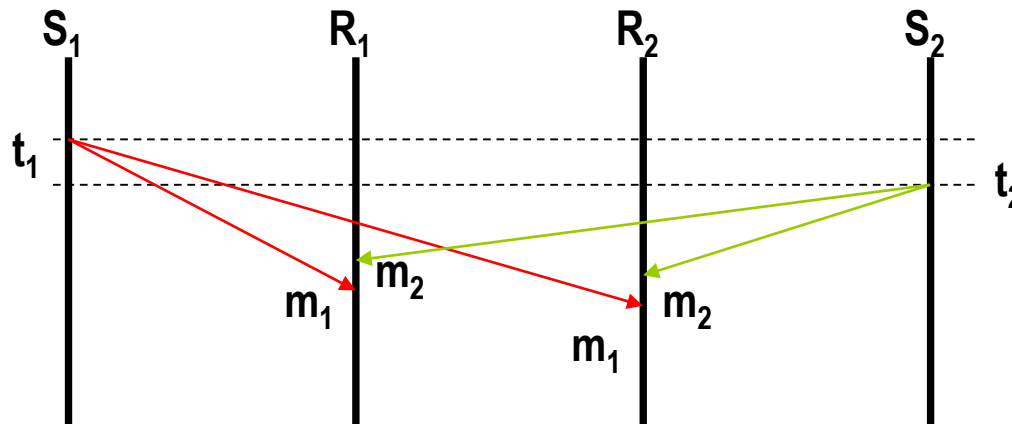
Replicação ativa

- FEs fazem *multicast* da requisição para o grupo de RMs
 - RMs executam as requisições de forma independente e respondem ao FE
 - RMs são todos equivalentes
 - Se qualquer RM sofrer colapso, não há impacto no desempenho, e RMs restantes irão responder da forma esperada
 - Garantia de ordem e funcionamento é o da semântica de *multicast*



Multicast totalmente ordenado

- Se um processo $p \in g$ executa $deliver(m)$ antes de $deliver(m')$, então qualquer outro processo $q \in g$ entregará m' após m
 - Garantia que todas as mensagens são entregues na mesma ordem em todos os processos mesmo que difira da ordem de emissão



Replicação ativa: etapas (1-3)

- **Requisição:**

- FE cria uma identificação única para as requisições e envia (*multicast*) para o grupo de gerenciadores de réplica
 - Multicast **totalmente ordenado** e **confiável**

- **Coordenação:**

- Semântica do multicast garante que todas as réplicas receberam a requisição na mesma ordem (total)

- **Execução:**

- Cada RM executa a requisição, e como todas são recebidas na mesma ordem, todos RMs ativos (e corretos) processam a requisição da mesma forma

Replicação ativa: etapas (4-5)

- **Consenso:**
 - Não há necessidade de consenso devido a semântica de multicast empregada
- **Resposta:**
 - Cada RM envia a resposta ao FE
 - Número de respostas coletadas pelo FE depende das condições de falhas previstas
 - Opções?

Replicação ativa: sequential consistency

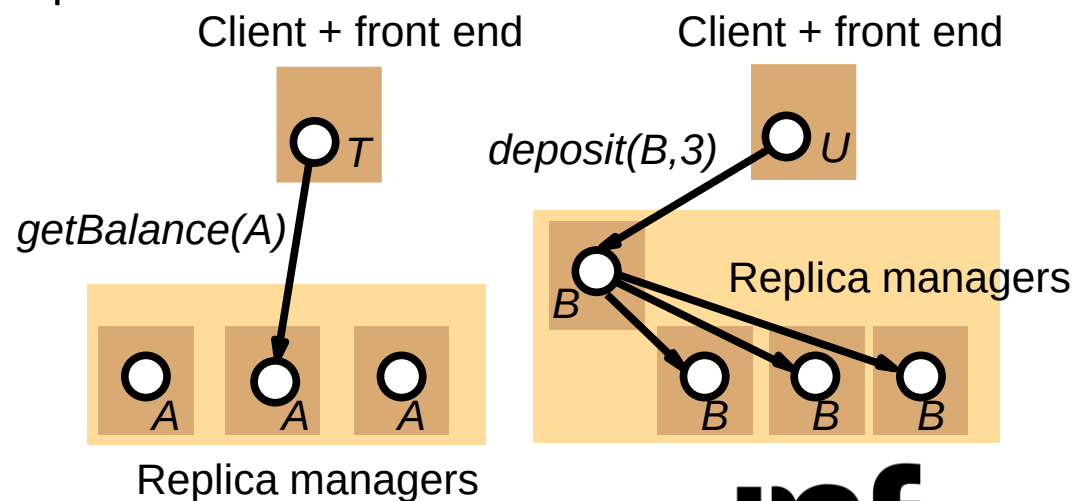
- Replicação ativa não suporta linearizability
 - Ordem total em que os RMs executam as requisições não é necessariamente consistente com a ordem em tempo real que clientes fizeram requisições
- Replicação ativa suporta sequential consistency
 - Requisições feitas em cada FE são processadas em ordem, uma por vez
 - Todas os RMs corretos processam a mesma sequência de requests

Atualização de réplicas: outras técnicas

- Protocolos disponíveis
 - Read-only
 - Read-one/write-all
 - Primary-copy
 - Quorum based

Read-only e read-one/write-all

- *Read-only*
 - Usado em conjunto de dados imutáveis
 - Lê de qualquer cópia e por definição não há escritas
- *Read-one/write-all*
 - Leitura é feita em qualquer réplica
 - Escrita é feita em todas as réplicas
 - Problema:
 - *Lock* em todas as réplicas para se fazer uma escrita → gargalo
 - Todas as réplicas devem estar disponíveis



Primary-copy

- Primary-copy
 - Uma réplica é destacada para ser a primária
 - Read é feito em qualquer cópia
 - Write é feito na primária
 - Servidores com cópias secundárias
 - Solicitam atualizações (*eager* vs. *lazy*)
 - São informados das alterações
 - **Problema:** semântica da consistência de dados

Quorum-based

- Problema comum às técnicas anteriores é particionamento da rede
 - Abordagem **pessimista**: previne que inconsistências ocorram durante particionamento, e quando restaurado, apenas atualiza cópias
- Uso de quorum: n réplicas de um objeto O
 - Leitura: para ler é necessário ler **no mínimo r réplicas** (*read quorum*)
 - Escrita: necessário fazer **no mínimo em w réplicas** (*write quorum*)
- Condição necessária: **$r + w > n$ & $w > n/2$**
 - Deve haver no mínimo uma réplica que esteja simultaneamente no quorum de leitura e de escrita
- Algumas réplicas podem ficar desatualizadas. Como determinar?
 - Necessário identificar a réplica mais atual
 - Número de versão em cada réplica que é atualizado a cada modificação
 - Réplica com o maior número de versão é a mais atual

Leitura e escrita baseada em *quorum*

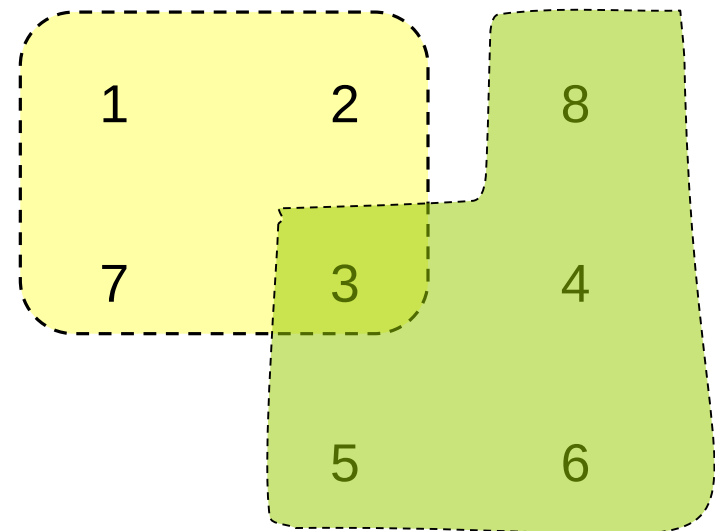
- **Read**

- Recupera as r réplicas de leitura
- Seleciona réplica com a maior versão
- Lê dado da réplica selecionada

- **Write**

- Recupera as w réplicas
- Seleciona réplica com a maior versão
- Incrementa número de versão
- Escreve novo valor e nova versão em todas as réplicas

$n = 8; r = 4; w = 5$



Leituras adicionais

- Coulouris, G.; Dollimore, J.; Kindberg, T. and Blair, G.—
“*Distributed Systems: Concepts and Design*” (5th edition),
Addison Wesley, 2012
 - Capítulo 18

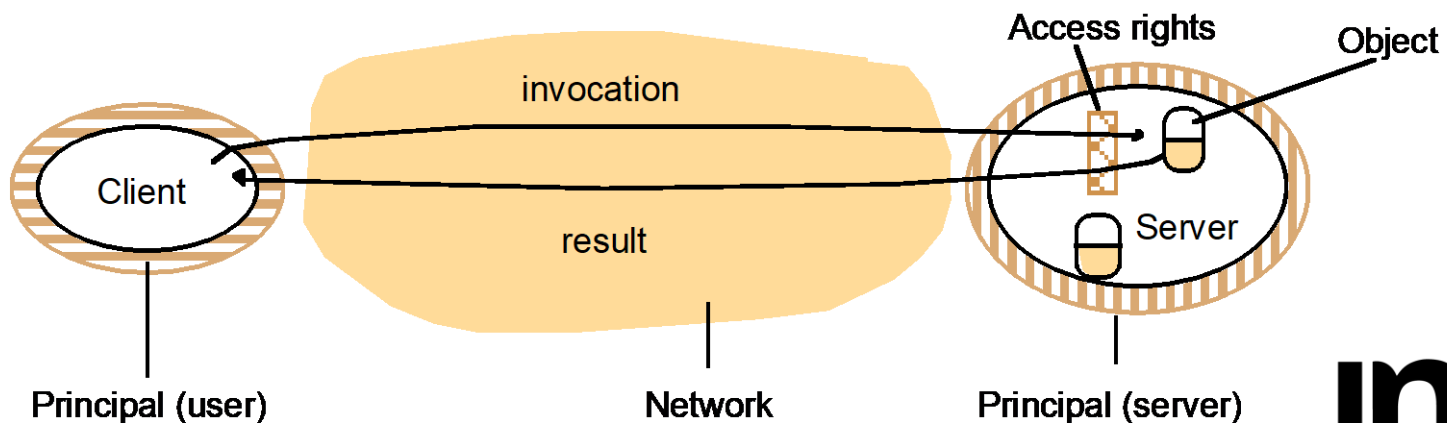
INF01151 – Sistemas Operacionais II

Aula 27 - Segurança

Prof. Alberto Egon Schaeffer Filho

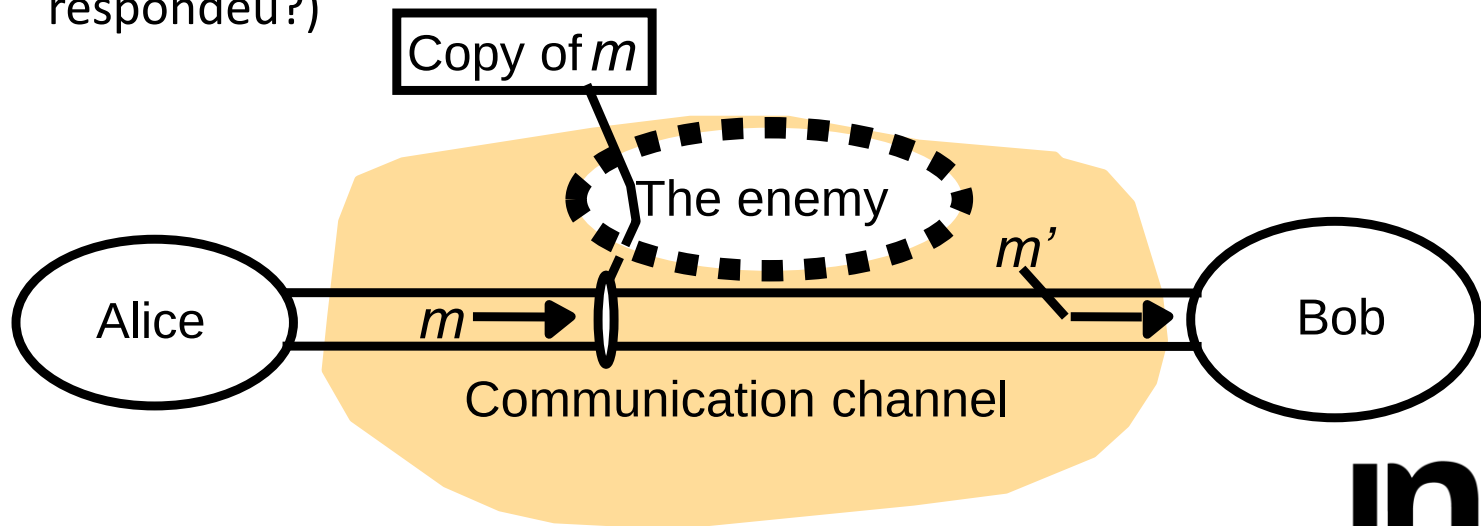
Introdução

- Sistemas distribuídos são caracterizados por compartilhar recursos
 - Necessário manter privacidade, integridade e disponibilidade
- Objetivo:
 - Tornar seguro os **recursos** e os **canais de comunicação** usados em sua interação, protegendo-os contra acessos não autorizados
 - Controle de acesso, cifragem do canal de comunicação, identificação de quem solicita e de quem responde (*principals*)



Ameaças e ataques

- Processos se comunicam através de mensagens
 - É possível que usuário malicioso copie, altere e injete mensagens na rede que são transmitidas entre *Alice* e *Bob*
- Um processo precisa saber com quem ele está se comunicando
 - Servidor: é feito para atender requisições de clientes (quem é o cliente?)
 - Cliente: é feito para receber uma mensagem de resposta (quem respondeu?)



Principais ameaças

- Vazamento (*leakage*)
 - Aquisição de informações por usuários não autorizados
- Falsificação (*tampering*)
 - Modificação não autorizada da informação
- Vandalismo (*vandalism*)
 - Interferência na operação correta de um sistema, sem ganho para o invasor

Principais ataques

- Intromissão (*eavesdropping*)
 - Obter cópias de mensagens sem autorização
- Mascaramento (*masquerading*)
 - Enviar/receber mensagens com a identidade de outro participante, sem sua autorização
- Falsificação de mensagem (*message tampering*)
 - Interceptar mensagens e alterar seu conteúdo, antes de passá-las para o destinatário desejado (e.g.: man-in-the-middle)
- Reprodução (*replaying*)
 - Armazenar mensagens interceptadas e enviá-las em uma data posterior.
 - Eficiente mesmo em certos casos onde a mensagem é autenticada e cifrada
- Negação de serviço (*denial-of-service*)
 - Saturar o canal ou recursos com mensagens para impedir (negar) o acesso de outras pessoas

Aspectos de segurança em sistemas distribuídos

- Em segurança da informação é importante garantir três propriedades:
 - **Confidencialidade:** não disponibilizar (conteúdo das) mensagens p/ processos não autorizados
 - **Integridade:** prevenir que dados sejam modificados indevidamente
 - **Disponibilidade:** garantir acesso ao serviço sempre que esse for necessário
- Porém há outras duas correlacionadas:
 - **Autenticidade:** identificar as partes envolvidas (possivelmente ambos)
 - **Não repúdio:** provar que a mensagem (ação) foi feita por seu remetente
- Uso de criptografia auxilia na obtenção dessas propriedades
 - Base para autenticação, não-repúdio, confidencialidade e integridade

Criptografia

- Criptografia é a base para os mais populares mecanismos de segurança
 - Uso primário: processo de codificação de uma mensagem de forma a ocultar seu conteúdo
 - Uso de algoritmos para cifrar e decifrar uma mensagem
 - Baseado no conhecimento de segredos (chaves)
 - Também pode ser usado para autenticação e assinaturas digitais
- Duas classes de algoritmos de criptografia
 - Criptografia de chave secreta (simétrica)
 - Emissor e destinatário compartilham um segredo (chave secreta)
 - Criptografia de chave pública (assimétrica)
 - Baseado em um par de chaves para cada usuário: uma pública, outra privada
 - A chave privada é de conhecimento apenas de seu possuidor
 - A chave pública é de conhecimento de todos
 - Requer de 100-1000 vezes mais poder de processamento para decifrar mensagem comparado com algoritmos de chave secreta

Criptografia: princípio básico de funcionamento

- Codificação (cifrar):
 - Determinar uma função f que a partir de um texto puro (P) se obtém um texto cifrado (C) com ajuda de uma chave K_e

$$f(K_e, P): P \mapsto C$$

- Decodificação (decifrar):
 - Permitir a operação inversa a partir de uma chave K_d

$$f(K_d, C): P \quad \text{ou} \quad f(K_d, f(K_e, P)) = P$$

$$K_e = K_d \rightarrow \text{simétrica}$$

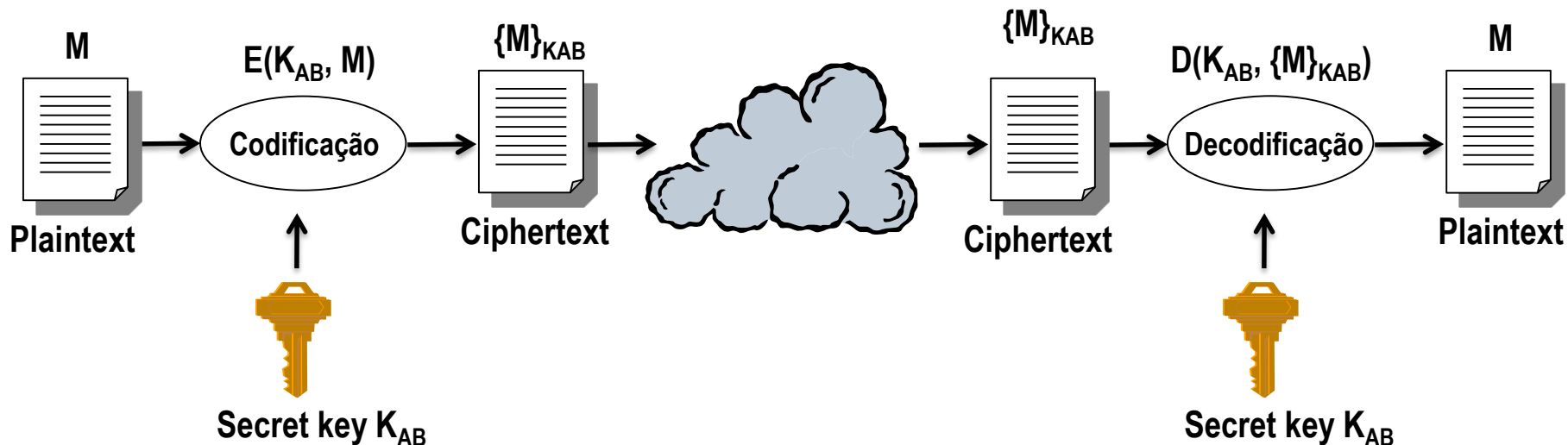
$$K_e \neq K_d \rightarrow \text{assimétrica}$$

Criptografia: principais usos

- Criptografia desempenha **três papéis principais** na implementação de segurança:
 - **Confidencialidade e integridade** (*secrecy and integrity*): proteger informação quando ela for transmitida através da rede
 - Por exemplo, durante transmissão na rede que pode ser vulnerável à *eavesdropping* e *tampering* (papel original militar)
 - Explora a propriedade de que algo cifrado com uma chave só pode ser decifrado por alguém que possuir a chave correspondente
 - **Autenticação** (*authentication*): receptor que consegue decifrar mensagem com uma dada chave, pode verificar se ela está correta (e.g., *checksum*)
 - Seguro inferir que o emissor tinha a chave de cifragem, assim deduzindo identidade do emissor
 - Se chaves foram mantidas em segredo, sucesso na decifragem autentica o emissor
 - **Assinaturas digitais** (*digital signatures*): emula papel de assinaturas convencionais
 - Garante que mensagem é uma cópia inalterada daquela produzida na origem

Cenário: confidencialidade com chave secreta

- Alice deseja enviar uma informação de forma confidencial para Bob, utilizando uma chave compartilhada K_{AB}



Assumindo que a chave não foi comprometida, e que o algoritmo utilizado seja forte o bastante. Problemas?

Problema 1: Como Alice enviou chave secreta K_{AB} para Bob?

Problema 2: Como Bob sabe que $\{M\}_{KAB}$ não é uma cópia de uma mensagem enviada por Alice anteriormente?
(e.g., requisição para pagar alguém)

Confidencialidade

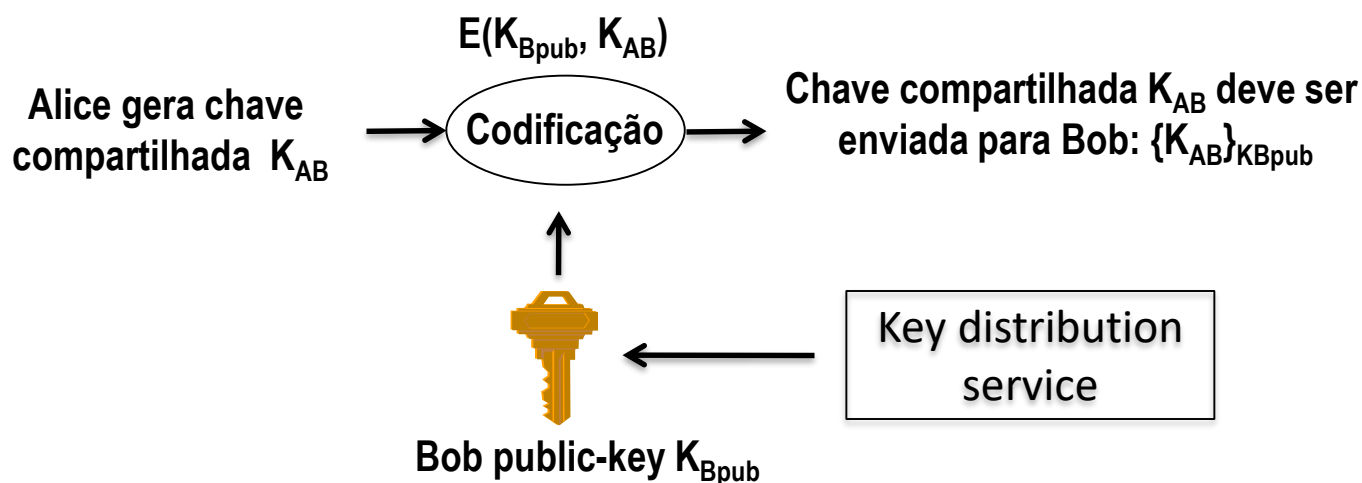
- Confidencialidade é resolvida cifrando a mensagem
- Normalmente:
 - Criptografia assimétrica é usada para negociar uma chave de sessão (chave simétrica) que é usada para cifrar os dados

	Criptografia simétrica	Criptografia assimétrica
Custo computacional	Baixo	Alto
Qtde de chaves	$n*(n-1)/2$	2 por participante ($2*n$)
Distribuição/armazenamento	Difícil	Fácil (certificado digital)
Uso típico	Cifragem de grande quantidade de dados	Autenticação, não repúdio

Protocolos híbridos de criptografia

Cenário: autenticação com chave pública

- Bob gerou um par de chaves pública/privada, e Alice e Bob desejam estabelecer uma chave compartilhada K_{AB}



Somente Bob pode recuperar K_{AB} , utilizando sua chave privada para decifrar mensagem

Problemas?

Problema: vulnerável a ataque man-in-the-middle. Attacker pode interceptar a requisição inicial de Alice, e enviar para Alice a chave pública do Attacker. Dessa forma, todas as mensagens que Alice pensa trocar com Bob são interceptadas pelo usuário malicioso

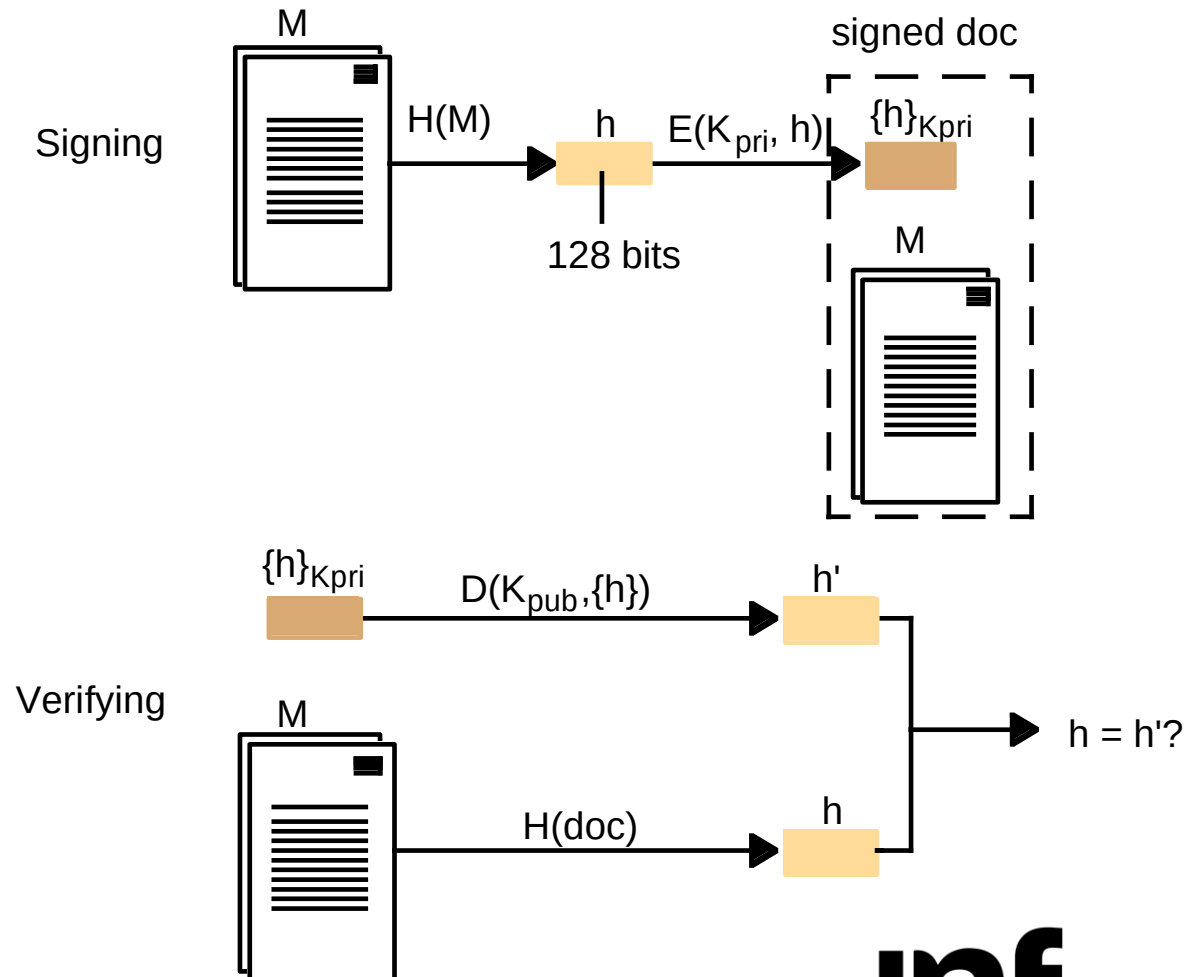
Solução: garantir que a chave publica de Bob (certificado) foi assinado por alguma autoridade bem conhecida

Assinatura digital

- Objetivo é garantir a autenticidade de uma mensagem (documento)
 - Convencer receptor que emissor assinou o documento + integridade
 - Associar identidade da origem à sequência de bits que constitui documento
- Técnicas para assinatura digital baseadas em criptografia:
 - **Digital signing with public-keys**
 - Simples, não requer comunicação entre receptor e origem para validação
 - Participante **A** assina documento **M** utilizando sua chave privada K_{pri}
 - Geralmente utilizam Digest functions (secure hash functions)
 - Denotadas por $H(M)$, e projetadas para que $H(M) \neq H(M')$ se $M \neq M'$
 - **Digital signatures with secret keys**
 - Solução menos conveniente que método utilizando chave pública/privada
 - Tecnicamente também pode ser utilizada para gerar a assinatura, mas o desafio é compartilhar essa chave
 - Vantagem: desempenho (não faz encryption)

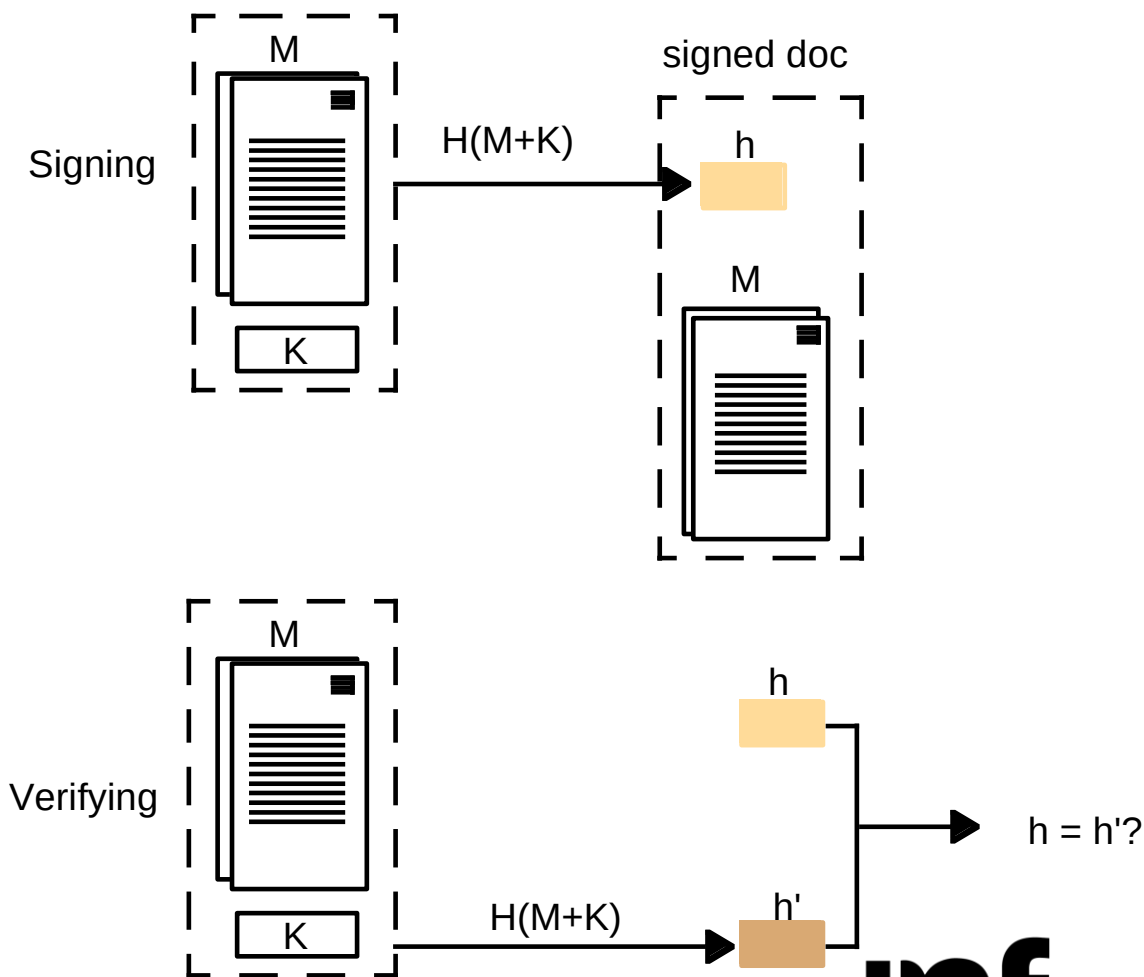
Assinatura digital usando chave pública

1. **A** gera um par de chaves K_{pub} e K_{pri} , e publica chave pública K_{pub}
2. **A** calcula digest de **M**, $H(M)$ usando uma função **H** conhecida por ambos
3. **A** cifra $H(M)$ utilizando chave privada K_{pri} , produzindo assinatura $S = \{H(M)\}_{K_{pri}}$
4. **A** envia mensagem assinada **M, S** para **B**
5. **B** decifra **S** usando K_{pub} e computa o hash de **M**, $H(M)$. Se forem iguais, assinatura é válida



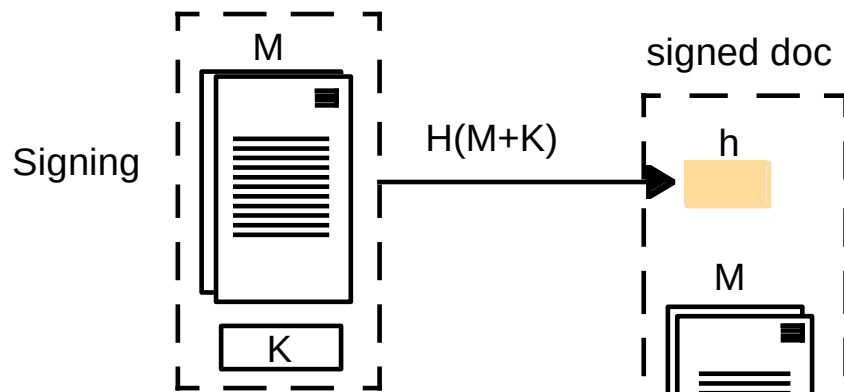
Assinatura digital usando chave secreta

1. **A** gera uma chave **K** para assinar documentos, e distribui ela “*de alguma forma segura*” para outros participantes
2. **A** concatena **M+K**, e utiliza função de hash para calcular digest **$h = H(M+K)$**
3. **A** envia o documento assinado **M, h** (chave **K** não é comprometida pela divulgação de **h**)
4. **B** concatena a chave secreta **K** com o documento recebido **M**, e computa **$h' = H(M+K)$** . Se forem iguais, assinatura é válida



Assinatura digital usando chave secreta

1. **A** gera uma chave **K** para assinar documentos, e distribui ela “*de alguma forma segura*” para outros participantes
2. **A** concatena **M+K**, e utiliza função de hash para calcular

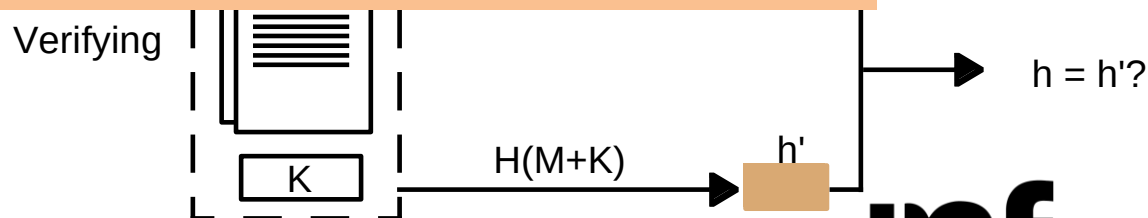


Propriedades de uma secure digest function $h = H(M)$:

1. Dado **M**, é fácil computar **h**
2. Dado **h**, é difícil computar **M**
3. Dado **M**, é difícil encontrar outra mensagem **M'**, tal que $H(M) = H(M')$

Exemplos: **MD5, MD6, SHA-2, SHA-3**

M, e computa $h' = H(M+K)$. Se forem iguais, assinatura é válida



Protocolos de autenticação

- Autenticação user + password é inconveniente em sistemas distribuídos
 - Necessário uma base de dados com todos possíveis usuários
 - Autenticação a cada ação do usuário no sistema
 - Solução é definir protocolos de autenticação ...
- Princípio básico
 - Identificar se o usuário é quem ele diz ser e isso é feito com base em algo que só ele conhece ou possui: chave
 - Protocolos de autenticação
 - **Needham-Schroeder**
 - **Kerberos**

Protocolo de autenticação Needham-Schroeder

- Solução para autenticação e distribuição de chaves
 - Objetivo: permitir comunicação segura entre processo A e um processo B
- Baseado em um servidor de autenticação
 - Servidor mantém uma tabela com o nome e uma **chave secreta** para cada participante
 - Usada para autenticar o cliente p/ servidor de autenticação
 - Usada para transmissão segura entre o cliente e servidor de autenticação
 - **Problema:** como distribuir a **chave compartilhada**?
 - Servidor gera **chaves compartilhadas** para para pares de clientes A e B
 - Protocolo baseado na geração e transmissão de **tickets** pelo servidor de autenticação
 - Ticket: mensagem cifrada que contém chave secreta para comunicação entre A e B
- Existe também versão baseada em chave pública...

Protocolo Needham-Schroeder: mensagens

Header	Message	Notes
1. A→S:	A, B, N_A	A requests S to supply a key for communication with B.
2. S→A:	$\{N_A, B, K_{AB}, \{K_{AB}, A\}_{K_B}\}_{K_A}$	S returns a message encrypted in A's secret key, containing a newly generated key K_{AB} and a 'ticket' encrypted in B's secret key. The nonce N_A demonstrates that the message was sent in response to the preceding one. A believes that S sent the message because only S knows A's secret key.
3. A→B:	$\{K_{AB}, A\}_{K_B}$	A sends the 'ticket' to B.
4. B→A:	$\{N_B\}_{K_{AB}}$	B decrypts the ticket and uses the new key K_{AB} to encrypt another nonce N_B .
5. A→B:	$\{N_B - 1\}_{K_{AB}}$	A demonstrates to B that it was the sender of the previous message by returning an agreed transformation of N_B .

Kerberos

- Baseado em dois serviços

- Kerberos Authentication Server (KAS)

- Autentica o usuário e fornece um ticket para acessar o TGS

- Ticket Granting Server (TGS)

- Fornece tickets para usuários empregarem outros serviços

Ticket p/ cliente C acessar servidor S:

$\{C, S, t_1, t_2, K_{CS}\}_{K_S}$

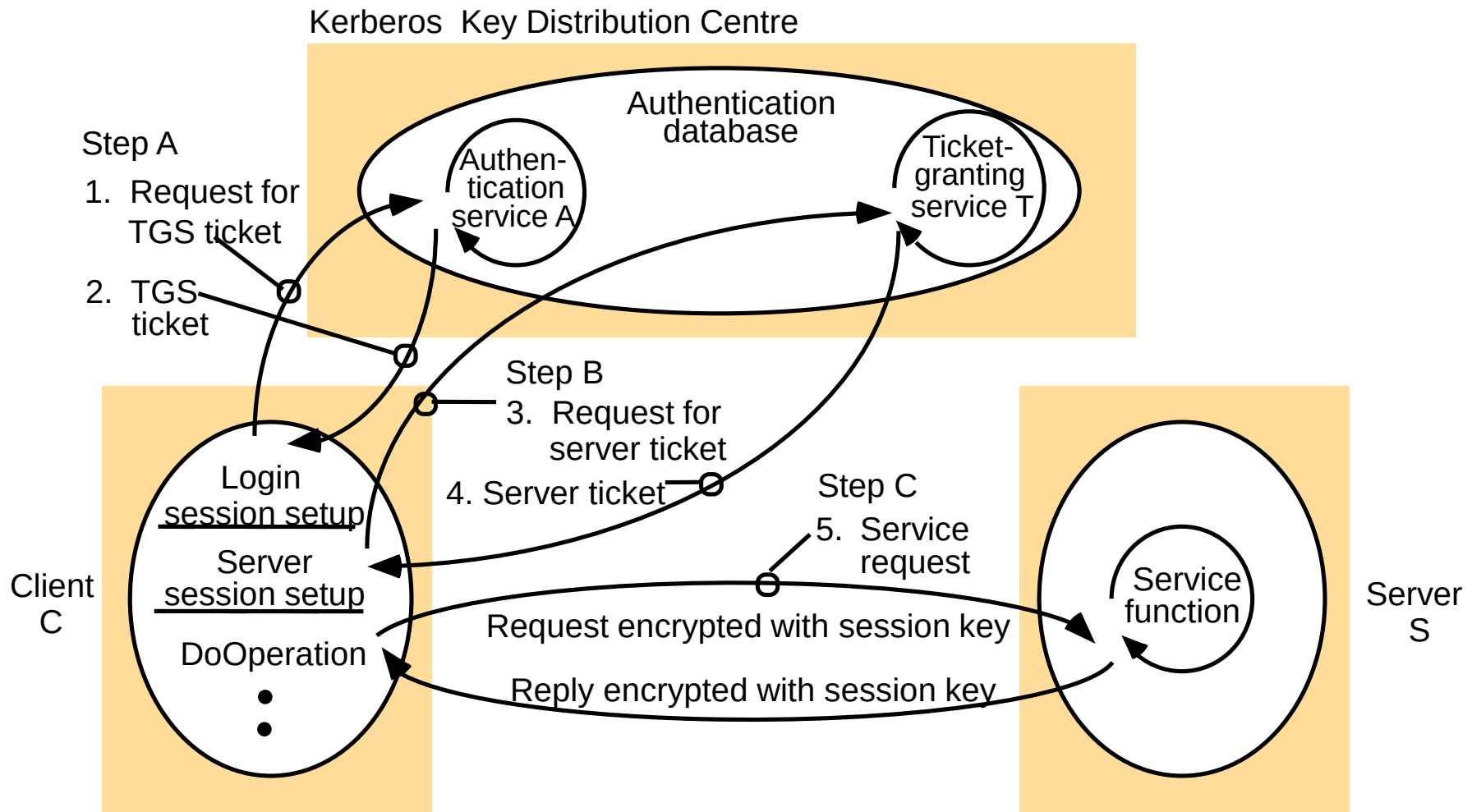
Authenticator de cliente C
p/ acessar servidor S:

$\{C, t\}_{K_{CS}}$

- Arquitetura Kerberos define 3 elementos de segurança:

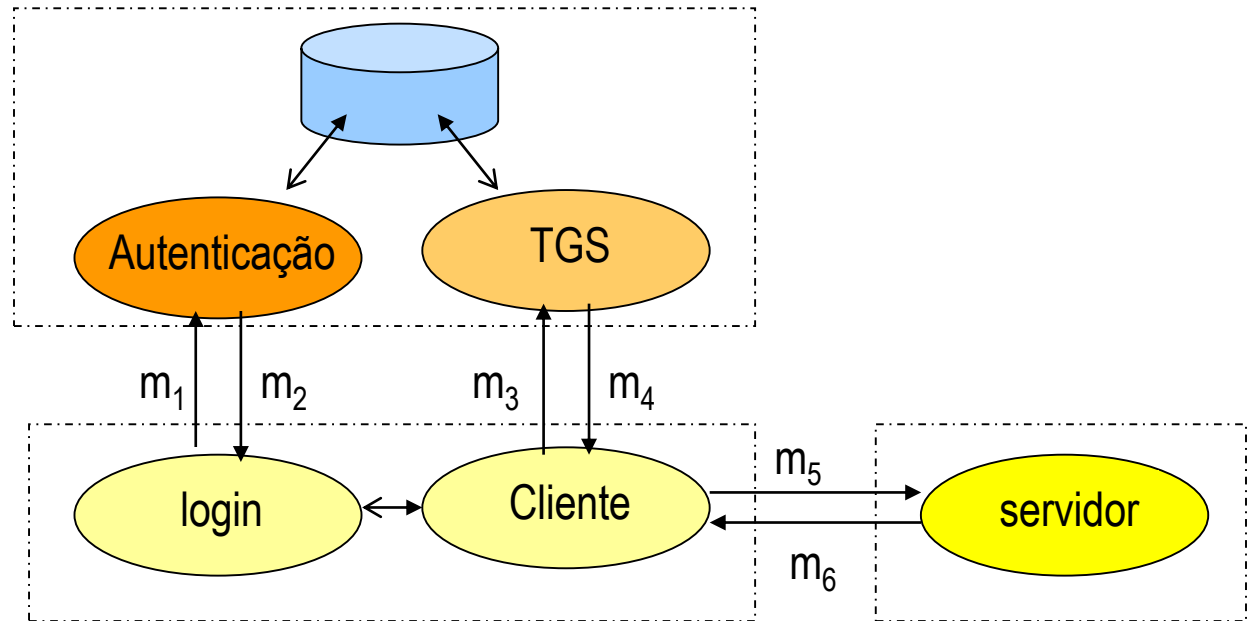
- **Ticket**: Implementa noção de **capability** (privilégio p/ executar ação). Token emitido pelo TGS, que deve ser apresentado para utilização de um serviço S
 - Indica que processo foi recentemente autenticado pelo Kerberos
 - Inclui *expiry time* e *session key* gerada
 - **Authenticator**: token construído pelo cliente para provar sua identidade. Só pode ser utilizado uma vez para acessar serviço S
 - Contém nome do cliente, timestamp e é encriptado com a session key
 - **Session key**: gerada randomicamente pelo Kerberos e emitida para que cliente se comunique com um determinado serviço. Usada para cifrar o authenticator
 - Também usada para cifrar a comunicação com o serviço S (se necessário)

Arquitetura do sistema Kerberos



Protocolo de autenticação do Kerberos

ID_g : id. TG
 ID_c : id. cliente
 ID_s : id. servidor
 N_i : nonce
 K_c : chave secreta cliente
 K_s : chave secreta servidor
 K_g : chave secreta TGS
 K_1 : chave sessão TGS
 K_2 : chave sessão servidor
 T_{si} : tempo inicial ticket
 T_{ei} : tempo final ticket
 T_i : timestamp



$m_1 = (ID_c, N_1)$
 $m_2 = C_2 = E((N_1, K_1, C_1), K_c)$
 $m_3 = (ID_s, N_2, C_1, C_3)$
 $m_4 = C_5 = E((N_2, K_2, C_4), K_1)$
 $m_5 = (C_4, C_6)$
 $m_6 = C_7 = E(T_3, K_2)$

$C_1 = E((ID_c, ID_g, T_{s1}, T_{e1}, K_1), K_g)$
 $C_3 = E((ID_c, T_1), K_1)$
 $C_4 = E((ID_c, ID_s, T_{s2}, T_{e2}, K_2), K_s)$
 $C_6 = E((ID_c, T_2), K_2)$
 $T_3 = T_2 + 1$

Leituras adicionais

- Coulouris, G.; Dollimore, J.; Kindberg, T. and Blair, G.—
“*Distributed Systems: Concepts and Design*” (5th edition),
Addison Wesley, 2012
 - Capítulo 11

Secure Socket Layer (SSL)

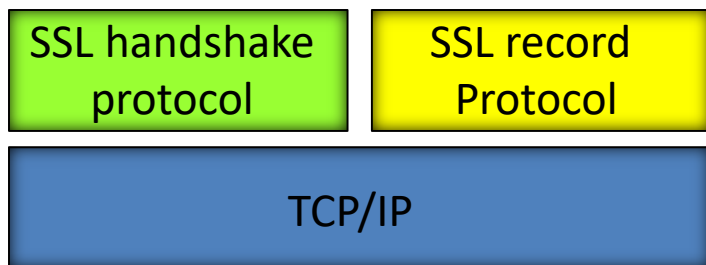
- Desenvolvido originalmente pela Netscape Corporation
 - Versão estendida de SSL foi adotada como padrão na internet
 - Transport Layer Security (TLS), descrito na RFC 2246
 - TLS suportado pela maioria dos browsers, utilizado em e-commerce
- Projetado para fornecer cifragem de dados entre um cliente e um servidor web
 - Não é limitado a web, pois oferece uma API de programação para aplicação



Exemplos de aplicações na Internet: HTTPS (443); SSMTP(465); POP (995); IMAP (993); etc

Arquitetura SSL

- Organizado em dois protocolos básicos:
 - Protocolos específicos empregado no gerenciamento do SSL
 - Usado no estabelecimento
 - SSL Record Protocol
 - Empregado na utilização de uma conexão segura
- Possui ainda um protocolo para troca de chaves e alerta



- Conjunto de algoritmos de cifragem e opções distintas
- Compactação
- Restrições quanto a chaves e uso

Características SSL

- Apresentação mútua
 - Negociação do algoritmo de cifragem (DES, IDEA, etc) e de chave de sessão
 - Autenticação do servidor ao cliente e do cliente ao servidor (opcional)
- Autenticação do servidor
 - Permite confirmação da identidade do servidor
 - Cliente mantém listas de ACs com suas chaves públicas
 - Obtém certificado do servidor (isto é, sua chave pública)
 - Verifica autenticidade do certificado (assinatura de um AC)
- Autenticação do cliente (opcional)
 - Análoga ao servidor cliente envia certificado ao servidor
- Sessão SSL
 - Transferência de dados (cifrado)

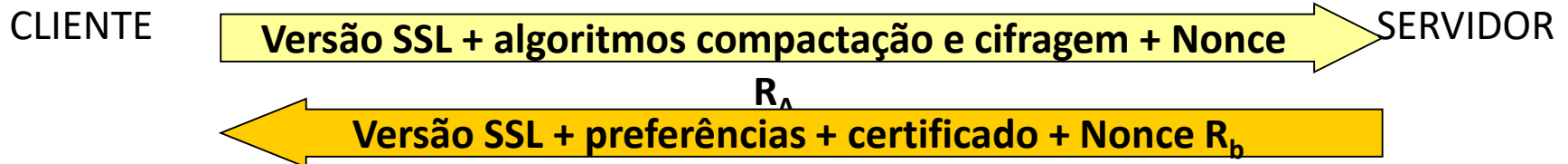
Características do SSL

- Prove autenticação e confidencialidade
- Emprega cifragem assimétrica (chave pública/privada) e chaves de sessão
- Duas partes:
 - SSL handshake protocol
 - Fornece a autenticação dos pares (criptografia assimétrica)
 - Usado para gerar uma chave de sessão (chave privada)
 - Uso de certificados digitais
 - SSL record protocol
 - Fornece a confidencialidade
 - Usado para cifrar as mensagens
 - Autenticidade (integridade + autenticação) é via assinatura digital da mensagem

Protocolo *Handshake*

- Parte mais complexa do SSL.
- Possibilita a autenticação mútua de clientes e servidores.
- Negocia codificação, algoritmo MAC (*Message Authentication Code*) e chaves de criptografia.
- Empregado antes que qualquer dado seja transmitido.

Funcionamento SSL no https (*handshake*)



1. AC do certificado está na lista?

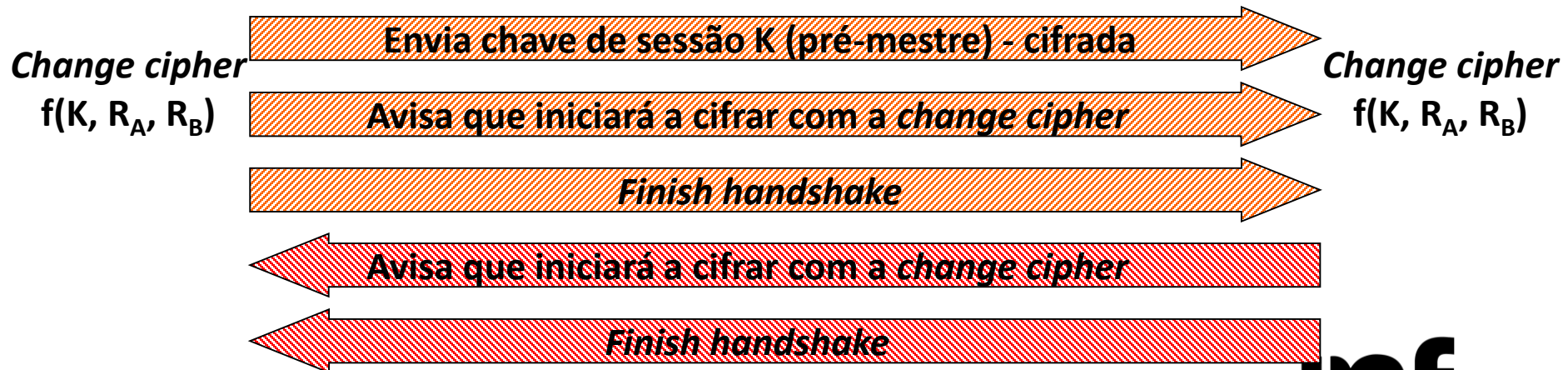
NÃO: avisa usuário

SIM:

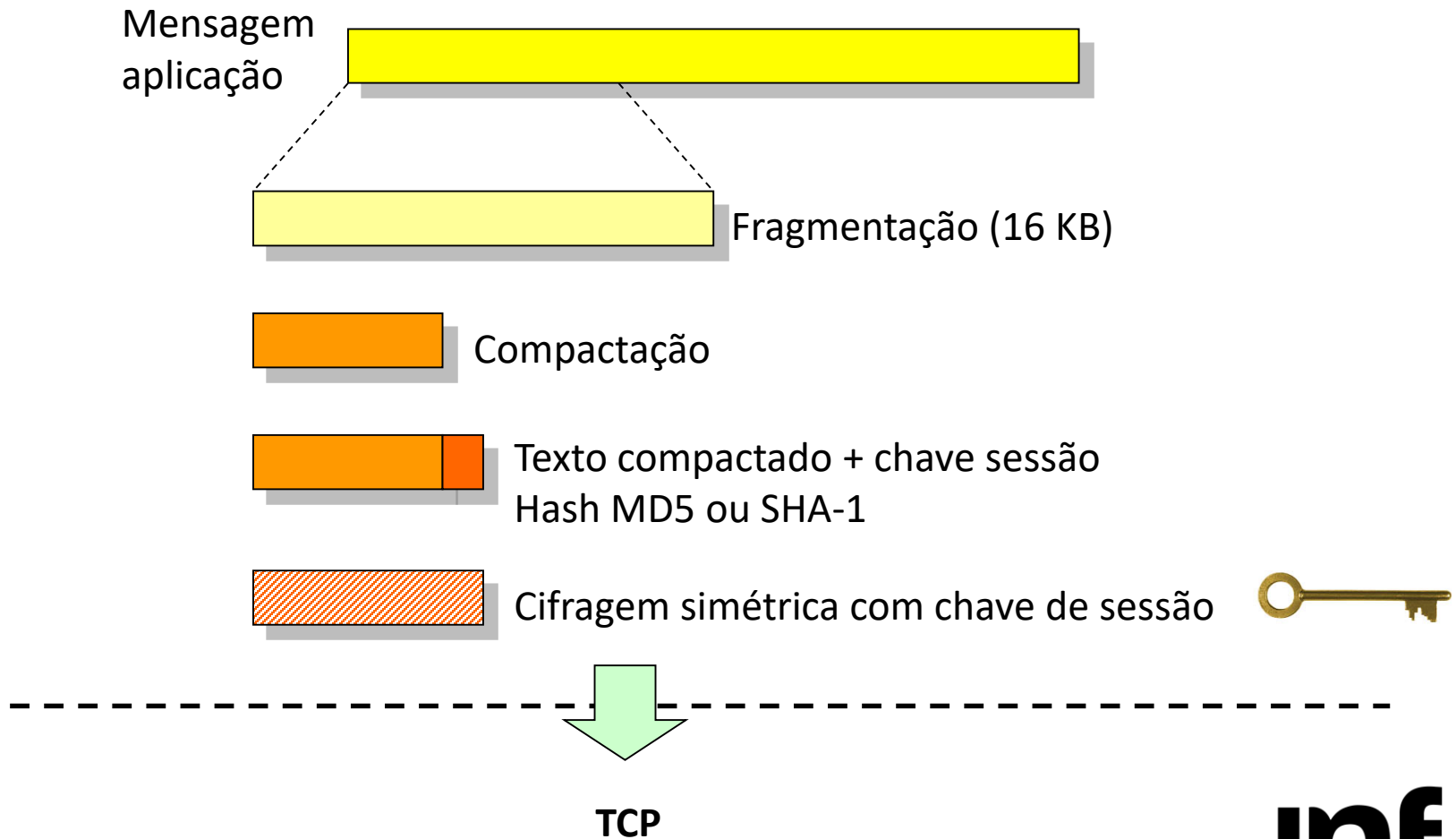
- valida certificado com a chave pública da AC e obtém chave pública do servidor

Opcional: solicitação do certificado do cliente

2. Gera chave de sessão K



Funcionamento do SSL (envio de dados)



INF01151 – Sistemas Operacionais II

Aula 28 - Virtualização

Prof. Alberto Egon Schaeffer Filho

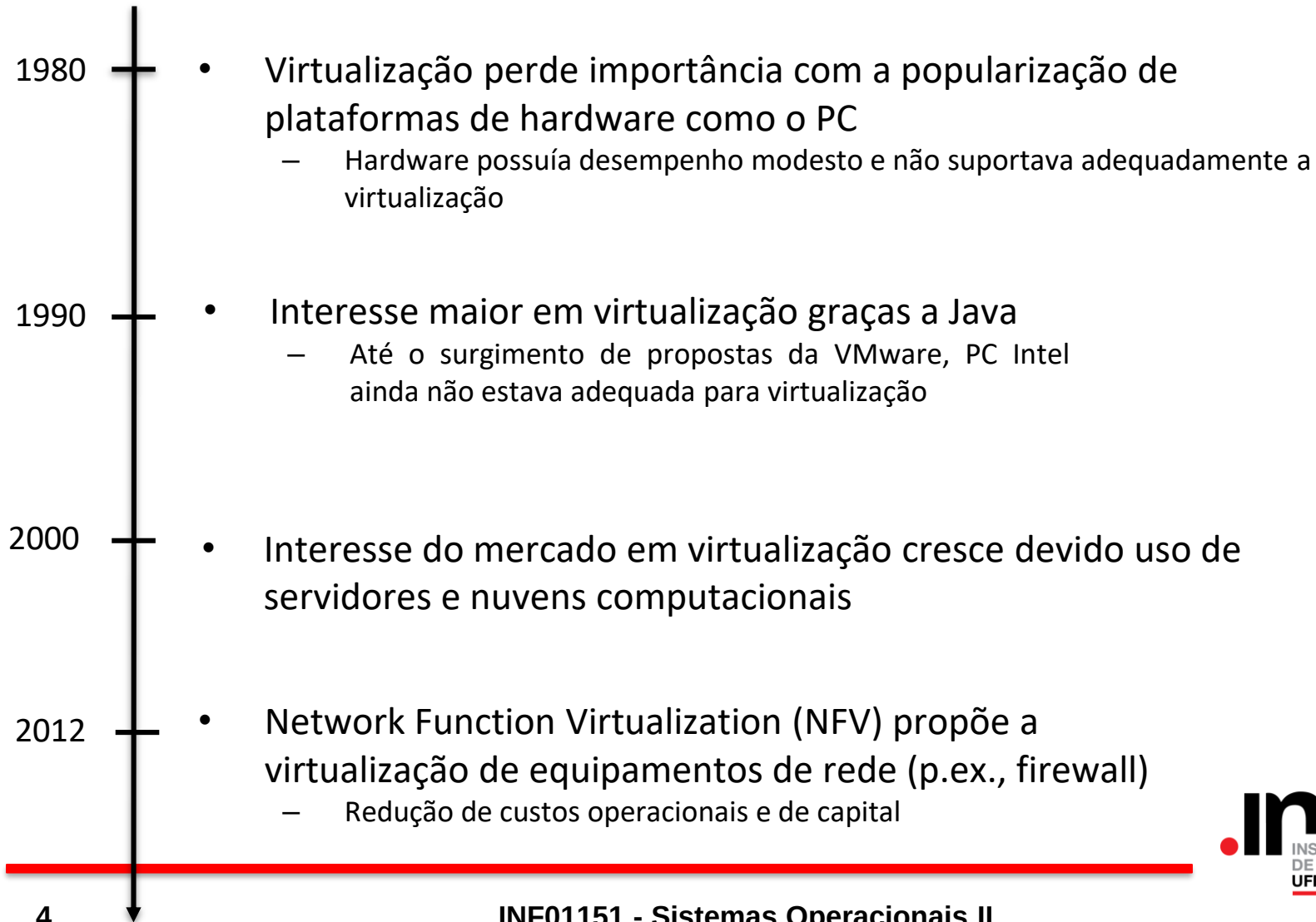
Conceito de virtualização

- Definição [Popek and Goldberg]:
 - *“Uma máquina virtual é vista como uma duplicata eficiente e isolada de uma máquina real. Essa abstração é construída por um monitor de máquina virtual (VMM - Virtual Machine Monitor).”*
- Conceito de virtualização de recursos é relativamente antigo
 - Recentes tecnologias de virtualização vem ganhando ênfase por darem suporte a importantes paradigmas de computação
 - Em ambientes de Cloud Computing, recursos podem ser adquiridos e alocados segundo a demanda
 - Consolidação de servidores
 - Elasticidade de recursos
 - Infrastructure as a Service (IaaS), Platform as a Service (PaaS), ...

Histórico

- 
- 1960
 - Desenvolvimento do sistema operacional experimental M44/44X pela IBM
 - Objetivo era fornecer a cada usuário ambiente monousuário completo, com SO próprio e aplicações isoladas dos ambientes dos demais usuários
 - Início de uma série de sistemas suportando virtualização
 - 1970
 - Popek & Goldberg formalizam vários conceitos para máquinas virtuais e requisitos para que uma plataforma de hardware suporte virtualização
 - Surgimento das primeiras experiências com a execução de aplicações sobre máquinas virtuais
 - Ambiente *UCSD p-System*, no qual programas Pascal eram compilados para execução sobre o hardware virtual *P-Machine*

Histórico

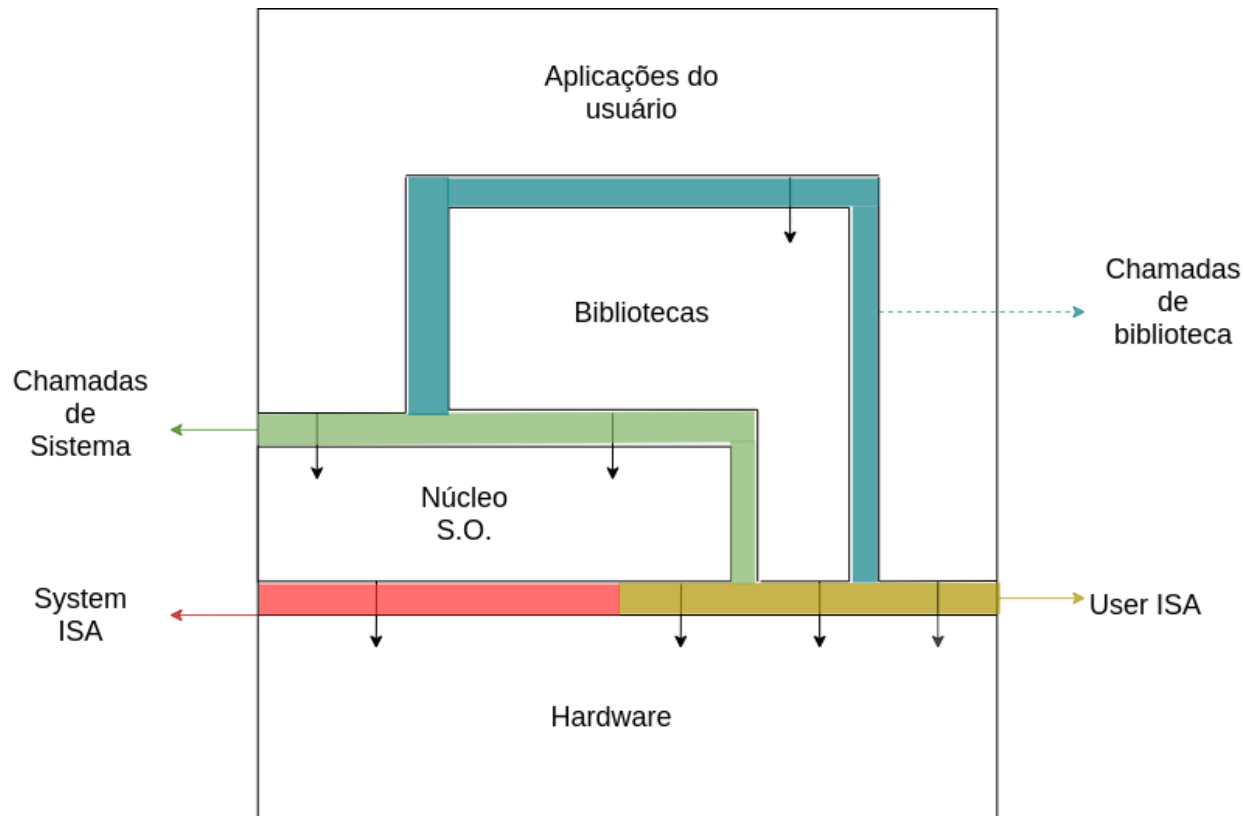


Conceito de virtualização

- Interfaces de sistema
 - Sistemas são compostos de vários componentes
 - Hardware, sistema operacional e aplicações
 - Tais componentes fornecem abstrações, que são separadas através de interfaces
 - Abstração encapsula abstrações de níveis inferiores, simplificando construção do sistemas de computação
 - Hardware fornece um conjunto de instruções para o SO
 - SO fornece conjunto de chamadas de sistema, controlando o acesso ao hardware (acesso I/O)
 - Conjunto de interfaces entre esses componentes:
 - **Conjunto de instruções (ISA)**
 - **Chamadas de sistema (syscalls)**
 - **Chamadas de biblioteca (libcalls)**

Conceito de virtualização

- Interfaces de sistema



Adaptação de Maziero, 2019

Conceito de virtualização

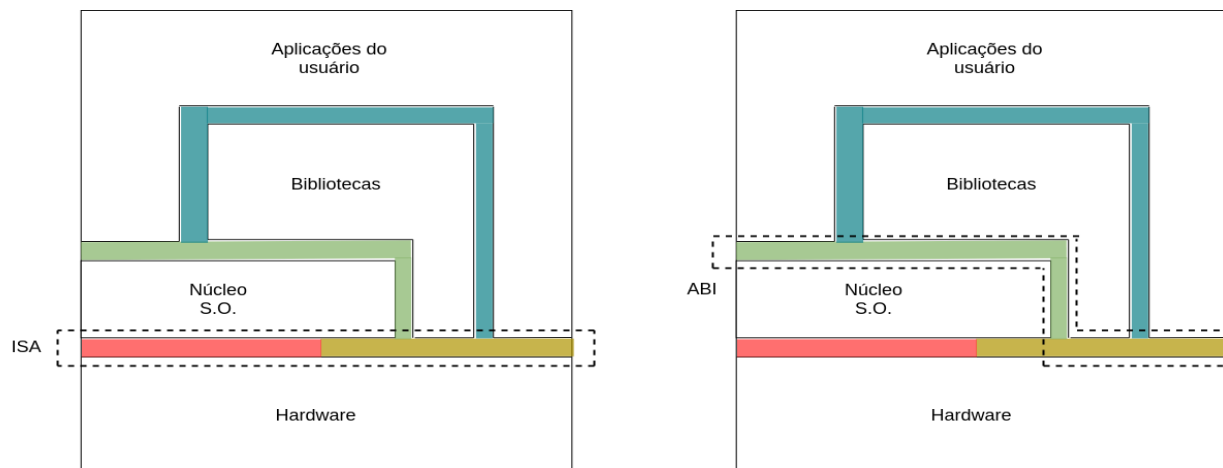
- Interfaces de sistema
 - Conjunto de instruções ISA (*Instruction Set Architecture*)
 - Interface básica entre hardware e software
 - Instruções de máquina aceitas pelo processador e operações de acesso aos recursos de hardware
 - Divididas em instruções de usuário e instruções de sistema
 - **Instruções de usuário (User ISA)**: instruções de acesso a recursos executadas em modo usuário
 - **Instruções de sistema (System ISA)**: instruções de acesso a recursos, acessíveis apenas ao núcleo do sistema, executando em modo privilegiado

Conceito de virtualização

- Interfaces de sistema
 - Chamadas de sistema (*syscalls*)
 - Conjunto de operações oferecidas pelo núcleo do sistema operacional aos processos no espaço de usuário
 - Permitem o acesso controlado das aplicações aos periféricos, à memória e às instruções privilegiadas do processador
 - Chamadas de bibliotecas (*libcalls*)
 - Bibliotecas fornecem funções para construção de programas
 - Muitas encapsulam chamadas do SO, para tornar seu uso mais simples
 - Cada biblioteca possui uma interface própria, chamada API
 - Exemplos: LibC (UNIX) com funções como `fopen()` e `printf()`

Compatibilidade entre interfaces

- Execução de programas e bibliotecas sobre uma plataforma
 - Necessário respeitar conjunto de instruções do processador em modo usuário (User ISA) e as chamadas de sistema do SO
 - Conjunto dessas duas interfaces (**User ISA + syscalls**) é chamada **Interface Binária de Aplicação (ABI – Application Binary Interface)**
 - Similarmente, SO só irá executar se tiver sido construído e compilado respeitando interface ISA (User/System ISA) do hardware



Adaptação de Maziero, 2019

Compatibilidade entre interfaces

- Embora a definição de interfaces seja útil, ela cria uma rigidez no projeto de sistemas:
 - SO só irá funcionar sobre o hardware (ISA) para qual foi construído
 - Bibliotecas só irão funcionar sobre a ABI para a qual foi projetada
 - Aplicações devem obedecer as ABIs e APIs pré-definidas
- Interação inflexível entre interfaces de componentes de um sistema operacional traz alguns problemas
 - **Baixa portabilidade:** sem mobilidade de código e interoperabilidade, acoplando de forma excessiva aplicações ao SO + hardware
 - **Barreiras de inovação:** interfaces rígidas dificultam interações entre aplicações e dispositivos de hardware, inércia à evolução
 - **Otimizações intercomponentes:** otimizações são feitas apenas a nível de cada componente (abstração)

Virtualização de interfaces

- Contornar problemas relacionados à inflexibilidade de interfaces através de **camada de virtualização** em software
 - Essencialmente permitindo que um programa desenvolvido para uma plataforma possa executar em outra
- Camada de virtualização utiliza serviços de uma interface do sistema para **oferecer outra interface de mesmo nível**
 - Nova interface de sistema fornecida através da camada de virtualização é chamada **máquina virtual**
 - Camada de virtualização em si é chamada de **hypervisor** ou **monitor de máquina virtual (VMM)**

Virtualização de interfaces

- Um ambiente de máquina virtual consiste em três partes:
 - **Sistema real, nativo ou hospedeiro** (*host system*), contendo os recursos reais do sistema
 - **Camada de virtualização**, chamada de *hypervisor* ou VMM, que fornece uma interface virtual
 - **Sistema virtual ou convidado** (*guest system*), executando sobre a camada de virtualização

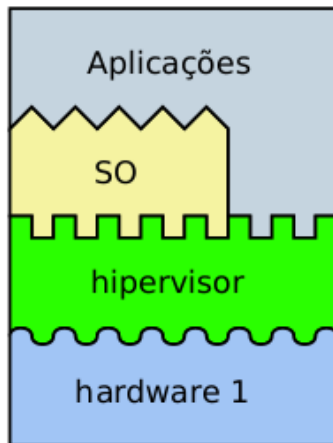
Virtualização de interfaces

- Tipos de virtualização
 - **Emulação de hardware:** sistema operacional convidado e suas aplicações desenvolvidas para uma plataforma são executadas sobre outra plataforma distinta
 - **Emulação de sistema operacional:** aplicações construídas para um sistema operacional são executadas sobre outro sistema operacional
 - **Otimização dinâmica:** instruções de máquina das aplicações são traduzidas durante a execução em outras instruções mais eficientes para a mesma plataforma
 - **Replicação de hardware:** várias instâncias virtuais de um mesmo hardware real, cada uma executando seu próprio sistema operacional convidado e suas respectivas aplicações

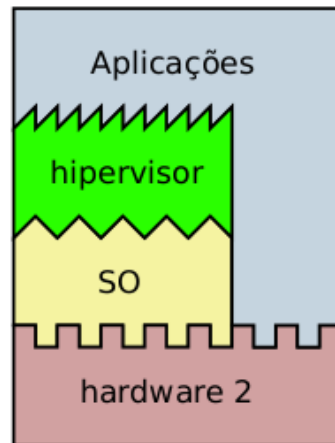
** Emulação é uma forma de virtualização onde hypervisor virtualiza integralmente uma interface de hardware ou SO*

Virtualização de interfaces

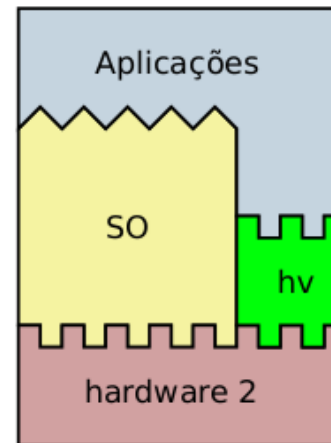
- Tipos de virtualização



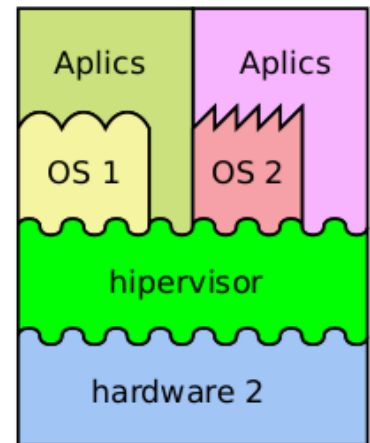
emulação
de hardware



emulação do SO



otimização dinâmica

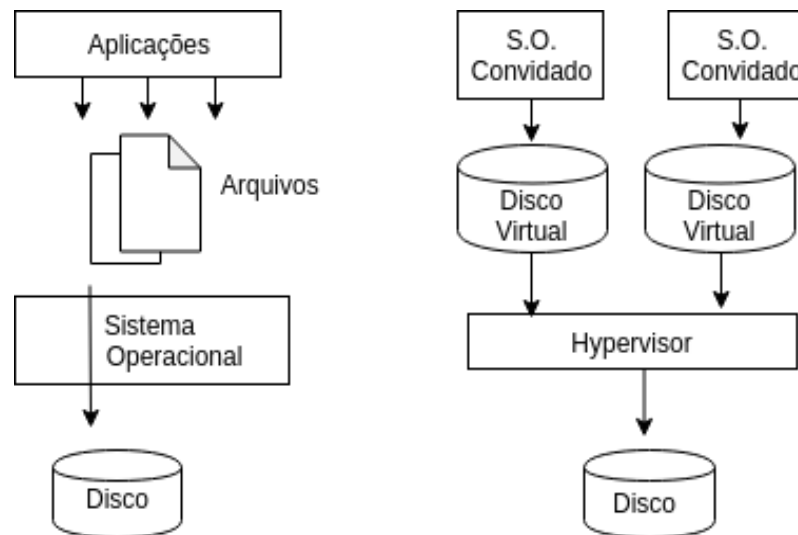


replicação
de hardware

Fonte: Maziero, 2019

Diferença entre virtualização e abstração

- Objetivos diferentes
 - Sistema operacional fornece visão alto nível de recursos de hardware em níveis de abstração crescentes
 - Virtualização cria novas interfaces a partir das existentes
 - Detalhes de baixo nível da plataforma real não são necessariamente ocultos, como ocorre na abstração de recursos

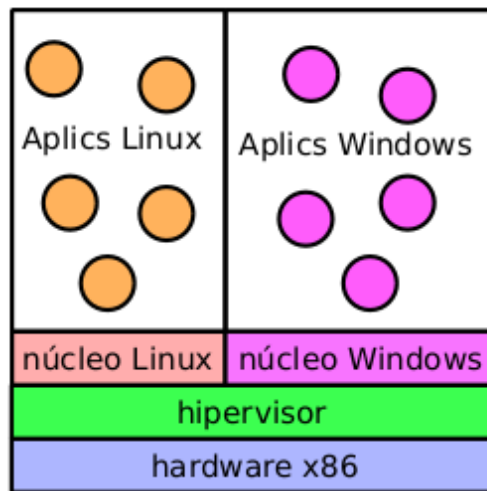


Tipos de máquinas virtuais

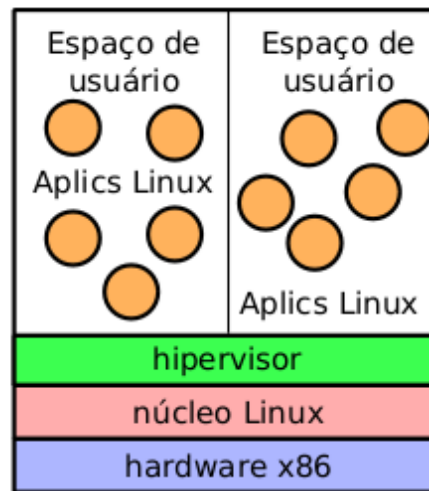
- Máquinas virtuais podem ser classificadas de acordo com o ambiente virtual fornecido
 - **Máquinas virtuais de sistema**
 - Construídas para emular plataformas de hardware completas
 - Suportam sistemas operacionais executando aplicações
 - Ex: KVM, VMware e VirtualBox
 - **Máquinas virtuais de sistema operacional**
 - Suportam espaços de usuário distintos sobre um mesmo SO
 - Possuem recursos lógicos próprios (armazenamento, interfaces de rede, ...)
 - Ex: Docker, Solaris Containers, ...
 - **Máquinas virtuais de processo**
 - Ou máquinas virtuais de aplicação ou linguagem
 - Suportam a execução de apenas um processo ou aplicação convidada específica
 - Ex: Java, Valgrind

Tipos de máquinas virtuais

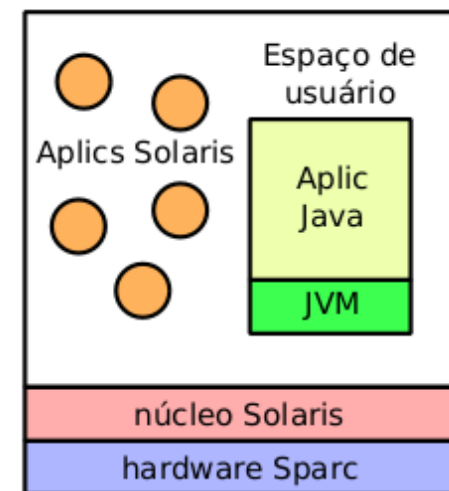
- Máquinas virtuais podem ser classificadas de acordo com o ambiente virtual fornecido



VM de sistema
(hardware)



VM de sistema
operacional



VM de processo

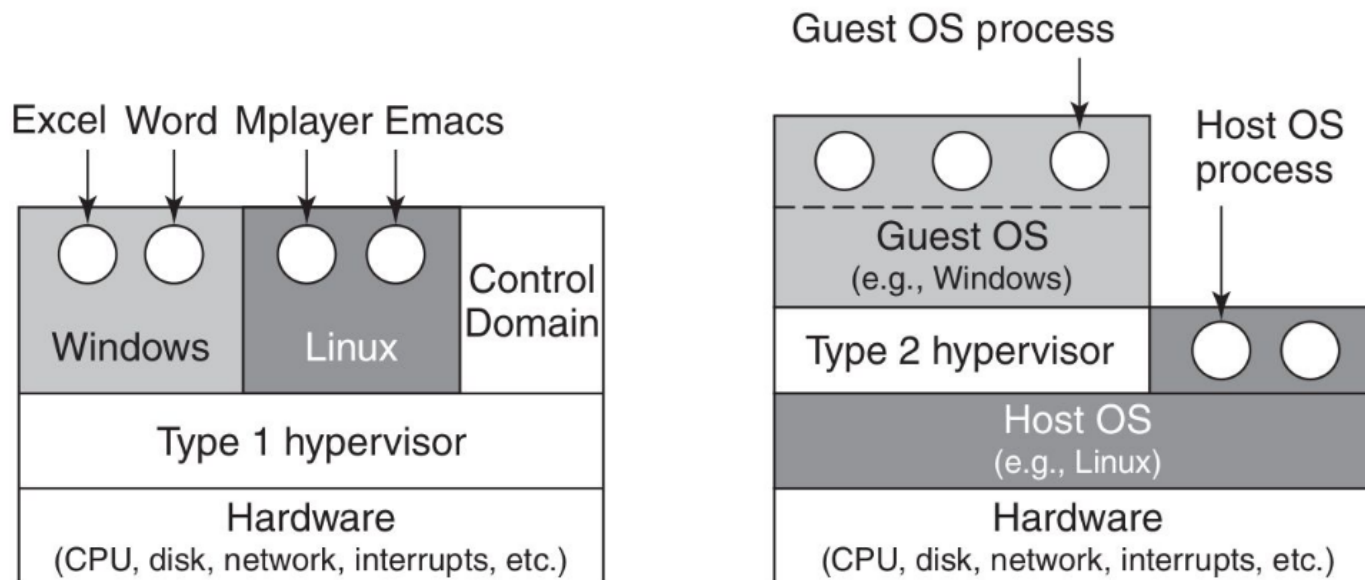
Fonte: Maziero, 2019

Tipos de máquinas virtuais

- **Máquinas virtuais de sistema**
 - Cada sistema tem a ilusão de executar sozinho sobre uma plataforma de hardware exclusiva
 - Hypervisor intercepta e virtualiza o acesso aos recursos
 - Forte isolamento entre máquinas virtuais em uma mesma plataforma
 - Existem duas arquiteturas para hypervisors
 - **Hypervisor Nativo (tipo I)**
 - Executa diretamente sobre hardware, sobre nenhum sistema operacional
 - Cada máquina virtual se comporta como um computador completo
 - Ex: VMware ESX Server e Xen
 - **Hypervisor Convidado (tipo II)**
 - Executa como processo de um SO subjacente, que gerencia o hardware
 - Hypervisor oferece recursos virtuais aos SOs convidados
 - Ex: VMware Workstation e VirtualBox

Tipos de máquinas virtuais

- Máquinas virtuais de sistema



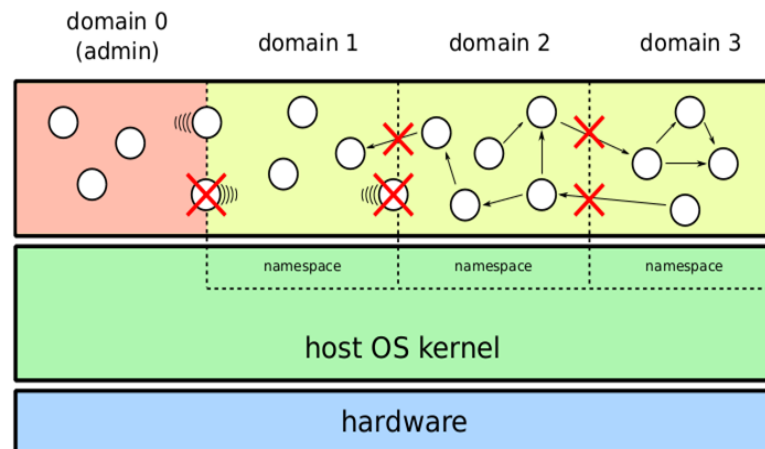
Hypervisors tipo I e tipo II. Fonte: Tanenbaum (2015)

Tipos de máquinas virtuais

- **Máquinas virtuais de sistema operacional**
 - Custo da virtualização de plataforma de hardware sobre o desempenho do sistema pode ser elevado
 - Virtualização dos recursos (periféricos) da máquina real
 - Possível necessidade de emular instruções do processador real
 - Virtualização do espaço de usuário (*userspace*) provê alternativa para implementar **isolamento entre aplicações** ou subsistemas
 - Espaço de usuário do SO é dividido em áreas isoladas denominadas **domínios** ou **contêineres**
 - Parcela dos recursos do sistema operacional alocada a cada domínio
 - Núcleo do sistema é o mesmo para todos os domínios
 - Conjunto de chamadas de sistema é preservado
 - Ex. LXC e Docker

Tipos de máquinas virtuais

- **Máquinas virtuais de sistema operacional**
 - Processos de um domínio podem interagir entre si, mas não com processos de outro domínio (a não ser através da rede)
 - Normalmente processos no domínio d0, chamado de domínio inicial, privilegiado ou de gerência
 - Têm visibilidade e acesso aos processos e recursos dos demais domínios



Fonte: Maziero, 2019

Tipos de máquinas virtuais

- **Máquinas virtuais de processo**
 - Suporta execução de um processo ou aplicação individual
 - Criada e destruída para execução de uma aplicação
 - Hypervisor + aplicação visto como único processo dentro do SO
 - Hypervisor pode implementar mesma interface de hardware (ISA) da máquina real subjacente, ou interface distinta
 - Exemplos de máquinas virtuais de aplicação
 - Sistemas operacionais multitarefa (PCB + registradores!)
 - Tradutores dinâmicos, alguns depuradores de memória
 - Mapeamento de chamadas de sistema e de bibliotecas usadas pela aplicação para chamadas equivalentes oferecidas pelo SO da máquina real
 - Ambiente Wine: permite executar aplicações Windows em plataformas Unix
 - Mapeamento interface ISA abstrata para interface ISA de hardware real
 - Compilação de programa Java gera bytecodes para máquina abstrata

Requisitos básicos para hypervisors

- Monitor ou hypervisor deve atender a alguns requisitos básicos [Popek and Goldberg, 1974]
 - **Equivalência**
 - Prover ambiente idêntico à máquina real, aplicações executam com mesmo comportamento que teriam em máquina real
 - **Controle de recursos**
 - Hypervisor possui controle completo dos recursos na máquina real e intermedia acesso a eles pela VM
 - **Eficiência**
 - Grande parte das instruções do processador virtual devem ser executadas pelo processador real, sem intervenção do hypervisor

Requisitos complementares para hypervisors

- Propriedades adicionais frequentemente associadas a hypervisors [Rosenblum, 2004]
 - **Isolamento**
 - Sem interação direta das aplicações de uma VM com outras aplicações, com o hypervisor ou com o sistema hospedeiro
 - **Introspecção**
 - Hypervisor tem acesso e controle sobre o estado da VM, incluindo registradores, conteúdo de memória, ...
 - **Recursividade**
 - Pode ser possível executar um hypervisor dentro de uma máquina virtualizada

Construção de máquinas virtuais

- No entanto, hypervisor ideal nem sempre pode ser construído sobre plataformas de hardware existentes
- Possibilidade de construção de hypervisor em determinada plataforma definida pelo **teorema de Popek e Goldberg [1974]**
 - “Para qualquer computador convencional de terceira geração, um hypervisor pode ser construído se o conjunto de instruções sensíveis daquele computador for um subconjunto de seu conjunto de instruções privilegiadas.”
- Definições
 - **Instruções sensíveis:** podem consultar ou alterar estado do processador, p.ex., registradores que armazenam status da execução
 - **Instruções privilegiadas:** acessíveis somente por código em modo privilegiado
 - Quando executadas por código não privilegiado devem gerar uma interrupção a ser tratada

Construção de máquinas virtuais

- Implicações do teorema de Popek e Goldberg
 - Toda instrução sensível deve ser também privilegiada
 - Quando instrução sensível é executada por programa não privilegiado (p.ex. kernel convidado), provocará uma interrupção
 - Interrupção pode ser usada para ativar rotina de interpretação dentro do hypervisor, que irá simular o efeito da instrução sensível
 - Maior número de instruções sensíveis, leva a maior o volume de interpretação de código
 - Pior será o desempenho da máquina virtual
- Alternativa em sistemas que não satisfazem o teorema
 - Paravirtualização: reescrever parte do sistema convidado para não usar essas instruções sensíveis

Suporte de hardware

- Suporte de HW para hypervisors deficiente por muito tempo
 - Intel Pentium IV (e anteriores) possuía 17 instruções sensíveis que podiam ser executadas em modo usuário (violando teorema)
 - **SGDT/SLDT**: ler registrador com posição/tamanho de tabelas de segmentos
 - **SMSW**: ler registrador com status do processador
 - **PUSHF/POPF**: empilha/desempilha valor do registrador com status do processador
 - Técnicas iniciais de virtualização para Intel/AMD se baseavam em redução de privilégios do SO convidado (anéis de proteção)
 - Hypervisor e SO hospedeiro executam no nível 0
 - SO convidado executa no nível 1 ou 2
 - Aplicações do sistema convidado executam no nível 3
 - Algumas instruções do SO convidado precisavam ser reescritas (**paravirtualização!**) dinamicamente no momento da carga
 - Pois SO foi originalmente construído para executar no nível 0

Suporte de hardware

- Por volta de 2005, fabricantes de microprocessadores (Intel e AMD) incorporaram suporte básico à virtualização
 - IVT (Intel Virtualization Technology) e AMD-V (AMD Virtualization)
- Ideia central de ambas as tecnologias
 - Definir dois modos possíveis de operação do processador
 - **Modo root:** equivale ao funcionamento de processador convencional, destinado à execução de um hypervisor
 - **Modo non-root:** se destina à execução de máquinas virtuais
 - Ambos os modos suportam os quatro níveis de privilégio
 - Permite executar SOs convidados sem necessidade de reescrita dinâmica de código

Níveis de virtualização

- Virtualização implica na reescrita de interfaces de sistema e de acordo com a interface, há diferentes níveis de virtualização
 - **Virtualização do hardware:** virtualiza toda a interface ISA, que dá acesso ao hardware
 - **Virtualização da interface de sistema:** virtualiza a System ISA, que corresponde ao conjunto de instruções sensíveis do processador
 - **Virtualização de dispositivos de I/O:** virtualiza dispositivos físicos, como discos e interfaces de rede
 - **Virtualização do sistema operacional:** cada VM é como uma instância distinta do mesmo SO subjacente
 - **Virtualização de chamadas de sistema:** virtualiza chamadas de sistema de um sistema A para um sistema B
 - **Virtualização de chamadas de biblioteca:** virtualiza chamadas de biblioteca

Técnicas de virtualização

- Estratégias mais utilizadas atualmente para virtualização:
 - Emulação completa do hardware
 - Virtualização da interface de sistema
 - Tradução dinâmica de código
 - Paravirtualização

Técnicas de virtualização

- **Emulação completa do hardware**
 - Virtualiza toda a interface de hardware, podendo oferecer ao sistema convidado uma interface distinta da física
 - Incluindo instruções do processador, a memória e dispositivos
 - Realiza tradução dinâmica das instruções
 - Exemplo: VMs QEMU (oferecem processador x86 a sistema convidado)
- **Virtualização da interface de sistema**
 - Interface ISA de usuário é mantida, e apenas instruções privilegiadas e dispositivos (discos, interfaces de rede, etc.) são virtualizados
 - Instruções sensíveis devem gerar interrupções
 - Sistema convidado e aplicações enxergam o processador real
 - Melhor desempenho do SO convidado, próximo à execução em hardware real
 - Exemplos: VMware Workstation, VirtualBox e KVM

Técnicas de virtualização

- **Tradução dinâmica**

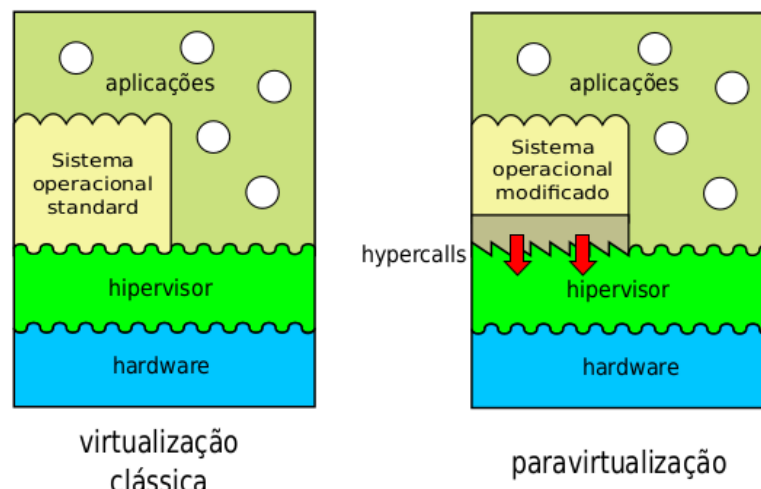
- Hypervisor analisa, reorganiza e traduz sequências de instruções realizadas pelo sistema convidado
- Conforme código é carregado em memória ou durante a execução
- Possíveis objetivos:
 - Adaptar instruções geradas pelo sistema convidado à interface ISA do sistema real
 - Detectar e analisar instruções sensíveis não-privilegiadas
 - Melhorar o desempenho da execução das instruções através de otimizações
- Exemplo: bytecodes Java executando em JVM são interpretados e traduzidos para interface do sistema real

Técnicas de virtualização

Fonte: Maziero, 2019

- **Paravirtualização**

- Modifica interface entre hypervisor e SOs convidados para fornecer hardware virtual similar ao real
- Hypervisor define API de chamadas de hypervisor (*hypercalls*) para que cada convidado acesse a interface de sistema de hardware virtual
 - Modificações na interface de sistema do hardware virtual (system ISA) exigem adaptação dos sistemas operacionais convidados
- Interface de usuário (user ISA) do hardware é preservada
 - Permite que aplicações convidadas executem sem necessidade de modificações
- Exemplo: Xen permite SOs convencionais como Linux e Windows (modificados) executar sobre um hypervisor



Migração de máquinas virtuais

- Permite transferir VM e seu sistema convidado através da rede de um hypervisor para outro, executando em equipamentos distintos, sem necessidade de reiniciá-la
 - **Stop-and-copy:** suspende a VM e transfere o *conteúdo de sua memória* para o hypervisor de destino e depois retoma sua execução
 - Simples, porém implica em parar completamente os serviços
 - **Demand-migration:** VM suspensa apenas durante a cópia das *estruturas de memória* do SO convidado para hypervisor de destino
 - Após, máquina virtual é retomada e restante das páginas de memória é transferido sob demanda
 - **Pre-copy:** copia para o hypervisor destino todas as páginas de memória da máquina virtual enquanto ela executa
 - VM é suspensa e as páginas modificadas durante o processo são novamente copiadas para o destino

Virtualização na prática

- Aplicações da virtualização
 - Construção de binários portáveis
 - Compartilhamento de hardware
 - Suportar aplicações legadas
 - Experimentação em redes
 - Ensino
 - Segurança
 - Desenvolvimento de baixo nível
 - Tolerância a falhas
 - Computação sob demanda
 - Virtualização de redes

Referências

- Maziero, Carlos – “Sistemas Operacionais: Conceitos e Mecanismos”, 2019
(<http://wiki.inf.ufpr.br/maziero/doku.php?id=socm:start>)
 - Capítulo 31
 - Capítulo 32
 - Capítulo 33
 - Capítulo 34
- Tanenbaum, A. Boss, H. – “Modern Operating Systems” (4th edition), Pearson, 2015.
 - Capítulo 7